

IFP - Innovación en Formación Profesional
Proyecto Intermodular de Desarrollo de Aplicaciones Multiplataforma.

Plataforma para la Gestión Integral de Centro de Día para Adultos Mayores

*Desarrollo de una solución multiplataforma con arquitectura Multi-tenant mediante
Spring Boot, Angular y Docker*

Autores

María Elena Miranda Villablanca
James Alexander Ovalle Barrera
Juan José Restrepo Arango

Tutor

José Manuel Rayo Ortigüella

RESUMEN

Al observar el funcionamiento diario de los centros de día para adultos mayores, es evidente que el personal se enfrenta a una falta de herramientas tecnológicas adaptadas a sus necesidades reales. La mayoría de las soluciones disponibles actualmente están pensadas para residencias tradicionales, lo que obliga a los trabajadores a lidiar con funciones irrelevantes que complican su labor diaria en lugar de ayudarles.

Para resolver este problema, se ha desarrollado una plataforma bajo el modelo SaaS que permite centralizar toda la operativa en una sola herramienta fácil de usar. Con esta aplicación, el trabajador puede gestionar de manera eficiente desde las fichas de salud y el control de medicación hasta la logística de las rutas de transporte y el seguimiento de incidencias. El objetivo fundamental es que la tecnología sea un apoyo intuitivo que permita al personal dedicar más tiempo y atención al bienestar de los usuarios.

En cuanto a la parte técnica, el sistema utiliza un backend desarrollado con Java 21 y Spring Boot 4. La interfaz es dinámica y está creada con Angular 21 para que la experiencia de uso sea natural y sencillo para el trabajador. Además, el proyecto destaca por una arquitectura multi-tenant que permite que los datos de cada centro estén siempre aislados y seguros. Todo el entorno se despliega mediante contenedores Docker para que el sistema sea escalable y muy sencillo de instalar en cualquier equipo.

ABSTRACT

When observing the daily operations of adult day care centers, it becomes evident that staff members face a significant lack of technological tools tailored to their actual needs. Most currently available solutions are designed for traditional nursing homes, forcing workers to navigate irrelevant features that complicate their daily tasks rather than assisting them.

To address this issue, a SaaS-based platform has been developed to centralize all operations into a single, user-friendly tool. Through this application, staff can efficiently manage everything from health records and medication control to transport logistics and incident tracking. The fundamental goal is for technology to serve as an intuitive support system, enabling personnel to dedicate more time and attention to the well-being of the users.

On the technical side, the system features a backend developed with Java 21 and Spring Boot 4. The interface is dynamic and built with Angular 21, providing a natural and straightforward user experience. Additionally, the project stands out for its multi-tenant architecture, which ensures that each center's data remains isolated and secure. The entire environment is deployed using Docker containers, making the system scalable and extremely easy to install on any device.

ÍNDICE

1. INTRODUCCIÓN Y OBJETIVOS.....	6
1.1. Introducción.....	6
1.2. Objetivos.....	7
1.2.1. Objetivo General.....	7
1.2.2. Objetivos Específicos.....	7
1.3. Justificación del Proyecto.....	8
1.4. Alcance del Proyecto.....	9
2. ANÁLISIS DEL SISTEMA.....	10
2.1. Estudio de Viabilidad.....	10
2.1.1. Viabilidad Técnica.....	10
2.1.2. Viabilidad Económica.....	10
2.2. Análisis de Requisitos.....	10
2.2.1. Requisitos Funcionales (RF).....	10
2.2.2. Requisitos No Funcionales (RNF).....	11
2.3. Casos de Uso y Reglas de Negocio.....	12
2.3.1. Caso de Uso: Control de Asistencia Diaria.....	12
2.3.2. Caso de Uso: Reporte de Incidencias.....	12
2.3.3. Reglas de Negocio (BR).....	13
3. DISEÑO Y ARQUITECTURA.....	13
3.1. Arquitectura de la Aplicación.....	13
3.1.1. Organización del Backend en Capas.....	13
3.1.2. Modelo SaaS: Sistema para varios centros.....	14
3.1.3. Contenerización.....	14
3.2. Diseño de la Base de Datos.....	14
3.2.1. Modelo Relacional y Relaciones.....	14
3.2.2. Estrategia de Borrado y Trazabilidad.....	14
3.3. Diseño de la Interfaz (UI/UX).....	15
3.3.1. Principios de Diseño.....	15
3.3.2. Justificación del Uso de Angular.....	15
3.3.3. Comparativa Técnica de Frameworks.....	16
3.4. Diagramas de Flujo y lógica de procesos.....	16
3.4.1. Proceso de Registro de Jornada del Trabajador.....	16
3.4.2. Control de Asistencia de los Usuarios.....	16
3.4.3. Planificación del transporte.....	17
3.4.4. Seguimiento de incidencias: Centro / Usuario.....	17
3.4.5. Alta de Usuarios.....	17
3.4.6. Seguridad y acceso mediante JWT.....	17
3.4.7. Representación gráfica de los procesos.....	18
4. METODOLOGÍA DE TRABAJO Y PLANIFICACIÓN.....	24
4.1. Marco metodológico: Colaboración y Especialización.....	24
4.2. Fase de investigación y análisis de campo.....	24
4.3. Diseño compartido: El modelo de datos - Database First.....	24
4.4. División de roles y responsabilidades.....	24

4.5. Apoyo mutuo y cohesión del equipo.....	25
4.6. Planificación temporal: Carta Gantt.....	25
5. DESARROLLO TÉCNICO.....	27
5.1. Etapa 2: Diseño de la Base de Datos y Desarrollo del Backend.....	27
5.1.1. Diseño de la Base de Datos.....	27
5.1.2. Modelos en Java.....	28
5.1.3. Auditing y BaseEntity.....	31
5.1.4. Repositorios.....	33
5.1.5. DTOs (Data Transfer Object).....	34
5.1.6. Capa de Mapeo (MapStruct Mappers).....	36
5.1.7. Capa de Servicios (Business Logic Layer).....	38
5.1.8. Capa de Controladores (API REST Controllers).....	40
5.1.9. Seguridad y Arquitectura de Aislamiento Multi-tenant.....	42
5.2. Etapa 3: Diseño de la Interfaz y Desarrollo del Frontend.....	48
5.2.1. Angular 21: Novedades y Enfoque.....	48
5.2.2. Arquitectura del Proyecto (SaaS multi-tenant).....	49
5.2.3. Flujo de Trabajo.....	50
5.2.4. Explicación Modelo.....	51
5.2.5. Explicación de un Servicio.....	53
5.2.6. Componente y su plantilla.....	54
5.2.7. Funcionamiento de JWT.....	56
5.2.8. Shared Components.....	58
5.2.9. Sistema de Rutas.....	60
5.2.10. Conclusión Desarrollo Angular.....	61
5.3. Etapa 4: Contenedorización con Docker y Despliegue del Sistema.....	62
5.3.1. Backend.....	62
5.3.2. Frontend.....	63
5.3.3. Orquestación.....	65
6. GUÍA DE FUNCIONAMIENTO DEL SISTEMA.....	67
6.1. Acceso al Sistema y Autenticación.....	67
6.1.1. Página de Inicio (Landing Page).....	67
6.1.2. Acceso y Validación de Credenciales.....	68
6.2. Panel de Control y Navegación (Dashboard).....	69
6.3. Portal del Trabajador.....	70
6.3.1. Gestión de Actividades (Ampliación MVP).....	70
6.3.2. Gestión de Usuarios.....	71
6.3.3. Control de Asistencia Diaria.....	76
6.3.4. Gestión de Pertenencias (Ampliación MVP).....	77
6.3.5. Gestión de Incidencias.....	77
6.3.6. Gestión de Higiene y Turnos de Baño.....	78
6.3.7. Gestión de Transporte.....	79
6.3.8. Gestión de los Trabajadores.....	80
6.3.9. Gestión de Configuraciones (Catálogos).....	81
6.4. Portal de Administración.....	82

6.4.1. Gestión de Clientes.....	82
6.4.2. Planes de Suscripción.....	82
6.4.3. Administradores del Sistema.....	83
6.4.4. Administradores del Sistema.....	83
7. CONCLUSIONES Y LÍNEAS FUTURAS.....	84
7.1. Conclusiones Generales.....	84
7.2. Líneas de Trabajo Futuro.....	84
7.3. Experiencia de Aprendizaje Individual.....	85
7.3.1. María Elena Miranda Villablanca.....	85
7.3.2. James Alexander Ovalle Barrera.....	86
7.3.3. Juan José Restrepo Arango.....	87
8. REFERENCIAS BIBLIOGRÁFICAS.....	88

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Introducción

Este proyecto se centra en el desarrollo de una aplicación SaaS (Software as a Service), diseñada específicamente para la administración integral de centros de día para adultos mayores. Su propósito es optimizar la gestión operativa diaria de estos centros, facilitando el trabajo del personal y mejorando la calidad de la atención ofrecida a los usuarios.

Actualmente existe un problema real en el sector, y es que los centros de día carecen de herramientas tecnológicas hechas a su medida. El personal se ve en la obligación de adaptar sus procesos a programas informáticos complejos que fueron diseñados para residencias de mayores tradicionales, lo que termina entorpeciendo enormemente sus flujos de trabajo diarios en lugar de facilitarlos. Al tener a su disposición módulos irrelevantes para su contexto, como número de camas disponibles o la gestión de turnos de noche, sumado a una interfaz poco amigable, su labor se complejiza enormemente, haciendo que pierdan tiempo valioso en lo que realmente importa: la atención y el bienestar de los usuarios.

Ante esta necesidad, y a grandes rasgos, el sistema busca unificar en una plataforma las áreas operativas del centro. Sus características principales abarcan desde la gestión de perfiles detallados de cada usuario, que recogen la situación personal y las necesidades que poseen para su cuidado diario, y los perfiles de trabajador para cada uno de los funcionarios del centro con sus roles específicos y funcionalidades, hasta la gestión de incidencias, tratamientos, medicación, gestión de transporte y registro de asistencia.

La elección de un modelo SaaS para implementar este proyecto es estratégica y fundamental. En la actualidad, la creación de una aplicación personalizada para un único centro de día es ineficiente, ya que existen múltiples centros con necesidades operativas y administrativas similares. El enfoque SaaS no sólo permite ofrecer una solución estandarizada y robusta a una amplia base de clientes, sino que también abre la puerta a un planteamiento de expansión significativo. Al desarrollar una plataforma escalable y multi-tenant, facilita la adopción de este por parte de diversos centros de día, maximizando el impacto y la viabilidad del proyecto a largo plazo.

En esencia, esta aplicación ofrecerá una solución tecnológica adaptada a las necesidades específicas de los centros de día, con una interfaz cómoda para sus empleados. Esto superará las limitaciones actuales, proporcionando un entorno digital que les permitirá trabajar con mayor eficiencia, a la vez que garantizará la seguridad y el bienestar de los adultos mayores.

1.2. Objetivos

1.2.1. Objetivo General

Desarrollar una plataforma web escalable bajo el modelo SaaS para la administración integral de centros de día para adultos mayores, que centralice la gestión operativa y mejore la calidad de la atención ofrecida a los usuarios, mediante un entorno digital adaptado a las necesidades específicas del sector.

1.2.2. Objetivos Específicos

- Implementar una base de datos relacional con PostgreSQL 18 bajo una arquitectura multi tenant para asegurar la persistencia, el aislamiento lógico y la disponibilidad de información de cada centro de día.
- Construir la lógica de negocio y la arquitectura del lado del servidor mediante Java 21 y Spring Boot 4, estableciendo una API RESTful sólida para procesar toda la gestión operativa del centro bajo un diseño de capas.
- Optimizar el código fuente y la transformación interna de datos mediante procesadores de anotaciones y herramientas como MapStruct y Lombok, para agilizar el proceso de desarrollo y la limpieza de la arquitectura.
- Integrar un sistema de seguridad y control de acceso estricto basado en roles y tokens (JSON Web Token) para resguardar la privacidad de los datos sensibles y restringir las funciones según el perfil del trabajador.
- Garantizar el seguimiento de las acciones realizadas mediante un sistema de auditoría automática que identifique al trabajador responsable y la temporalidad de los cambios efectuados en todos los registros y módulos de la aplicación.
- Desarrollar la interfaz de usuario de forma desacoplada utilizando Angular 21, para ofrecer al trabajador una herramienta visualmente amigable, rápida e intuitiva que facilite su labor diaria.
- Crear interfaces intuitivas para los diferentes módulos operativos de la aplicación, facilitando la interacción del trabajador con las funcionalidades específicas de gestión del centro de día.
- Implementar un diseño responsivo que asegure el acceso y la visualización de la plataforma desde diferentes dispositivos.
- Desplegar la aplicación y su entorno de base de datos a través de Docker para asegurar su portabilidad, mantenimiento y escalabilidad futura del sistema.

1.3. Justificación del Proyecto

Este proyecto surge de un diálogo productivo con las fundadoras del centro de día El Bosque, sumado a la experiencia directa de uno de los miembros del equipo que trabajó en dicho centro. Esta interacción y conocimiento de primera mano revelaron una carencia significativa en el sector tecnológico actual: la falta de soluciones de software diseñadas específicamente para las particularidades de los centros de día.

Una investigación exhaustiva y el análisis de mercado confirmaron que la mayoría de las herramientas digitales disponibles están orientadas a las operaciones de las residencias de mayores. Esta orientación genera inconvenientes sustanciales al intentar implementarlas en un centro de día. Las interfaces de estas aplicaciones suelen estar sobrecargadas con funcionalidades, menús y módulos que, aunque pertinentes para una residencia (como la gestión de camas, la planificación de turnos nocturnos o la administración de medicamentos en horario extendido), resultan redundantes o inaplicables en el entorno de un centro de día. Como resultado, el personal se ve obligado a navegar por sistemas complejos, ignorando gran parte de sus opciones y adaptando sus flujos de trabajo a las limitaciones impuestas por el software, en lugar de que este se adapte a sus necesidades operativas.

Esta situación no solo genera ineficiencia y frustración para los empleados de los centros de día, sino que también impacta negativamente en la experiencia de usuario. La estética y la usabilidad de estas plataformas a menudo no son óptimas, lo que desincentiva su uso y aumenta la curva de aprendizaje para el personal. La falta de una interfaz intuitiva y visualmente atractiva se traduce en una menor adopción del software y en una resistencia a la digitalización de procesos que, de otro modo, podrían ser muy beneficiosos.

Esta problemática, claramente identificada y percibida por los profesionales del sector, evidencia la necesidad de desarrollar una aplicación a medida. Esta iniciativa busca construir una herramienta desde cero, pensada y diseñada para abordar de forma precisa y eficiente las necesidades únicas de los centros de día. El objetivo principal es ofrecer una solución tecnológica que no solo optimice la gestión operativa, sino que también mejore significativamente la calidad del servicio ofrecido a los usuarios y facilite el trabajo diario del personal, eliminando la frustración asociada a las herramientas genéricas y proporcionando un entorno digital amigable y eficaz.

1.4. Alcance del Proyecto

Esta herramienta permitirá gestionar y facilitar el trabajo diario del centro de forma ágil y coordinada, facilitando la administración de los siguientes aspectos:

- **Gestión del Equipo y Personal:** Centralizará la información del personal del centro para organizar mejor las tareas diarias. La plataforma permitirá la asignación de roles, turnos y responsabilidades, facilitando la comunicación interna y el registro de incidencias laborales. Esto asegurará que cada profesional tenga acceso a la información necesaria para cumplir con su trabajo.
- **Gestión de Usuarios:** Registrará y administrará fichas personales, sociales y médicas detalladas para cada usuario. Esto incluirá tener información sobre sus necesidades personales como alergias, medicación o necesidades de asistencia especial, como el uso de oxígeno.
- **Asistencia, Seguimiento y Gestión de Incidencias:** Registrará la asistencia diaria de los usuarios para realizar un seguimiento individualizado. Además, se documentarán todas las incidencias, desde cambios en el estado de ánimo o salud, hasta necesidades especiales detectadas. Esta información es fundamental para la toma de decisiones del equipo, la comunicación con las familias y la adaptación de los planes de apoyo. También se registrarán las incidencias administrativas del Centro de Día, como limpieza, control de insumos y las relativas al personal.
- **Tratamientos y Control de Medicación:** Permitirá supervisar los tratamientos específicos y la administración de fármacos de cada usuario. El sistema controlará las pautas, horarios y vías de administración. También se registrarán enfermedades y alergias de los adultos mayores para evitar riesgos asistenciales. Este módulo incluirá la gestión del inventario de medicación, verificando fechas de caducidad y condiciones de almacenamiento. Cualquier incidencia, reacción adversa o ajuste en el tratamiento quedará registrado y se comunicará de inmediato a los profesionales sanitarios pertinentes mediante la plataforma, para que quede constancia de cada paso que se ha dado en su cuidado.
- **Logística y Gestión de Transporte:** Gestionará el servicio de transporte, enfocado a las rutas de recogida y entrega de los usuarios, registrando los horarios, los vehículos asignados y los detalles de cada itinerario, permitiendo que se cumplan las medidas de seguridad y comodidad necesarias en cada trayecto. Además, se mantendrá un historial de los recorridos y se registrará cualquier imprevisto que ocurra durante el camino, conectando directamente con el sistema de incidencias para que el centro tenga el control de lo que sucede incluso fuera de sus instalaciones.
- **Gestión de Higiene y Saneamiento:** Establecerá y documentará los estándares de higiene personal de los usuarios, incluyendo su limpieza y presentación. Esto implica la definición de horarios y procedimientos detallados de saneamiento, tales como momentos específicos para ir al baño, el cepillado de dientes y el cambio de pañales, asegurando así un control ordenado en el Centro de Día.

2. ANÁLISIS DEL SISTEMA

En esta fase se definen los pilares sobre los que se asienta el proyecto, identificando las necesidades que resuelve, a quién va dirigido y cuáles son las reglas técnicas que rigen su funcionamiento.

2.1. Estudio de Viabilidad

2.1.1. Viabilidad Técnica

El proyecto es técnicamente viable gracias al uso de un stack robusto y moderno:

- **Backend:** El uso de Java 21 junto con Spring Boot permite una gestión eficiente de la lógica de negocio y la seguridad.
- **Base de Datos:** Se ha seleccionado PostgreSQL 18, que garantiza la integridad de datos sensibles, como la salud o la información personal de los usuarios, mediante relaciones complejas y un sistema de auditoría.
- **Contenerización:** El uso de Docker permite que el entorno de desarrollo y el de producción sean idénticos. Gracias a esto, eliminamos los problemas de compatibilidad entre equipos, facilitando el despliegue y evitando errores de configuración.

2.1.2. Viabilidad Económica

Desde el punto de vista financiero, el proyecto destaca por su alta rentabilidad inicial. Al apoyarse por completo en tecnologías de código abierto y herramientas libres, hemos logrado que los costes de licencias sean inexistentes. Esto permite que la inversión principal se traduzca exclusivamente en las horas dedicadas al desarrollo, dejando para una fase posterior únicamente los gastos derivados del alojamiento en la nube de la base de datos, la API y la interfaz de usuario.

2.2. Análisis de Requisitos

Esta sección detalla las capacidades que debe tener el sistema para cumplir con su propósito, dividiéndose en las funciones directas que realizará el usuario y las características técnicas que aseguran su calidad.

2.2.1. Requisitos Funcionales (RF)

Son las acciones específicas que el sistema debe realizar para automatizar y facilitar el día a día en el centro:

- **RF-01 Administración Global del SaaS (Super Admin):** El sistema debe contar con un módulo de gestión de alto nivel que permita a los administradores del software dar de alta nuevos clientes (centros de día), gestionar sus suscripciones y aprovisionar automáticamente sus entornos de trabajo independientes.
- **RF-02 Gestión del Personal y Roles:** La plataforma debe permitir el alta de trabajadores y la asignación de roles específicos, como Administrador del centro, Cuidador, Enfermero, Conductor, lo cual determinará a qué menús y funciones tienen acceso dentro de la aplicación.
- **RF-03 Gestión de Jornada:** El sistema debe permitir a los trabajadores registrar su entrada y salida (fichaje). La aplicación debe calcular automáticamente el tiempo trabajado y detectar retrasos.

- **RF-04 Gestión de Usuarios del Centro (Usuarios):** El sistema debe permitir registrar y administrar fichas personales, sociales y médicas detalladas para cada usuario. Esto incluye necesidades como alergias, medicaciones, tratamientos y asistencia especial.
- **RF-05 Control de Asistencia e Higiene:** El sistema registrará la asistencia diaria en el centro y permitirá establecer y documentar los procedimientos de higiene personal de los usuarios (idas al baño, cambio de pañales).
- **RF-06 Administración y Control de Medicación:** La aplicación debe permitir el control de la administración de medicamentos, registrando horarios y vías de administración, así como la gestión del inventario de medicinas (caducidad y almacenamiento).
- **RF-07 Gestión de Incidencias:** El sistema debe permitir documentar incidencias, separando las incidencias operativas del centro (limpieza, insumos) de las incidencias asistenciales directas de los usuarios.
- **RF-08 Gestión de Logística y Transporte:** El software debe gestionar las rutas de recogida y entrega, asignando vehículos, conductores y copilotos. Debe contar con una vista de hoja de ruta digital interactiva para registrar la asistencia y ausencias durante el trayecto.
- **RF-09 Panel de Inicio (Dashboard):** El sistema debe proveer un panel de control principal que muestre un listado de incidencias para facilitar el acceso a la información más relevante del día a día.

2.2.2. Requisitos No Funcionales (RNF)

Definen las características de calidad y seguridad que hacen que la aplicación sea confiable y fácil de usar:

- **RNF-01 Arquitectura Multi-tenant (SaaS):** El sistema utilizará una única base de datos PostgreSQL centralizada, implementando una separación lógica de los datos mediante esquemas independientes para cada centro de día cliente, garantizando el aislamiento de la información.
- **RNF-02 Seguridad y Control de Acceso Basado en Roles (RBAC):** La seguridad, autenticación y autorización se gestionarán mediante Spring Security y tokens JWT (JSON Web Tokens). Se implementará un control de acceso estricto a nivel de método (utilizando anotaciones como @PreAuthorize) para asegurar que cada trabajador sólo pueda interactuar con los recursos permitidos para su rol.
- **RNF-03 Trazabilidad y Auditoría de Datos:** El sistema debe asegurar la trazabilidad de todos los eventos transaccionales. Las entidades de la base de datos heredarán de una clase abstracta base que registre de forma automática las fechas de creación, modificación y el identificador del usuario responsable de la acción, utilizando Spring Data JPA Auditing.
- **RNF-04 - Pila Tecnológica del Backend:** El servidor se desarrollará en Java 21 utilizando Spring Boot 4, siguiendo una arquitectura de diseño en capas (Controladores, Servicios y Repositorios) para separar de forma clara el enrutamiento web, la lógica de negocio y el acceso a datos.
- **RNF-05 - Arquitectura Frontend Desacoplada:** La interfaz de usuario cliente se construirá exclusivamente utilizando el framework Angular 21. La comunicación entre el frontend y el backend

se realizará de manera asíncrona consumiendo una API RESTful mediante el intercambio de datos en formato JSON.

- **RNF-06 Manejo Global de Excepciones:** Se implementará un mecanismo de manejo de excepciones global (ej. @ControllerAdvice) que proporcione respuestas de error estandarizadas al cliente Angular durante el desarrollo, pero que filtre los mensajes en producción para no exponer detalles sensibles del servidor o la base de datos.
- **RNF-07 - Validación de Datos:** Se implementarán validaciones declarativas estrictas (JSR-303/JSR-349 Bean Validation) en los datos de entrada al servidor (ej. @NotNull, @Size) para proteger la integridad de la base de datos médica y operativa.
- **RNF-08 - Portabilidad y Despliegue:** La base de datos y la aplicación deben estar preparadas para ejecutarse en entornos contenedorizados utilizando Docker.

2.3. Casos de Uso y Reglas de Negocio

Para entender cómo interactúan los usuarios con el sistema, se han definido los procesos y las normas que rigen el comportamiento de la aplicación. Es importante destacar que el sistema se adapta a los diferentes perfiles del centro, permitiendo realizar tareas comunes y específicas de forma coordinada:

2.3.1. Caso de Uso: Control de Asistencia Diaria

Cuando los usuarios llegan al centro, el trabajador responsable accede al listado general y marca el estado de presencia de cada uno. Este proceso requiere que el trabajador se haya identificado previamente en el sistema.

- **Actor:** Trabajador (Fisio, TAS, Admin).
- **Descripción:** El trabajador accede al listado de los usuarios y marca el estado de presencia.
- **Precondición:** El trabajador debe estar autenticado.

2.3.2. Caso de Uso: Reporte de Incidencias

Ante una situación inusual, cualquier empleado puede redactar una incidencia. El sistema lo guarda vinculándolo a su firma digital de forma automática para la auditoría. Para cerrar una incidencia, un administrador debe verificar primero que la situación ha sido resuelta.

- **Actor:** Cualquier trabajador.
- **Flujo:** El trabajador detecta una situación anómala (médica o logística), redacta una incidencia y el sistema lo guarda con su firma digital (auditoría).
- **Resolución:** Sólo un Administrador puede cambiar el estado de la incidencia a "Cerrada" tras verificar la solución.

2.3.3. Reglas de Negocio (BR)

Son las normas lógicas que el software debe obligar a cumplir para mantener la integridad del servicio:

- **BR-01 Inmutabilidad de Registros:** Un trabajador no puede modificar su propio registro de horas una vez que el turno ha finalizado. Cualquier corrección necesaria debe ser revisada y autorizada por un Administrador.
- **BR-02 Asignación Jerárquica:** Los horarios laborales son estables y sólo pueden ser modificados por perfiles que tengan asignado un rol administrativo.
- **BR-03 Trazabilidad Obligatoria:** No se permite realizar cambios anónimos en el sistema. Toda creación o modificación de información debe quedar vinculada al usuario que realizó la acción a través del sistema de auditoría.

3. DISEÑO Y ARQUITECTURA

Este capítulo detalla la estructura técnica y lógica sobre la cual se ha construido la plataforma. Se describe la organización del código, el modelado detallado de los datos y las decisiones de diseño que aseguran una experiencia de usuario eficiente para el personal del centro.

3.1. Arquitectura de la Aplicación

El sistema se ha diseñado bajo un modelo de estilo cliente-servidor. Esta decisión permite separar por completo la interfaz de usuario de la lógica de negocio, facilitando que ambas partes puedan evolucionar o recibir mantenimiento de forma independiente sin afectarse entre sí.

3.1.1. Organización del Backend en Capas

Para el desarrollo del servidor con Spring Boot, se ha implementado una arquitectura organizada en capas. Esta estructura es fundamental para mantener el orden y asegurar que cada parte del código tenga una responsabilidad única:

- **Capa de Controlador (API REST):** Gestiona las peticiones HTTP que llegan desde el navegador y define los puntos de acceso de la API (endpoints). Además, está preparada para generar documentación técnica automática mediante Swagger/OpenAPI. No solo eso, también se documentará en una colección de Postman.
- **Capa de Servicio:** Aquí reside la lógica de negocio y las validaciones. como la comprobación de horarios y retrasos al fichar.
- **Capa de Acceso a Datos (Repositorio):** Utiliza Spring Data JPA para gestionar la comunicación con la base de datos de forma segura y eficiente.
- **Capa de Modelo (Entidades):** Representa la estructura de las tablas de la base de datos y define cómo se relacionan entre sí mediante sus relaciones.
- **Capa de Transferencia de datos (DTO y Mappers):** En esta capa se utilizan herramientas como MapStruct y Lombok para transformar entidades en objetos de transferencia, denominados DTOs. Esto asegura que solo la información necesaria viaje hacia el frontend, protegiendo la estructura interna y la seguridad del servidor.

3.1.2. Modelo SaaS: Sistema para varios centros

La plataforma se concibe bajo un modelo de Software as a Service (SaaS). La idea es que diferentes centros de día puedan usar la misma aplicación pero de forma totalmente independiente. Para que no haya cruces de datos, se utilizó una arquitectura multi-tenant con aislamiento lógico. En PostgreSQL, separamos la información de cada centro mediante identificadores únicos; así, un trabajador de un centro jamás podrá ver los expedientes o rutas de otro. Es fundamental para garantizar la privacidad que un entorno sociosanitario exige.

3.1.3. Contenerización

Para evitar errores de configuración y que el sistema sea fácil de mover, se utilizó Docker. En lugar de instalar todo a mano cada vez, se empaquetan el backend, el frontend y la base de datos en contenedores. Con Docker Compose hacemos que todas estas piezas se entiendan y arranquen juntas de una sola vez. Esto otorga seguridad, ya que la aplicación funciona en diferentes entornos de desarrollo.

La contenerización se desarrolla a detalle en el apartado *5.3 Etapa 4: Contenedorización con Docker y Despliegue del sistema*. Allí explicamos los pasos realizados para el backend, el frontend y la orquestación final.

3.2. Diseño de la Base de Datos

La persistencia de la información se gestiona en PostgreSQL, utilizando un modelo relacional complejo, diseñado para cubrir la complejidad de todas las necesidades asistenciales del centro.

3.2.1. Modelo Relacional y Relaciones

El diseño soporta diferentes tipos de vínculos según la naturaleza de la información:

- **One-to-Many / Many-to-One:** Permiten vincular de forma lógica toda la actividad del centro. Por ejemplo, un único usuario puede tener asociados múltiples registros de asistencia o diversos contactos de emergencia a lo largo del tiempo. Del mismo modo, un trabajador puede gestionar diversas pautas de medicación o participar en múltiples rutas de transporte.
- **Many-to-Many:** Se utilizan en apartados específicos como la gestión de permisos, permitiendo que un trabajador pueda desempeñar diferentes funciones o roles dentro de la plataforma según las necesidades del centro.
- **Catálogos Dinámicos:** Se han implementado tablas para datos maestros como provincias, tipos de dependencia o sexos. Aunque el sistema ya cuenta con registros base, estas tablas permiten una gestión flexible para añadir nuevos valores en el futuro si la normativa o el centro lo requieren.

3.2.2. Estrategia de Borrado y Trazabilidad

Para garantizar la integridad histórica de los datos de los usuarios el sistema aplica la técnica de Soft Delete. Mediante el uso de un indicador de actividad, la plataforma permite ocultar registros de la vista del usuario sin llegar a eliminarlos físicamente de la base de datos. Esta decisión es fundamental en entornos asistenciales para garantizar que la integridad histórica de los cuidados siempre esté disponible para futuras consultas o auditorías.

3.3. Diseño de la Interfaz (UI/UX)

La interfaz se ha desarrollado con Angular 21, buscando siempre la velocidad de respuesta y la facilidad de uso para los trabajadores del centro.

3.3.1. Principios de Diseño

La interfaz no solo busca ser estética, sino responder a las necesidades operativas:

- **Estética y Rendimiento:** Se utiliza Tailwind CSS v4 nativo para el estilizado. Esto permite un diseño moderno y altamente personalizado sin la sobrecarga que generan otros frameworks de componentes, garantizando que la aplicación cargue de forma instantánea.
- **Optimización del Flujo de Trabajo:** El Dashboard principal ofrece una visión directa de las incidencias actuales tras el login. Además, se ha integrado un sistema de fichaje global en la cabecera, permitiendo que el trabajador registre su jornada desde cualquier sección de la app de forma rápida.
- **Adaptabilidad:** Aunque el uso principal es en escritorio para tareas administrativas, la estructura es Responsive. Esto asegura que, en el futuro, el personal pueda utilizar tablets para registrar datos mientras se desplaza por el centro.

3.3.2. Justificación del Uso de Angular

La elección de Angular como framework frontend para este proyecto responde a requisitos técnicos y arquitectónicos específicos de un sistema SaaS multi-tenant.

- **Tipado Nativo con TypeScript:** Angular está construido sobre **TypeScript**, lo que garantiza tipado estático en toda la aplicación. En un proyecto con más de 50 modelos de dominio y comunicación constante con un backend Java, el tipado estricto reduce errores en tiempo de compilación y documenta implícitamente los contratos de API.
- **Inyección de Dependencias Nativa:** A diferencia de otros frameworks donde la inyección de dependencias requiere librerías externas, Angular la incluye de forma nativa. Esto facilita la creación de **servicios singleton**, la gestión de estado y la implementación de interceptores HTTP sin acoplamiento innecesario.
- **Arquitectura Modular y Escalable:** La separación en módulos funcionales (pages, services, models, shared) y el soporte nativo para **lazy loading** permiten escalar la aplicación horizontalmente. Cada nueva funcionalidad (transporte, incidencias, medicación) se puede desarrollar, probar y desplegar de forma aislada.
- **Ecosistema Completo:** Angular proporciona soluciones oficiales para **routing**, **formularios**, **peticiones HTTP** y **testing**. Esto evita la fragmentación de dependencias que ocurre en otros ecosistemas, donde cada desarrollador puede optar por librerías distintas para resolver el mismo problema.

3.3.3. Comparativa Técnica de Frameworks

Para validar esta elección frente a otras alternativas, se ha realizado un análisis comparativo centrado en la robustez necesaria para el proyecto:

Característica	Angular 21	React 19	Vue 3
Tipado	Nativo (TS)	Opcional	Opcional
DI	Nativo	Externa	Externa
Router	Oficial	Externo	Oficial
Formularios	Oficial	Externo	Oficial
Estado reactivo	Signals	Signals	Reactivity

3.4. Diagramas de Flujo y lógica de procesos

En este apartado se describen los procesos lógicos que permiten que la aplicación responda a las necesidades reales y al funcionamiento diario del Centro de Día. El diseño de estos flujos busca que las acciones más importantes del personal se realicen de forma sencilla y segura, garantizando que siempre quede un registro claro de lo que ocurre en cada jornada mediante validaciones en el backend.

3.4.1. Proceso de Registro de Jornada del Trabajador

Para asegurar la integridad del registro de jornada, el backend ejecuta una lógica de validación en tres niveles. Primero, se realiza una verificación de calendario para confirmar que el centro está operativo ese día. Posteriormente, se procede al cálculo de puntualidad, donde el sistema compara la hora del servidor con el horario asignado en **WorkerSchedule** para detectar posibles incidencias. Finalmente, si estas comprobaciones son satisfactorias, se genera un registro auditado en **WorkerScheduleRecord** que queda vinculado de forma automática al perfil del trabajador, garantizando que la información sea exacta sin necesidad de intervención manual (ver Figura 3.1)

3.4.2. Control de Asistencia de los Usuarios

El registro de asistencia de los usuarios se ha diseñado para evitar errores al momento de ingresar la información en el sistema. En lugar de simplemente guardar el dato, el servidor hace una comprobación para asegurarse de que el día que estamos marcando realmente le corresponde a ese usuario en su planificación previa en **UserAttendanceDay**. Esto funciona como un filtro de seguridad que evita que, por un descuido o un error al hacer clic, se termine registrando la asistencia de una persona en la ficha de otra. Finalmente, el backend revisa que no exista ya un registro para esa misma fecha, lo que nos permite tener el historial limpio y sin duplicados. (ver Figura 3.2)

3.4.3. Planificación del transporte

La gestión logística de las rutas se ha automatizado para reducir la carga administrativa y optimizar el uso de los vehículos del centro. El proceso vincula a los usuarios con los conductores y vehículos disponibles en **RouteVehicle**, organizando de forma independiente los trayectos de recogida y de regreso. El backend supervisa en todo momento que no se supere la capacidad de plazas de cada vehículo, generando automáticamente la planificación que servirá de guía para organizar los traslados de forma ordenada y puntual (ver Figura 3.3).

3.4.4. Seguimiento de incidencias: Centro / Usuario

Este módulo centraliza cualquier evento importante, separando claramente las incidencias que afectan a la operativa del centro (mantenimiento, suministros) de las que afectan directamente al bienestar del usuario (salud, conducta o medicación). El flujo permite anotar el suceso, categorizar su importancia y vincularlo al área o persona afectada. Un punto clave es que el sistema identifica automáticamente al trabajador que realiza el registro, esto garantiza que siempre se sepa quién introdujo la incidencia y en qué momento exacto se creó el reporte, facilitando la comunicación y el seguimiento entre los diferentes turnos (ver Figura 3.4).

3.4.5. Alta de Usuarios

El alta de un nuevo usuario es el paso previo para cualquier gestión asistencial y requiere validar información médica y personal antes de su ingreso. El sistema guía al administrador para asegurar que no existan expedientes duplicados mediante el DNI y que toda la documentación necesaria esté vinculada antes de activar el perfil. Esto garantiza que el personal cuente con una ficha de usuario completa y fiable desde el primer momento en que este comienza a asistir al centro (ver Figura 3.5).

3.4.6. Seguridad y acceso mediante JWT

Para proteger la privacidad de los datos de los usuarios, se ha implementado un sistema de seguridad basado en tokens JWT mediante JSON Web Token. El proceso comienza cifrando las credenciales de acceso mediante el algoritmo BCrypt, asegurando que las contraseñas nunca se almacenen de forma vulnerable en la base de datos. Una vez validada la identidad del trabajador, el servidor genera una llave digital firmada que le permite navegar por la aplicación y acceder a las funciones permitidas según su rol, sin tener que identificarse de nuevo en cada petición. Este flujo garantiza que el acceso a la información sensible sea siempre controlada y transparente para el personal (ver Figura 3.5).

3.4.7. Representación gráfica de los procesos

A continuación se presentan los diagramas de flujo que ilustran visualmente la lógica y el funcionamiento de los módulos descritos anteriormente en el apartado 3.4.

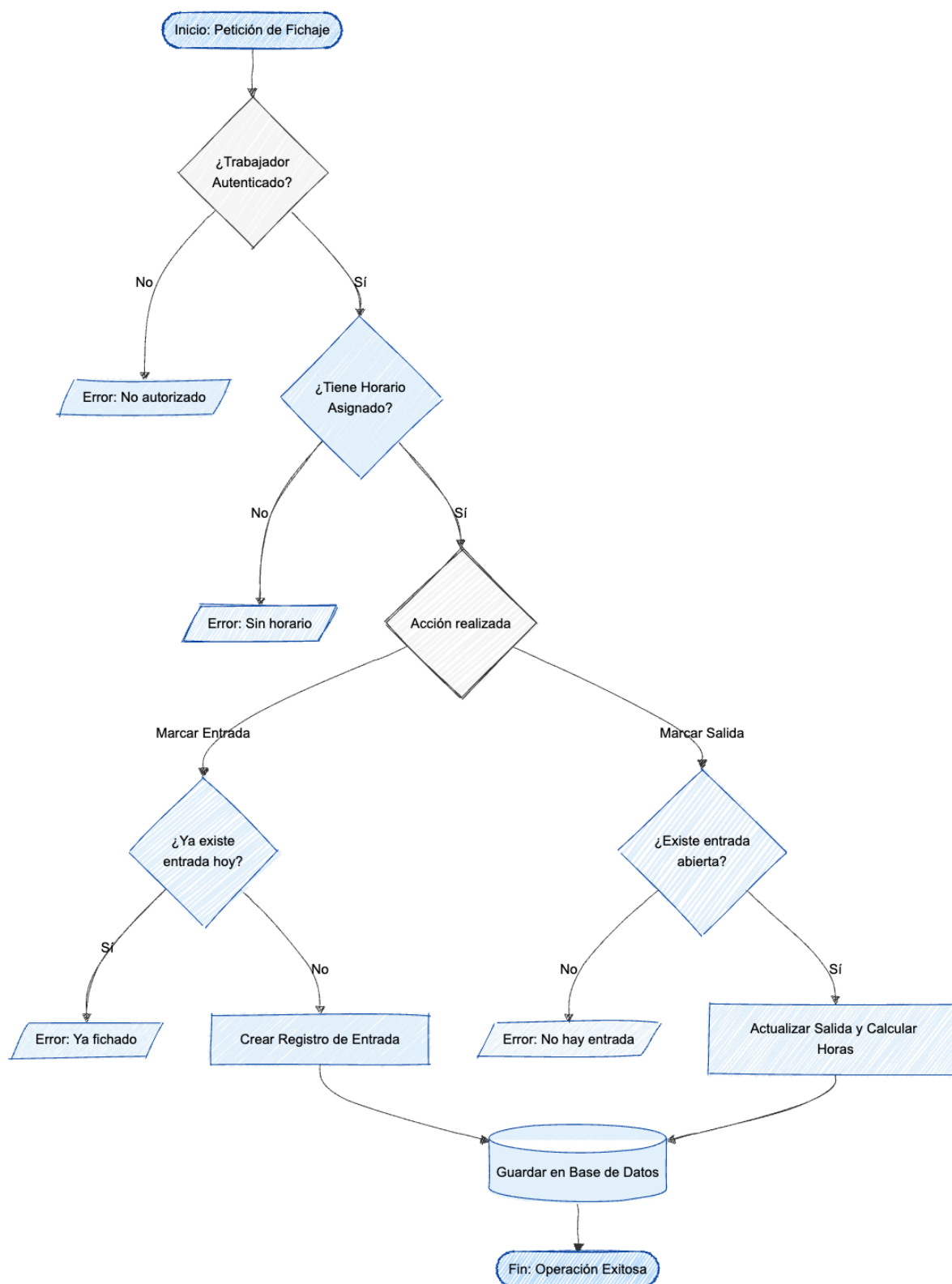


Figura 3.1: Flujo de registro de jornada del Trabajador

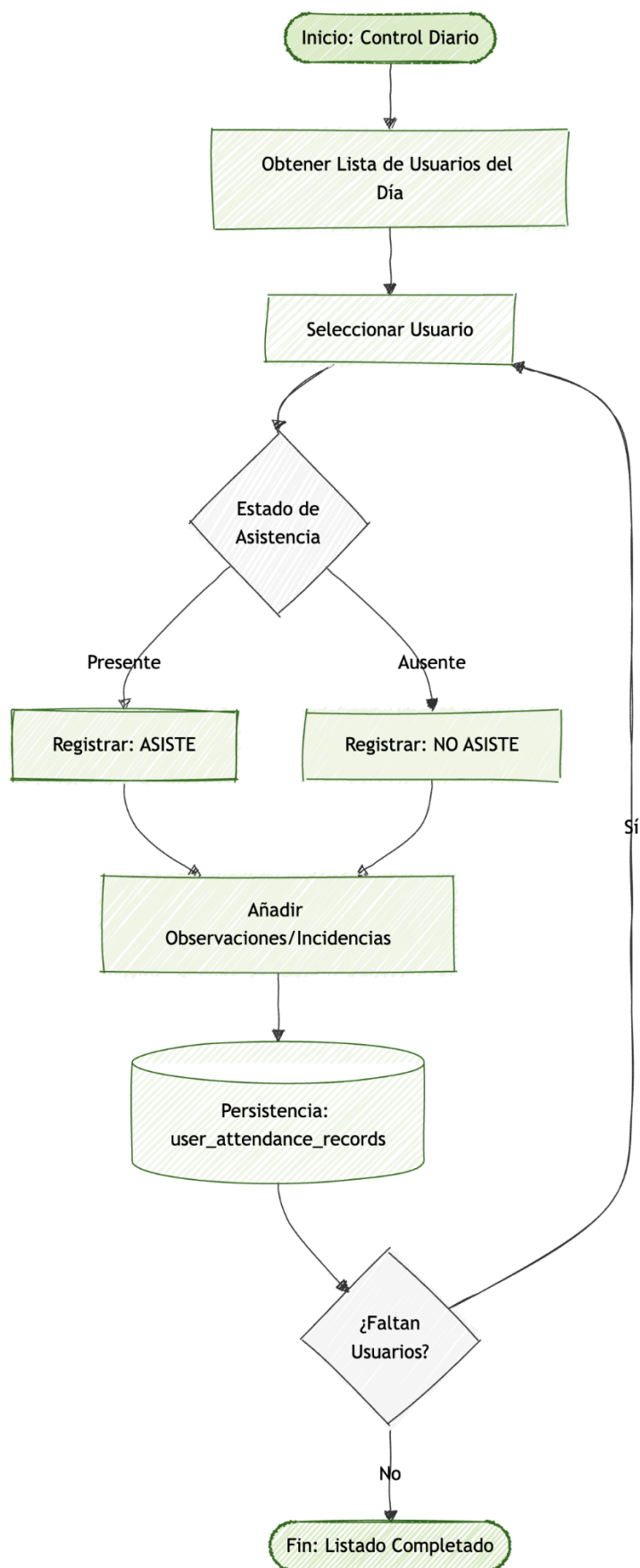


Figura 3.2: *Flujo de Control de Asistencia de Usuarios*

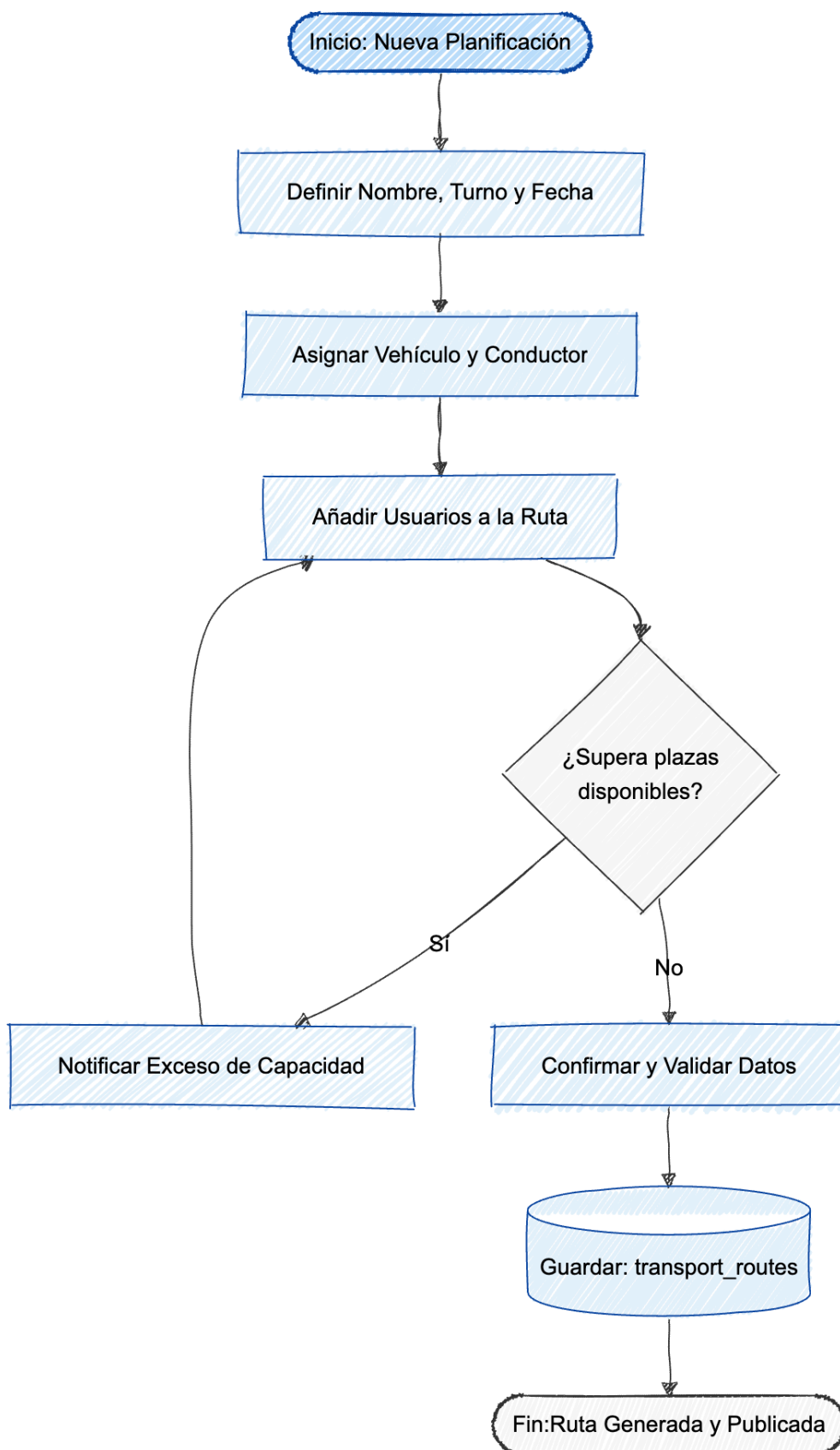


Figura 3.3: Flujo de Gestión de Rutas de Transporte

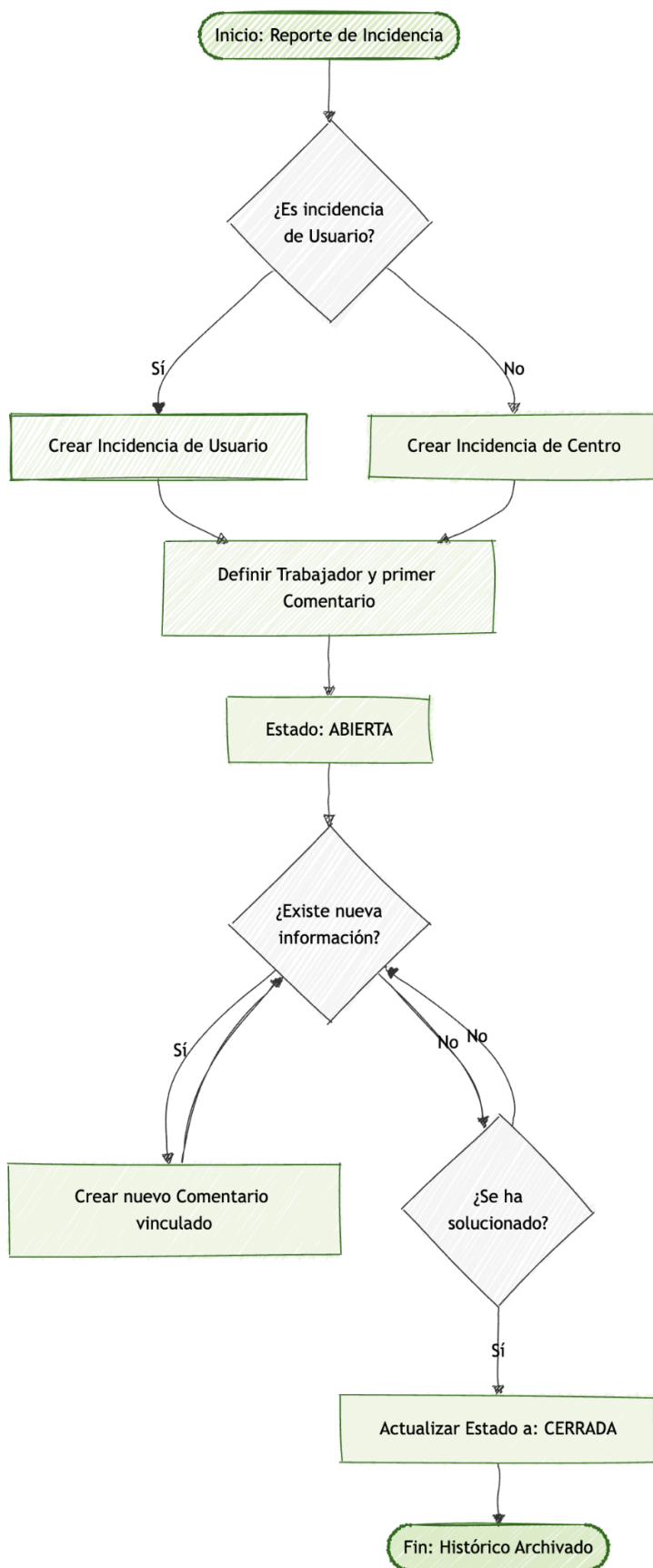


Figura 3.4: *Flujo de Gestión de Incidencias de Usuario / Centro*

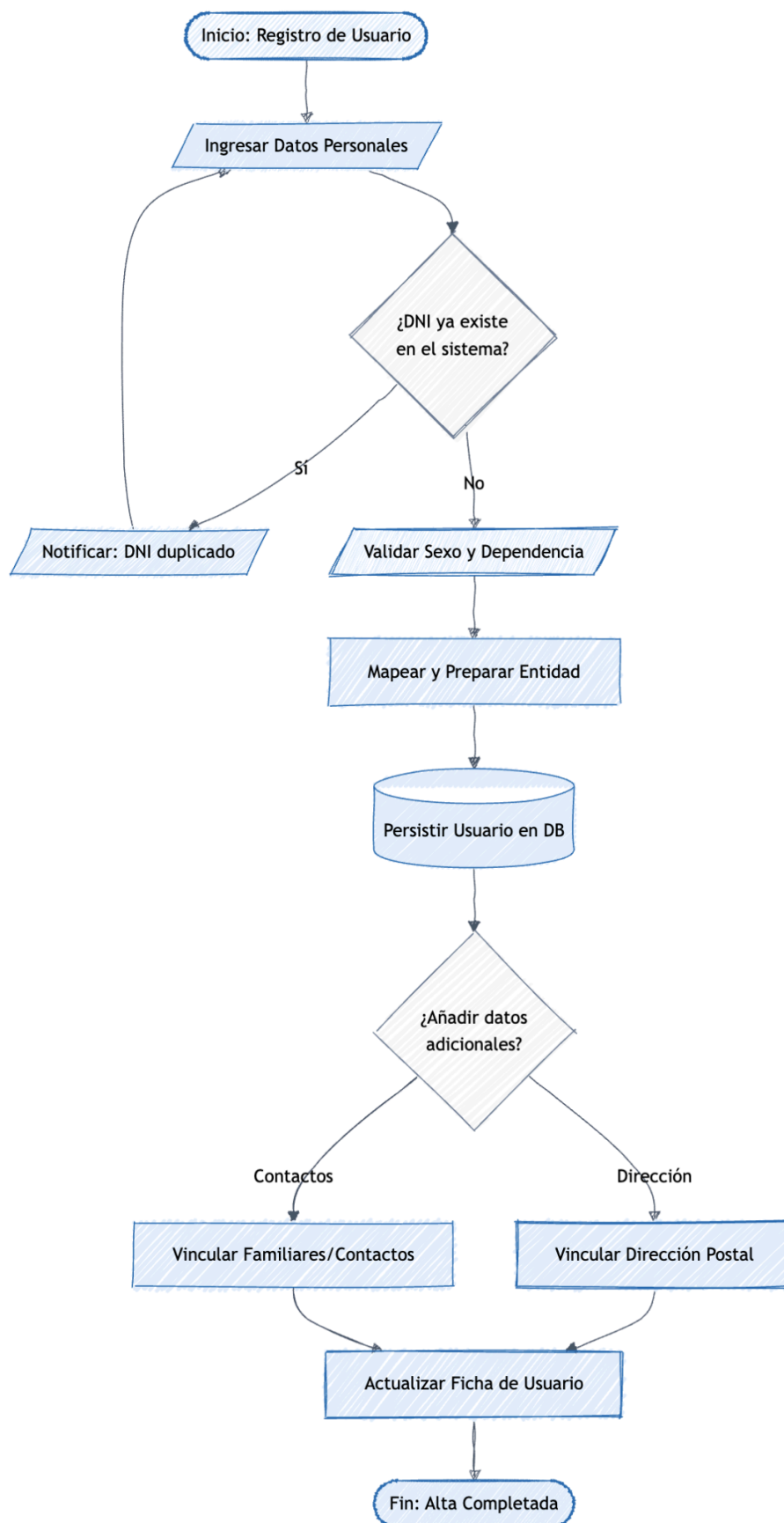


Figura 3.5: *Flujo de Gestión de Alta de Usuarios*

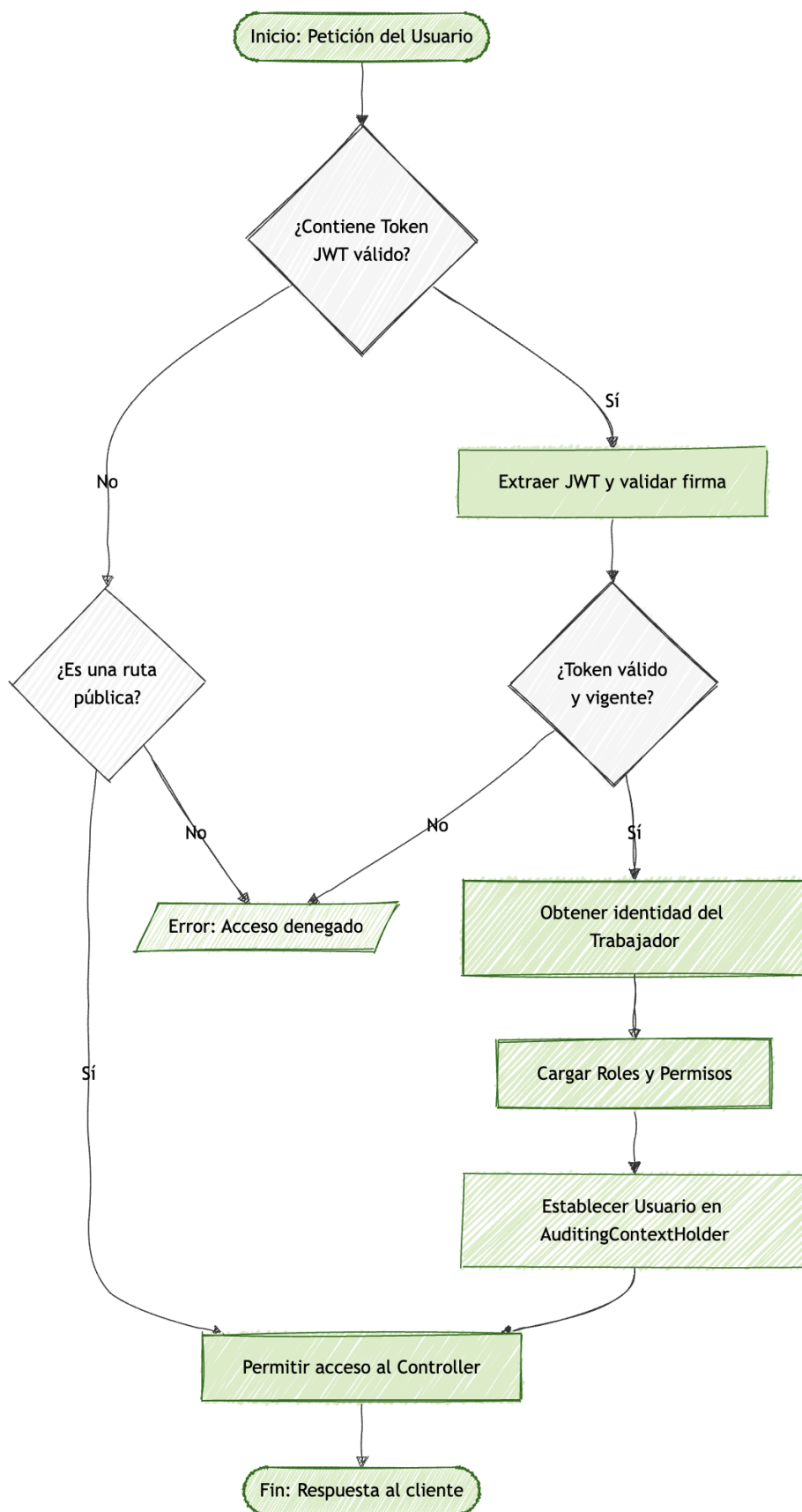


Figura 3.6: *Flujo de Gestión de seguridad y acceso JWT*

4. METODOLOGÍA DE TRABAJO Y PLANIFICACIÓN

4.1. Marco metodológico: Colaboración y Especialización

Para abordar el desarrollo de este proyecto, hemos adoptado una metodología de trabajo basada en la especialización por capas. El objetivo no ha sido simplemente dividir el trabajo, sino permitir que cada integrante se especialice en una parte del sistema para avanzar en paralelo, manteniendo siempre una comunicación constante para asegurar que todas las piezas encajen.

4.2. Fase de investigación y análisis de campo

Como punto de partida, llevamos a cabo un estudio en el Centro de Día El Bosque. Analizamos el software que utilizan actualmente y dialogamos con el personal para detectar los problemas reales de su jornada. Este análisis conjunto nos permitió definir requisitos que aporten valor real, eliminando funciones innecesarias de residencias tradicionales y centrándonos en lo que verdaderamente agiliza el trabajo en un centro de día.

4.3. Diseño compartido: El modelo de datos - Database First

Antes de comenzar con la implementación de cualquier línea de código, se realizó una fase de diseño conjunta de la base de datos en PostgreSQL mediante la aplicación **DrawDB**. Teníamos claro que, al tratarse de una plataforma SaaS con arquitectura Multi-tenant, la base de datos debía ser el núcleo sólido del proyecto.

Realizar este diseño entre los tres integrantes fue una decisión importante por varios motivos:

- **Esfuerzo conjunto:** Garantizamos que la fase de arquitectura más compleja y vital fuera un trabajo de equipo.
- **Contrato de datos:** Al definir juntos cada tabla y relación, establecimos un lenguaje común que nos permitió saber exactamente qué datos íbamos a manejar antes de iniciar nuestras tareas individuales.
- **Visión global:** Esta colaboración aseguró que todos tuviéramos conocimiento de todo el sistema, desde el aislamiento de seguridad por centros hasta la logística de las rutas de transporte.

4.4. División de roles y responsabilidades

Con el diseño de datos consolidado, dividimos el desarrollo en tres áreas de responsabilidad. Buscamos un reparto de tareas que fuera justo y equilibrado, asignando a cada integrante un reto tecnológico:

- **Desarrollo de la API y Lógica de Negocio:** Responsable de construir el corazón de la aplicación en Spring Boot. Su labor consistió en implementar los servicios y la lógica operativa que permite el funcionamiento diario del centro: control de fichajes, asistencia de usuarios y la gestión integral de las rutas de transporte.
- **Seguridad e implementación del modelo SaaS:** Esta área se centró en convertir la aplicación en un servicio en la nube real. Su trabajo se focalizó en la seguridad mediante tokens JWT y en asegurar el aislamiento Multi-tenant, garantizando que cada centro trabaja exclusivamente con sus propios datos de forma privada y segura.

- **Desarrollo de la interfaz de usuario - Frontend:** Responsable de dar vida al proyecto mediante Angular. Su reto consistió en traducir toda la complejidad técnica del Backend en una herramienta intuitiva y eficiente para el personal, priorizando una experiencia de usuario profesional que facilite el trabajo diario.

4.5. Apoyo mutuo y cohesión del equipo

Es fundamental destacar que, aunque existió una división de roles, el trabajo no se realizó de forma aislada. Mantuvimos una filosofía de apoyo mutuo constante: cuando un integrante se enfrentaba a un bloqueo, lo resolvíamos en grupo.

Esta cohesión fue clave, especialmente en la fase de integración, donde la comunicación entre el responsable de la API, el de Seguridad y el de Frontend fue permanente para lograr que el sistema funcionara como una unidad cohesionada.

4.6. Planificación temporal: Carta Gantt

La planificación que se adjunta refleja este desarrollo en paralelo. El esfuerzo inicial compartido en el diseño de la base de datos fue lo que nos permitió trabajar con seguridad en nuestras respectivas áreas, garantizando que la integración final resultara en la solución robusta que presentamos en esta memoria

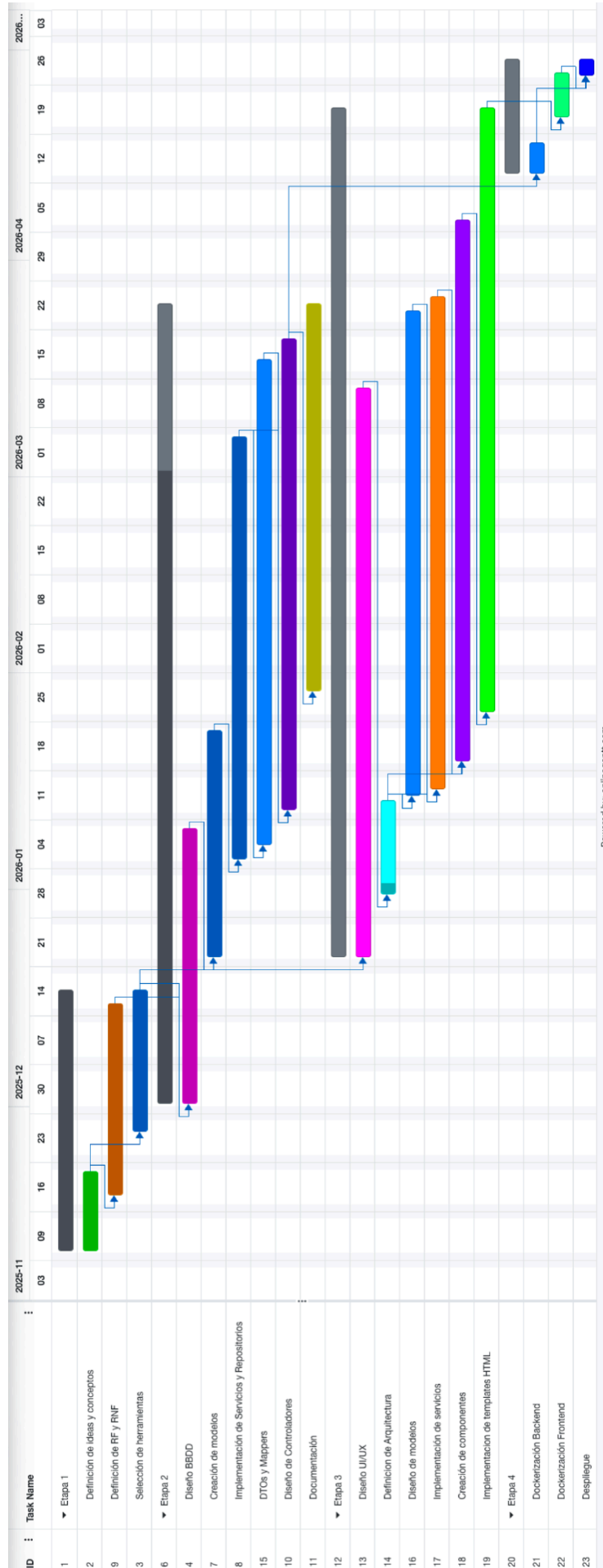


Ilustración - Carta Gantt: Cronograma de planificación y desarrollo del proyecto

5. DESARROLLO TÉCNICO

El desarrollo del proyecto se ha seccionado en etapas, la documentación señalada en las páginas anteriores corresponde a la **Etapa 1**, en donde definimos las ideas, conceptos, requisitos funcionales, no funcionales y la selección de las herramientas apropiadas.

En esta sección se profundizará sobre el desarrollo técnico del proyecto, señalando solamente los detalles únicos y relevantes. Esto con el objetivo de no extender innecesariamente el informe.

5.1. Etapa 2: Diseño de la Base de Datos y Desarrollo del Backend

5.1.1. Diseño de la Base de Datos

Para el diseño de la base de datos del proyecto se utilizó la plataforma **DrawDB**. Esta herramienta facilita el diseño visual de la estructura de la base de datos, permitiendo seleccionar el tipo **PostgreSQL** (entre otros), definir los tipos de datos de las columnas, y establecer las relaciones entre las tablas. Una vez finalizado el diseño, la plataforma permite exportar el esquema de la base de datos, en formatos como SQL según definiciones para **PostgreSQL**.

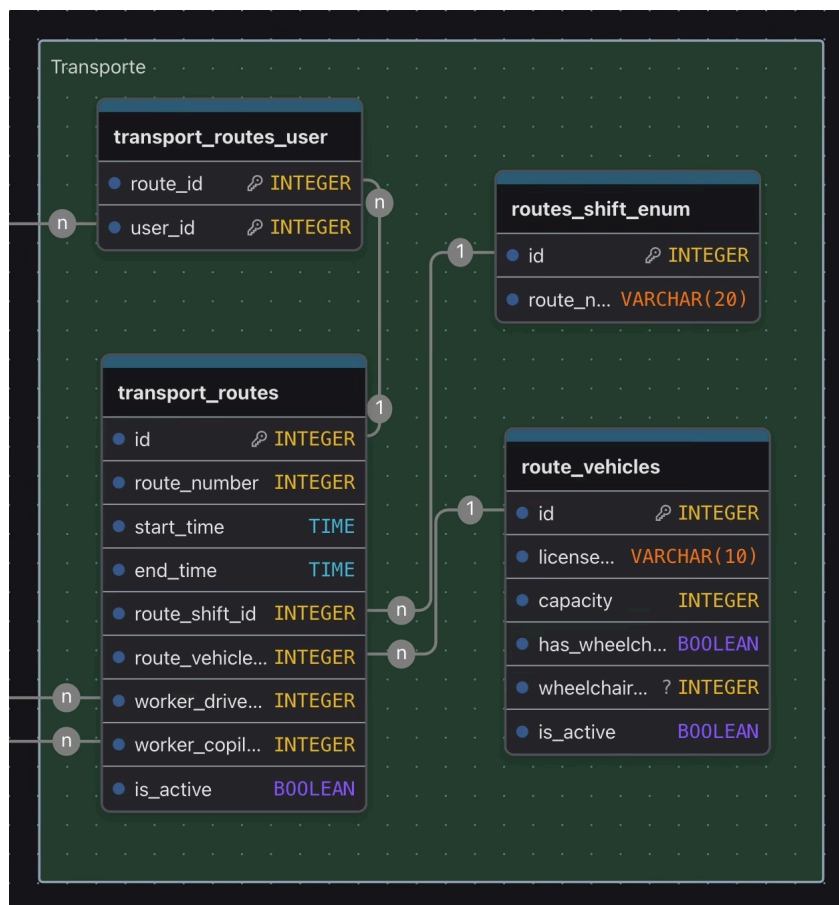


Ilustración - Representación de la sección de transporte en el diseño

Nota: El diseño completo está incluido como anexo externo a este informe.

5.1.2. Modelos en Java

Para facilitar la creación de los modelos y evitar boilerplate hemos optado por el uso de anotaciones JPA/Hibernate para la base de datos y Lombok.

```

TFG - User.java

1  @Entity
2  @Table(name = "users")
3  @Getter @Setter
4  @NoArgsConstructor
5  @AllArgsConstructor
6  @ToString(onlyExplicitlyIncluded = true, callSuper = false)
7  @EqualsAndHashCode(onlyExplicitlyIncluded = true, callSuper = false)
8  public class User extends BaseEntity {...}
  
```

***Ilustración** - Anotaciones implementadas en más del 95% de los modelos (variando según necesidad)*

La siguiente tabla explica la necesidad de cada una de las anotaciones:

Tipo	Anotación	Uso
JPA/Hibernate	@Entity	Le indica a JPA que esta clase es un componente persistente. Sin esto, Hibernate no sabría que debe mapear esta clase a una tabla en la base de datos.
JPA/Hibernate	@Table(name = "users")	Define el nombre específico de la tabla. Es una buena práctica porque, por defecto, JPA usaría el nombre de la clase (User), el cual podría entrar en conflicto con palabras reservadas de SQL.
Lombok	@Getter/@Setter	Generan los métodos para leer y modificar los atributos. Esto mantiene el principio de encapsulamiento sin tener que escribir código repetitivo.
Lombok	@NoArgsConstructor	Genera un constructor sin parámetros. Obligatorio para JPA , ya que Hibernate lo utiliza para instanciar la entidad antes de rellenar sus campos.
Lombok	@AllArgsConstructor	Genera un constructor con todos los campos. Útil para instanciar objetos rápidamente.
Lombok	@ToString(onlyExplicitlyIncluded = true, callSuper = false)	Genera el ToString de la clase, pero se implementan los atributos para: - Evitar que al imprimir el objeto se muestren todos los campos. Solo considerará los que se especifiquen. - Evitar llamar al ToString de la clase padre BaseEntity
Lombok	@EqualsAndHashCode(...)	Genera el EqualsAndHashCode de la clase, utilizando los mismos atributos que el ToString.

Para los campos de los modelos también se han implementados anotaciones para agilizar la escritura del código.

```

TFG - User.java

1  @Id
2  @GeneratedValue(strategy = GenerationType.IDENTITY)
3  @ToString.Include
4  @EqualsAndHashCode.Include
5  private Integer id;
6
7  @NonNull
8  @Column(name = "first_name", nullable = false, length = 25, columnDefinition = "VARCHAR")
9  private String firstName;
10
11 @Column(name = "second_name", length = 25, columnDefinition = "VARCHAR")
12 private String secondName;
13
14 @NonNull
15 @Column(name = "first_surname", nullable = false, length = 25, columnDefinition = "VARCHAR")
16 private String firstSurname;
17
18 @Column(name = "second_surname", length = 25, columnDefinition = "VARCHAR")
19 private String secondSurname;
  
```

Ilustración - Representación de campos tipo en los modelos

La siguiente tabla explica la necesidad de cada una de las anotaciones:

Tipo	Anotación	Uso
JPA/Hibernate	@Id	Define el atributo id como la Primary Key de la tabla, permitiendo identificar de forma única cada registro.
JPA/Hibernate	@GeneratedValue	Configura la estrategia de generación de la clave. Con IDENTITY , se delega al motor (MySQL/PostgreSQL) la creación del ID mediante un autoincremento.
JPA/Hibernate	@Column(name = "...", nullable = ..., length = ..., columnDefinition = "...")	Define las propiedades de la columna en la tabla: name para el nombre físico (snake_case), nullable = false para restricciones de integridad, y length/columnDefinition para el esquema DDL.
Lombok	@ToString.Include	Marca explícitamente el campo para que aparezca en el método toString() . Se relaciona con las anotaciones de la tabla anterior.
Lombok	@EqualsAndHashCode.Include	Cumple la misma función que ToString.Include
Lombok	@NonNull	Genera una comprobación en tiempo de ejecución. Si intentas instanciar un usuario con firstName o firstSurname nulos (vía constructor o setter), Lombok lanzará una NullPointerException.

Nota: Para los campos como **firstName** y **firstSurname** estamos aplicando una "**doble validación**": **@NonNull** de Lombok (a nivel de código Java) y **nullable = false** en **@Column** (a nivel de base de datos). Esto garantiza que el dato sea obligatorio en todas las capas.

Con respecto a las relaciones y la gestión de estas relaciones bidireccionales con métodos **Defensive Methods** para agregar o quitar vínculos.

```

TFG - User.java

1 // // UserAddress
2 @OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
3 private Set<UserAddress> userAddresses = new HashSet<>();
4
5 public void addUserAddress(UserAddress userAddress) {
6     userAddresses.add(userAddress);
7     userAddress.setUser(this);
8 }
9 public void removeUserAddress(UserAddress userAddress) {
10     userAddresses.remove(userAddress);
11     userAddress.setUser(null);
12 }
    
```

Ilustración - Representación tipo de una relación y los Defensive Methods

La siguiente tabla explica la necesidad de cada una de las anotaciones:

Tipo	Anotación / Elemento	Uso
JPA/Hibernate	@OneToMany	Define una relación de uno a muchos. Un usuario puede tener múltiples direcciones.
JPA/Hibernate	mappedBy = "user"	Indica que la relación es bidireccional y que el "dueño" de la relación (quien gestiona la clave foránea en la BD) es la entidad hija. Esto optimiza el rendimiento al evitar tablas intermedias innecesarias.
JPA/Hibernate	fetch = FetchType.LAZY	Configura la carga perezosa. Las colecciones solo se recuperan de la base de datos cuando se accede a ellas explícitamente, evitando sobrecargar la memoria con datos que no siempre son necesarios.
Collections	Set<Entity>	El uso de Set en lugar de List garantiza que no haya elementos duplicados en la colección y es más eficiente para las operaciones de búsqueda y eliminación coordinadas con Hibernate.
Methods	addUser / RemoveUser	Métodos de utilidad que aseguran la sincronización de ambos lados de la relación . Establecen el puntero del hijo hacia el padre y viceversa en una sola operación atómica de software.

Nota: Inicialización temprana de colecciones mediante HashSet para evitar errores de puntero nulo (**NullPointerException**) y garantizar la unicidad de los elementos.

Nota: Esta estructura busca que el sistema sea robusto (evitando errores de nulos e inconsistencias), eficiente (cargando solo lo justo) y escalable (preparado para manejar grandes volúmenes de datos sin degradar el rendimiento).

5.1.3. Auditing y BaseEntity

En esta parte del código implementamos un sistema de auditoría automática **Spring Auditing**. Su objetivo es registrar quién (mediante **Spring Security**) y cuándo se crea o modifica cualquier registro en la base de datos sin tener que escribir esa lógica manualmente en cada servicio ni el uso de Triggers.

Para lograrlo se configura **AuditingContextHolder** que permite recuperar la identidad del auditor (el usuario que realiza la acción) de forma dinámica.

```

TFG - AuditingContextHolder.java

1  public class AuditingContextHolder {
2
3      private static AuditorAware<String> auditorAware;
4
5      public static void setAuditorAware(AuditorAware<String> auditorAware) {
6          AuditingContextHolder.auditorAware = auditorAware;
7      }
8
9      public static Optional<String> getCurrentAuditor() {
10         if (auditorAware == null) {
11             return Optional.empty();
12         }
13         return auditorAware.getCurrentAuditor();
14     }
15 }

```

Ilustración - Implementación del Context Holder

Luego el **AuditingListener** actúa como un interceptor que automatiza la trazabilidad del sistema, capturando eventos del ciclo de vida en entidades para inyectar fechas y autores sin intervención manual.

```

TFG - AuditingListener.java

1  public class AuditingListener {
2
3      @PrePersist
4      public void touchForCreate(BaseEntity entity) {
5          LocalDateTime now = LocalDateTime.now();
6          String auditor = AuditingContextHolder.getCurrentAuditor().orElse("system");
7
8          entity.setCreatedAt(now);
9          entity.setCreatedBy(auditor);
10         entity.setUpdatedAt(now);
11         entity.setUpdatedBy(auditor);
12     }
13
14     @PreUpdate
15     public void touchForUpdate(BaseEntity entity) {
16         entity.setUpdatedAt(LocalDateTime.now());
17         entity.setUpdatedBy(AuditingContextHolder.getCurrentAuditor().orElse("system"));
18     }
19 }

```

Ilustración - Implementación de Auditing Listener

Finalmente mediante el uso de una clase **BaseEntity** y el **AuditingListener**, la aplicación intercepta las operaciones de persistencia para inyectar automáticamente la fecha actual y el usuario identificado a través del **AuditingContextHolder**.

```

TFG - BaseEntity.java

1  @MappedSuperclass
2  @EntityListeners(AuditingListener.class)
3  @Getter
4  @Setter
5  public abstract class BaseEntity {
6
7      @Column(name = "created_at", nullable = false, updatable = false)
8      private LocalDateTime createdAt;
9
10     @Column(name = "updated_at")
11     private LocalDateTime updatedAt;
12
13     @Column(name = "created_by", nullable = false, updatable = false)
14     private String createdBy;
15
16     @Column(name = "updated_by")
17     private String updatedBy;
18
19 }
  
```

Ilustración - Implementación de BaseEntity

Al ser una clase abstracta nos permite extenderlo en los modelos donde sea necesario (Todos los modelos en nuestro caso) y **@MappedSuperclass** le dice a JPA que esta clase sea heredada por otras entidades.

La siguiente tabla explica la necesidad de cada una de las anotaciones:

Tipo	Anotación / Elemento	Uso
JPA	@MappedSuperclass	Permite que los campos de BaseEntity sean heredados por otras entidades, mapeándose como columnas en sus tablas.
JPA	@EntityListeners	Vincula la entidad con una clase "observadora" (AuditingListener). Indica que JPA debe avisar a esta clase cada vez que ocurra un evento en el ciclo de vida de la entidad.
JPA	@PrePersist	Marca un método para que se ejecute automáticamente justo antes de insertar un nuevo registro en la base de datos.
JPA	@PreUpdate	Marca un método para que se ejecute automáticamente antes de actualizar un registro existente.
JPA	@Column(updatable = false)	Garantiza la integridad de la auditoría impidiendo que los campos createdAt y createdBy se modifiquen una vez que el registro ha sido creado.

Este diseño centralizado elimina la duplicación de código y asegura que todos los registros del sistema mantengan una trazabilidad homogénea y fiable.

5.1.4. Repositorios

Un Repositorio es una capa de abstracción que se encarga de manejar el acceso a los datos. Básicamente, es el intermediario entre tu código (la lógica de negocio) y la base de datos. Esto ocurre gracias a Spring Data JPA. En lugar de escribir largas consultas SQL manualmente, tú defines una interfaz que extiende de clases como JpaRepository o CrudRepository.

```
TFG - WorkerRepository.java
1 public interface WorkerRepository extends JpaRepository<Worker, Integer> {...}
```

Ilustración - Implementación Repository tipo

Al extender de estas interfaces, ya tenemos métodos como .save(), .findAll(), .findById() y .delete() sin escribir una sola línea de lógica.

Una de las funciones más potentes es que puedes crear consultas personalizadas simplemente nombrando el método siguiendo una convención llamada **Query Methods**.

```
TFG - WorkerRepository.java
1 Optional<Worker> findByEmail(String email);
```

Ilustración - Implementación Query Method para encontrar a un trabajador por su email

Para consultas más complejas u optimización de rendimiento usamos la anotación @Query es la herramienta que utiliza en **Spring Data JPA** cuando las consultas automáticas (derivadas del nombre del método) no son suficientes o cuando necesitas optimizar el rendimiento.

```
TFG - WorkerRepository.java
1 @Query("SELECT w FROM Worker w LEFT JOIN FETCH w.roles WHERE w.email = :email")
2 Optional<Worker> findByEmailWithRoles(@Param("email") String email);
```

Ilustración - Implementación de una @Query que trae a los usuarios y sus roles

La consulta anterior resuelve un problema importante normalmente, si pides un Worker, JPA no carga sus Roles hasta que los necesites, lo que genera múltiples consultas a la base de datos (el famoso problema n+1). La solución es usar Fetch, le dice a Hibernate: "Trae al trabajador y sus roles en una sola consulta de una vez". Esto hace que la aplicación sea mucho más rápida si sabes que vas a usar esos roles de inmediato.

5.1.5. DTOs (Data Transfer Object)

Los DTOs son objetos diseñados exclusivamente para transportar datos entre diferentes capas de una aplicación, por ejemplo, entre el frontend y el backend. Su objetivo principal es evitar exponer directamente las entidades de la base de datos, mejorando la seguridad, el rendimiento y desacoplando la lógica interna de lo que ve el usuario.

En este proyecto, se utilizan tres tipos especializados de DTOs para manejar el ciclo de vida de los datos de forma clara y segura: **XxxRequestDTO**, **XxxResponseDTO** y **XxxUpdateDTO**.

```

TFG - RouteVehicleRequestDTO.java

1  public record RouteVehicleRequestDTO(
2
3      @NotBlank(message = "La matrícula es obligatoria")
4      @Size(max = 10, message = "La matrícula no puede exceder los 10 caracteres")
5      String licensePlate,
6
7      @NotNull(message = "La capacidad es obligatoria")
8      @Min(value = 1, message = "La capacidad debe ser al menos 1")
9      Integer capacity,
10
11     @NotNull(message = "Debe indicar si tiene silla de ruedas")
12     Boolean hasWheelchair,
13
14     Integer wheelchairCapacity
15 ) {}
    
```

Ilustración - RequestDTO tipo

```

TFG - RouteVehicleResponseDTO.java

1  public record RouteVehicleResponseDTO(
2
3      Integer id,
4      String licensePlate,
5      Integer capacity,
6      Boolean hasWheelchair,
7      Integer wheelchairCapacity,
8      Boolean isActive,
9      String createdAt,
10     String createdBy,
11     String updatedAt,
12     String updatedBy
13 ) {}
    
```

Ilustración - ResponseDTO tipo

```

TFG - RouteVehicleUpdateDTO.java
1  @Getter @Setter
2  @Builder
3  @NoArgsConstructor
4  @AllArgsConstructor
5  public class RouteVehicleUpdateDTO {
6
7      @Size(max = 10, message = "La matrícula no puede exceder los 10 caracteres")
8      private String licensePlate;
9
10     @Min(value = 1, message = "La capacidad debe ser al menos 1")
11     private Integer capacity;
12
13     private Boolean hasWheelchair;
14
15     private Integer wheelchairCapacity;
16
17     private Boolean isActive;
18
19 }

```

Ilustración - UpdateDto tipo

El uso de tres DTOs distintos para una misma entidad no es una redundancia, sino una práctica de arquitectura orientada a la separación de responsabilidades. Aunque compartan campos similares, sus contextos de uso y reglas de validación son fundamentalmente diferentes.

A continuación se justifica la importancia de cada DTO:

5.1.5.1. Desacoplamiento de la entrada y la salida

No toda la información que se guarda en el sistema debe ser devuelta al cliente, y viceversa.

- **RequestDTO (Entrada):** Se centra en lo que el usuario debe proporcionar. Aquí no existen campos como la fecha de creación o el ID, ya que el usuario no tiene control sobre ellos al momento de dar de alta un recurso.
- **ResponseDTO (Salida):** Se centra en lo que el usuario necesita ver. Incluye metadatos, IDs y estados calculados que son útiles para la interfaz, pero que nunca deberían ser enviados por el usuario en una petición de escritura.

5.1.5.2. Especificidad en las validaciones

Las reglas de integridad varían según la operación que se esté realizando. En una **Creación (RequestDTO)**, la mayoría de los campos suelen ser obligatorios, mediante anotaciones como **@NotNull** o **@NotBlank**, ya que se está construyendo el objeto desde cero. Sin embargo, en una **Actualización (UpdateDTO)**, las validaciones pueden ser más flexibles o incluso incluir campos de control, como un "motivo de cambio" o un "estado de activación", que no tienen sentido durante el alta inicial. Separarlos evita que las anotaciones de validación de un flujo interfieran con el otro.

5.1.5.3. Seguridad y Control de Datos (Over-posting)

El uso de un único DTO para todas las operaciones abre la puerta a ataques de *Mass Assignment* u *Over-posting*. Si se usara la misma entidad para crear y actualizar, un usuario malintencionado podría intentar modificar campos protegidos, como su rol de usuario o un saldo bancario, dentro de una petición de actualización. Al contar con un **UpdateDTO** específico, el sistema solo procesa los campos que han sido definidos explícitamente para esa operación.

5.1.5.4. Mantenibilidad y Evolución técnica

Con el tiempo, las necesidades de la interfaz cambian. Es posible que para mostrar un objeto (Response) necesites transformar un código de país en su nombre completo para facilitar la lectura, mientras que para actualizarlo (Update) el sistema siga requiriendo únicamente el código original. Esta flexibilidad se apoya en dos decisiones de diseño:

- **Uso de records para la respuesta:** Proporciona la inmutabilidad y eficiencia necesarias al enviar los datos procesados hacia el frontend.
- **Utilizar clases con Lombok para la actualización:** Permite la mutabilidad requerida para facilitar procesos de mapeo parcial y lógica interna del servidor.

En resumen, esta estructura basada en tres tipos de DTO asegura que la API sea robusta, segura y fácil de escalar, evitando que los cambios en una operación específica afecten accidentalmente a las demás.

5.1.6. Capa de Mapeo (MapStruct Mappers)

Para gestionar la transformación de datos entre las entidades de la base de datos y los DTOs definidos en secciones anteriores, se ha implementado una capa de mapeo utilizando la dependencia **MapStruct**. Esta herramienta genera automáticamente el código de conversión en tiempo de compilación, lo que garantiza un rendimiento óptimo y elimina la necesidad de escribir manualmente métodos getter y setter repetitivos.

```

TFG - RouteVehicleMapper.java

1  @Mapper(
2      componentModel = "spring",
3      unmappedTargetPolicy = ReportingPolicy.IGNORE,
4      nullValuePropertyMappingStrategy = NullValuePropertyMappingStrategy.IGNORE,
5      imports = MapperHelper.class
6  )
7  public interface RouteVehicleMapper {...}
    
```

Ilustración - Configuración global de la interfaz RouteVehicleMapper

5.1.6.1. Configuración y Estrategia de Mapeo

Como se observa en la configuración inicial, los mappers están integrados totalmente con el ecosistema de Spring mediante el parámetro **componentModel = "spring"**. Esto permite que sean inyectados como cualquier otro componente en la capa de servicios. Además, se han definido políticas estrictas para ignorar campos no mapeados de forma explícita, evitando advertencias innecesarias durante la compilación.

5.1.6.2. Transformaciones de Salida (Entity a ResponseDTO)

Uno de los mayores beneficios de esta capa es la capacidad de realizar transformaciones complejas en una sola línea. En el método **toResponse**, se ha integrado un **MapperHelper** personalizado para dar formato a las fechas de auditoría.

```

TFG - RouteVehicleMapper.java

1 @Mapping(target = "createdAt", expression = "java(MapperHelper.formatDateTime(vehicle.getCreatedAt()))")
2 @Mapping(target = "updatedAt", expression = "java(MapperHelper.formatDateTime(vehicle.getUpdatedAt()))")
3 RouteVehicleResponseDTO toResponse(RouteVehicle vehicle);
4
5 @Mapping(target = "id", ignore = true)
6 @Mapping(target = "isActive", ignore = true)
7 RouteVehicle toEntity(RouteVehicleRequestDTO dto);
8
9 @Mapping(target = "id", ignore = true)
10 void updateFromDto(RouteVehicleUpdateDTO dto, @MappingTarget RouteVehicle vehicle);
11

```

Ilustración - Definición de mapeos con expresiones personalizadas para fechas

Al utilizar expresiones de Java dentro de la anotación **@Mapping**, el sistema transforma automáticamente objetos **LocalDateTime** de la base de datos en cadenas de texto legibles para el frontend, centralizando así la lógica de presentación y asegurando que el usuario final reciba siempre el formato de fecha correcto.

5.1.6.3. Transformaciones de Entrada y Actualización Parcial

Para las operaciones de escritura, el mapper gestiona dos escenarios que protegen la integridad de los datos:

- **Creación (toEntity):** Al transformar un **RequestDTO** en una entidad, el sistema ignora deliberadamente campos como el id y el estado **isActive**. Esto asegura que sea la base de datos y la lógica de negocio quienes controlen estos valores, evitando que un usuario externo pueda manipular identificadores de forma malintencionada.
- **Actualización Parcial (updateFromDto):** Se utiliza la anotación **@MappingTarget** para actualizar una entidad ya existente con los datos provenientes de un **UpdateDTO**.

Esta técnica permite que el servicio cargue primero el objeto desde la base de datos y luego solo se modifiquen los campos necesarios, manteniendo intactos el resto de los atributos y optimizando el uso de recursos al no tener que recrear el objeto completo.

5.1.7. Capa de Servicios (Business Logic Layer)

La capa de servicios constituye el núcleo funcional de la plataforma. Su misión es orquestar la lógica de negocio, actuando como intermediaria entre los controladores, que reciben la petición, y los repositorios, que acceden a los datos. En este proyecto se ha implementado el patrón **Interface e Implementación (Service e ServiceImpl)**, lo que asegura un código escalable, fácil de testear y totalmente desacoplado.

5.1.7.1. Definición del Contrato (Interface)

El uso de interfaces permite definir un "contrato" de operaciones sin revelar los detalles de su ejecución interna. Esto es fundamental para asegurar que las capas superiores, como los controladores, dependan únicamente de las firmas de los métodos y no de una implementación concreta, facilitando la escalabilidad del sistema.

```

TFG - RouteVehicleService.java

1  public interface RouteVehicleService {
2
3      Page<RouteVehicleResponseDTO> findAll(Pageable pageable);
4
5      Page<RouteVehicleResponseDTO> findAllInactive(Pageable pageable);
6
7      Page<RouteVehicleResponseDTO> findAllIncludingInactive(Pageable pageable);
8
9      Optional<RouteVehicleResponseDTO> findById(Integer id);
10
11     RouteVehicleResponseDTO save(RouteVehicleRequestDTO dto);
12
13     Optional<RouteVehicleResponseDTO> update(Integer id, RouteVehicleUpdateDTO dto);
14
15     Optional<RouteVehicleResponseDTO> delete(Integer id);
16
17 }
```

Ilustración - Interfaz RouteVehicleService

Como se observa en la definición, el servicio integra el uso de objetos **Page** para gestionar consultas paginadas y **Optional** para manejar de forma segura la posible ausencia de registros, evitando errores de tipo `NullPointerException`.

5.1.7.2. Gestión de Dependencias e Inmutabilidad

Siguiendo las mejores prácticas de Spring Boot,, la implementación del servicio utiliza la anotación **@Service** para ser detectada por el contenedor de Spring y **@RequiredArgsConstructor** para la gestión de dependencias.

```

TFG - RouteVehicleServiceImpl.java

1  @Service
2  @RequiredArgsConstructor
3  public class RouteVehicleServiceImpl implements RouteVehicleService {...}
```

Ilustración - Definición de la clase de servicio y dependencias inyectadas.



TFG - RouteVehicleServiceImpl.java

```
1 private final RouteVehicleRepository routeVehicleRepository;
2 private final TransportRouteRepository transportRouteRepository;
3 private final RouteVehicleMapper routeVehicleMapper;
```

Ilustración - Definición de campos final para la inyección de dependencias por constructor.

Al declarar los repositorios y el mapper como campos **private final**, se garantiza la inmutabilidad de las dependencias. La inyección por constructor es la práctica recomendada actualmente, ya que asegura que el servicio cuente con todos sus componentes necesarios desde el momento de su instanciación.

5.1.7.3. Orquestación y Lógica de Negocio

El servicio es el responsable de gestionar el flujo completo de los datos, coordinando la transformación de los objetos mediante los mappers y asegurando que se cumplan las reglas del sistema. En un flujo típico de guardado, el proceso sigue un orden lógico estricto:

- Recibe un **RequestDTO** desde el controlador.
- Aplica las reglas de negocio y validaciones necesarias.
- Transforma el DTO en una **Entidad** para su persistencia en la base de datos.
- Devuelve un **ResponseDTO** procesado de vuelta al controlador.

Un ejemplo de esta orquestación se encuentra en la implementación del método **save**:



TFG - RouteVehicleServiceImpl.java

```
1 @Override
2 @Transactional
3 public RouteVehicleResponseDTO save(RouteVehicleRequestDTO dto) {
4     if (routeVehicleRepository.existsByLicensePlate(dto.licensePlate().trim().toUpperCase())) {
5         throw new BadRequestException("La matrícula ya está registrada");
6     }
7
8     RouteVehicle vehicle = routeVehicleMapper.toEntity(dto);
9     vehicle.setLicensePlate(dto.licensePlate().trim().toUpperCase());
10
11     if (!Boolean.TRUE.equals(dto.hasWheelchair())) {
12         vehicle.setWheelchairCapacity(null);
13     }
14
15     RouteVehicle saved = routeVehicleRepository.save(vehicle);
16     return routeVehicleMapper.toResponse(saved);
17 }
```

Ilustración - Lógica de validación, normalización de datos y flujo de guardado.

En esta implementación, el servicio actúa como guardián de la integridad de los datos del centro de día:

- **Validación de Unicidad:** Verifica mediante el repositorio si la matrícula ya existe en el sistema, lanzando una excepción controlada en caso de duplicidad.
- **Normalización:** Estandariza datos mediante `.trim().toUpperCase()` para asegurar la consistencia en futuras búsquedas y reportes.
- **Adaptación de Datos:** Gestiona lógica condicional, como la capacidad para sillas de ruedas, antes de proceder con el mapeo definitivo a la entidad.

5.1.7.4. Consultas Paginadas y Gestión de Transacciones

Para asegurar la integridad de la información (propiedades ACID), se ha utilizado la anotación **@Transactional**. Esto garantiza que, si alguna parte de una operación compleja falla, el sistema realice un rollback automático, dejando la base de datos en su estado anterior.



```
TFG - RouteVehicleServiceImpl.java

1  @Override
2  @Transactional(readonly = true)
3  public Page<RouteVehicleResponseDTO> findAll(Pageable pageable) {
4      return routeVehicleRepository.findAllByIsActiveTrue(pageable)
5          .map(routeVehicleMapper::toResponse);
6  }
```

Ilustración - Implementación de consulta paginada con optimización de lectura.

En los métodos de consulta, se utiliza el parámetro **readOnly = true** para mejorar la eficiencia del motor de base de datos. Además, se integra el filtrado por el campo **isActive**, cumpliendo con la estrategia de **Soft Delete** definida en la arquitectura; esto permite recuperar únicamente los vehículos activos mientras se mantiene la trazabilidad histórica de los registros eliminados lógicamente, un requisito indispensable para la auditoría de servicios en centros de adultos mayores.

5.1.8. Capa de Controladores (API REST Controllers)

Los controladores actúan como el punto de enlace entre las solicitudes que llegan desde el cliente (frontend) y la lógica interna del servidor. Su responsabilidad es recibir las peticiones HTTP, validar que los datos enviados sean correctos y devolver una respuesta organizada, delegando siempre la ejecución de los procesos complejos a la capa de servicios.

5.1.8.1. Configuración y Estándares RESTful

Para asegurar una comunicación fluida entre el frontend (Angular) y el backend, se han utilizado anotaciones de Spring que definen el comportamiento de cada endpoint de forma precisa.

TFG - AuthController.java

```
1 @RestController
2 @RequestMapping("/api/auth")
3 @RequiredArgsConstructor
4 public class AuthController {...}
```

Ilustración - Definición del controlador y rutas base

En la cabecera del controlador destacan tres elementos importantes:

- **@RestController**: Indica que la clase maneja peticiones REST y que todas las respuestas se enviarán automáticamente en formato JSON.
- **@RequestMapping**: Define la URL base, en este caso `/api/auth`, lo que permite organizar los recursos de forma jerárquica y profesional.
- **@RequiredArgsConstructor**: Al igual que en las capas anteriores, garantiza la inyección de los servicios necesarios de forma inmutable a través del constructor, asegurando la estabilidad del componente.

5.1.8.2. Gestión de Respuestas Estandarizadas

Para garantizar que la comunicación con el frontend sea siempre predecible y profesional, se ha priorizado la homogeneidad en las respuestas. Esto se logra mediante una clase envoltorio denominada **ApiDataResponse**, que estandariza la salida de información de todos los endpoints de la API.

```
TFG - AuthController.java

1 private final AuthService authService;
2
3 @PostMapping("/register")
4 public ResponseEntity<ApiDataResponse<AuthResponse>> register(@Valid @RequestBody RegisterRequest request) {
5     AuthResponse response = authService.register(request);
6     return ResponseEntity.status(HttpStatus.CREATED)
7         .body(new ApiDataResponse<>("Trabajador registrado exitosamente", response, 201));
8 }
9
10 @PostMapping("/login")
11 public ResponseEntity<ApiDataResponse<AuthResponse>> login(@Valid @RequestBody LoginRequest request) {
12     AuthResponse response = authService.login(request);
13     return ResponseEntity.ok(new ApiDataResponse<>("Login exitoso", response, 200));
14 }
```

Ilustración - Implementación de los endpoints y gestión de respuestas mediante ApiDataResponse.

Gracias a este diseño, el frontend recibe siempre una estructura predecible que incluye un mensaje informativo y el cuerpo de los datos. Además, se hace un uso estricto de los códigos de estado HTTP, devolviendo por ejemplo un **201 CREATED** tras un registro exitoso, lo cual es fundamental para asegurar la estabilidad y la comunicación entre todas las capas de la aplicación.

5.1.8.3. Validación de Datos de Entrada y Seguridad

A modo de filtro preventivo, el sistema verifica que la información recibida sea correcta y esté completa antes de procesarse. Esta validación asegura que solo los datos que cumplen con los requisitos técnicos avancen hacia la capa de servicios.

Para ello, se utiliza la anotación **@Valid** junto con **@RequestBody**. Esto permite que el sistema compruebe automáticamente los DTOs de entrada, por ejemplo, validar el formato de un correo electrónico o confirmar que no faltan campos obligatorios. Si algo no es correcto, el controlador detiene la petición de inmediato y devuelve un error claro, evitando que el servidor trabaje con datos inconsistentes y protegiendo la integridad de toda la base de datos.

5.1.9. Seguridad y Arquitectura de Aislamiento Multi-tenant

Esta sección detalla los mecanismos técnicos implementados para garantizar la integridad, privacidad y aislamiento de los datos de cada centro de día en un entorno de Software as a Service (SaaS).

5.1.9.1. Arquitectura Multi-tenant (Schema-per-Tenant)

El sistema emplea una **estrategia de aislamiento lógico** mediante el uso de esquemas independientes para cada cliente dentro de una instancia de **PostgreSQL 18**.

- **Modelo de Datos:** Cada centro de día dispone de su propio esquema, lo que garantiza que las consultas de un cliente no puedan interferir con los datos de otro.
- **Patrón de Nomenclatura:** Se ha definido un estándar de nombrado **tenant_{slug}**, donde el *slug* representa el identificador único y normalizado de cada institución.
- **Segregación de Responsabilidades:**
 - **Esquema public:** Actúa como el catálogo maestro del sistema, almacenando metadatos de suscripciones, configuración de planes y credenciales de los administradores globales.
 - **Esquemas tenant_{slug}:** Albergan la lógica operativa y los datos sensibles de cada centro, tales como historias clínicas, registros de asistencia y rutas de transporte.

```
@Override
public Connection getConnection(String tenantId) throws SQLException {
    log.debug(format: "DEBUG-DB: Hibernate solicita conexión para tenantId: '{}'", tenantId);

    final Connection connection = getAnyConnection();
    try (Statement stmt = connection.createStatement()) {
        if (tenantId != null && !tenantId.equals(anObject: "public")) {
            String schemaName = "tenant_" + tenantId.replace(target: "-", replacement: "_");

            if (!schemaName.matches(regex: "^tenant_[a-z0-9_]+$")) {
                connection.close();
                throw new IllegalArgumentException("TenantId inválido: " + tenantId);
            }

            log.debug(format: "DEBUG-DB: Ejecutando SET search_path TO {}, public", schemaName);
            stmt.execute("SET search_path TO " + schemaName + ", public");
        } else {
            log.debug(msg: "DEBUG-DB: Usando schema public");
            stmt.execute(sql: "SET search_path TO public");
        }
    } catch (SQLException e) {
        log.error(format: "DEBUG-DB: Error al establecer el search_path para tenant {}", tenantId);
        connection.close();
        throw e;
    }
    return connection;
}
```

Ilustración - Lógica de conexión dinámica y aislamiento de esquemas por tenant

5.1.9.2. Gestión de Concurrencia y Aislamiento de Hilos

Dado que Spring Boot procesa las peticiones HTTP mediante un **pool de hilos de Tomcat**, se ha prestado especial atención a evitar la contaminación de datos entre hilos reutilizados.

- **Riesgo de Variables Globales:** El uso de estados estáticos para almacenar el identificador del tenant podría derivar en **fugas de datos** (*data leaks*) entre sesiones concurrentes.
- **Solución mediante ThreadLocal:** Se utiliza el mecanismo **ThreadLocal** para encapsular el contexto del tenant de forma privada para cada hilo. Esto asegura que el identificador sea accesible únicamente durante el ciclo de vida de la petición específica que está siendo procesada.

5.1.9.3. Componente de Soporte: TenantContext

```

21 public final class TenantContext {
22
23     private static final ThreadLocal<String> CURRENT = new ThreadLocal<>();
24
25     private TenantContext() {}
26
27     public static void set(String tenantId) {
28         CURRENT.set(tenantId);
29     }
30     public static String get() {
31         return CURRENT.get();
32     }
33
34     public static void clear() {
35         CURRENT.remove();
36     }
37 }

```

Ilustración - Clase TenantContext para la gestión del estado del hilo.

Para gestionar el estado del hilo de ejecución, se ha desarrollado la clase utilitaria **TenantContext**, que actúa como envoltorio del **ThreadLocal**.

- **Funciones Principales:** Proporciona métodos estáticos para establecer (**set**), recuperar (**get**) y, fundamentalmente, limpiar (**clear**) el identificador del tenant.
- **Ciclo de Vida:** Al no ser un componente gestionado por el contenedor de Spring (**Spring Bean**), su estado está estrictamente ligado al hilo de ejecución y no al ciclo de vida de la aplicación.

5.1.9.4. Seguridad mediante JWT y OAuth2 Resource Server

La arquitectura de seguridad del sistema se fundamenta en el uso de **Spring Security**, configurado específicamente como un servidor de recursos OAuth2 (*Resource Server*). Esta configuración permite delegar la responsabilidad de la validación de identidad al framework, asegurando un procesamiento automatizado y estandarizado de los tokens de acceso en cada petición.

- **Esquema de autenticación:** Se ha implementado un mecanismo de autenticación basado en el estándar **JSON Web Token (JWT)** de tipo *Bearer*. Siguiendo las buenas prácticas de seguridad en APIs REST, el cliente debe incluir este token en la cabecera HTTP **Authorization** para que el servidor pueda procesar la solicitud.

- **Validación de Claims:** Durante el proceso de decodificación del token, el sistema realiza una extracción y validación obligatoria de atributos clave o *claims*, que son fundamentales para la operatividad del entorno multi-tenant:
 - **tenant_id:** Identificador crítico que permite al sistema vincular dinámicamente la sesión con el esquema de base de datos correspondiente al centro de día.
 - **roles:** Atributos necesarios para la aplicación del control de acceso basado en roles (**RBAC**), determinando los permisos de ejecución del usuario sobre los distintos recursos.
 - **id de usuario:** Información indispensable para las tareas de trazabilidad y los procesos de auditoría automática implementados en la plataforma.

5.1.9.5. Gestión de Claves y Seguridad Criptográfica

La robustez de los tokens JWT se garantiza mediante una configuración de seguridad avanzada en el servidor.

- **Almacenamiento:** La clave secreta se gestiona de forma centralizada en el archivo **application.properties**, utilizando cadenas de alta entropía.
- **Algoritmo de Firma:** Se emplea **HMAC-SHA256** para asegurar la autenticidad e integridad de los tokens.
- **Codificación:** Se ha implementado una decodificación en **BASE64** para la lectura de la clave, corrigiendo errores comunes de interpretación en UTF-8 y cumpliendo con las especificaciones técnicas de la librería **jjwt**.

5.1.9.6. Sistema Dual de Autenticación (Workers vs. Admins)

La plataforma diferencia claramente entre el personal operativo de los centros y los administradores del SaaS.

- **Tokens para Workers:** Incluyen un array de roles específicos (ejemplo: **ROLE_DIRECTOR**), el identificador del esquema (**tenant_id**) y el plan contratado.
- **Tokens para Admins:** Carecen de vinculación a un esquema específico y poseen un claim de rol global para la gestión de la infraestructura.
- **Normalización:** El componente **JwtAuthenticationConverter** estandariza ambos formatos, permitiendo que Spring Security evalúe los permisos de manera unificada independientemente del origen del usuario.


```
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(jwt -> {
        List<GrantedAuthority> authorities = new ArrayList<>();

        Object roles = jwt.getClaim("roles");
        if (roles instanceof List<?> rolesList) {
            rolesList.stream()
                .filter(r -> r instanceof String)
                .map(r -> new SimpleGrantedAuthority((String) r))
                .forEach(authorities::add);
        }

        String role = jwt.getClaim("role");
        if (role != null && !role.isBlank()) {
            authorities.add(new SimpleGrantedAuthority("ROLE_" + role));
        }

        return authorities;
    });

    return converter;
}
```

Ilustración - Normalización de autoridades mediante JwtAuthenticationConverter.

5.1.9.7. Flujo de Ejecución de la Petición Multi-tenant

Cada solicitud al backend sigue un flujo de control estricto para garantizar el aislamiento:

- **Validación:** El filtro JWT comprueba la firma y establece la identidad en el **SecurityContext**.
- **Extracción:** El **TenantFilter** recupera el identificador del esquema del token y lo inyecta en el **TenantContext**.
- **Resolución:** Hibernate invoca al **TenantIdentifierResolver** para determinar qué esquema debe consultarse.
- **Conexión:** El proveedor de conexiones JDBC ejecuta la instrucción **SET search_path TO tenant_{slug}, public** para posicionar la base de datos en el contexto correcto.
- **Finalización:** Tras emitir la respuesta, se ejecuta obligatoriamente el método **clear()** para liberar el hilo de cualquier referencia al tenant.

5.1.9.8. Prevención de Inyecciones SQL y Fallback de Seguridad

Se han implementado salvaguardas para proteger la integridad del motor de base de datos:

- **Validación de Slugs:** Se aplica una expresión regular estricta (**^[a-z0-9-]+\$**) que impide el uso de caracteres especiales en la selección de esquemas.
- **Mecanismo de Fallback:** En caso de recibir un identificador inválido o inexistente, el sistema redirige la conexión automáticamente al esquema **public**, denegando el acceso a cualquier dato operativo.

5.1.9.9. Protección contra Abuso (Rate Limiting)

```

12 import java.util.Map;
13 import java.util.concurrent.ConcurrentHashMap;
14
15 @Component
16 public class LoginRateLimiter {
17
18     private static final Logger log = LoggerFactory.getLogger(LoginRateLimiter.class);
19
20     private static final int MAX_ATTEMPTS = 5;
21
22     private static final Duration REFILL_DURATION = Duration.ofMinutes(minutes: 15);
23
24     private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();
25
26     public void checkLimit(String ip) {
27         Bucket bucket = buckets.computeIfAbsent(ip, this::newBucket);
28
29         if (!bucket.tryConsume(1)) {
30             log.warn(format: "RATE LIMIT EXCEEDED ip={}", ip);
31             throw new TooManyRequestsException(
32                 message: "Demasiados intentos fallidos. Inténtalo en 15 minutos.");
33         }
34     }
35
36     private Bucket newBucket(String ip) {
37         Bandwidth limit = Bandwidth.builder()
38             .capacity(MAX_ATTEMPTS)
39             .refillGreedy(MAX_ATTEMPTS, REFILL_DURATION)
40             .build();
41         return Bucket.builder().addLimit(limit).build();
42     }
43
44     @Scheduled(fixedDelay = 3_600_000)
45     public void clearStaleEntries() {
46         int size = buckets.size();
47         buckets.clear();
48         log.info(format: "RATE_LIMITER_CLEANUP: {} entradas eliminadas", size);
49     }
50 }

```

Ilustración - Implementación del componente LoginRateLimiter mediante la librería Bucket4j.

Para mitigar ataques de fuerza bruta y asegurar la disponibilidad del servicio, se ha implementado el componente **LoginRateLimiter**.

- **Tecnología:** Uso de la librería **Bucket4j** para la gestión de cuotas de peticiones.
- **Política de Acceso:** Se restringe el acceso tras 5 intentos fallidos por dirección IP en un intervalo de 15 minutos, devolviendo un código de estado **429 (Too Many Requests)** cuando se supera el límite.

5.1.9.10. Control de Acceso Granular (RBAC)

La autorización se gestiona a nivel de método mediante la anotación **@PreAuthorize**.

- **Jerarquía de Roles:** Se definen permisos específicos para directores, enfermeros y técnicos de actividades socioculturales (TAS).
- **Lógica Compleja:** Se utiliza **SecurityService** junto con **SpEL (Spring Expression Language)** para validar reglas de negocio avanzadas, como verificar si un trabajador pertenece efectivamente al centro cuyos datos intenta consultar.

5.1.9.11. Auditoría Automática de Datos

La trazabilidad de la información se gestiona de manera transparente mediante **JPA Auditing**.

- **Entidad Base:** Todas las tablas heredan de **BaseEntity**, que proporciona campos automáticos de fecha y autor de creación y modificación.
- **Captura de Contexto:** El sistema intercepta el usuario autenticado desde el **SecurityContext** para persistir su identidad en cada operación de escritura, garantizando un registro de auditoría completo sin intervención manual del desarrollador.

5.1.9.12. Gestión de Migraciones con Flyway

El despliegue de cambios en la estructura de datos se realiza de forma automatizada y diferenciada:

- **Migraciones Globales:** Gestionan las tablas maestras en el esquema público.
- **Migraciones Programáticas:** El componente **TenantProvisioningService** ejecuta dinámicamente los scripts de Flyway sobre cada esquema de cliente recién creado, asegurando que todos los centros utilicen la misma versión de la base de datos.

5.1.9.13. Configuración del Contexto de Persistencia

Se ha desactivado la propiedad **spring.jpa.open-in-view** (OSIV) por motivos de seguridad y rendimiento.

- **Justificación:** Al cerrar la sesión de Hibernate al finalizar la transacción del servicio, se asegura que cualquier intento de acceso a datos fuera de los límites transaccionales no falle por una resolución de esquema ambigua o un **TenantContext** ya limpio.

5.1.9.14. Análisis de la Arquitectura: Ventajas y Limitaciones

- **Fortalezas:** El aislamiento por esquemas ofrece el nivel más alto de seguridad lógica en PostgreSQL, permitiendo además realizar copias de seguridad personalizadas por cliente.
- **Escalabilidad:** El sistema de aprovisionamiento automático permite dar de alta nuevos centros en segundos.
- **Consideraciones Futuras:** Ante un crecimiento masivo de clientes (decenas de miles), se prevé evaluar el uso de *Row Level Security* (RLS) para optimizar el rendimiento del catálogo del motor de base de datos.

5.2. Etapa 3: Diseño de la Interfaz y Desarrollo del Frontend

Esta etapa se centró en la creación de una interfaz moderna y que funcione bien, aprovechando las últimas tecnologías para ofrecer una experiencia de usuario confiable y ayude en la operativa diaria de un Centro.

5.2.1. Angular 21: Novedades y Enfoque

La decisión de utilizar Angular 21 se basa en su gran evolución técnica. Ha pasado de su versión original basada en **JavaScript (AngularJS)** hacia un framework moderno centrado en **TypeScript**, la programación reactiva y la optimización del rendimiento. Por ello, está pensado para que la aplicación cargue rápido y responda bien a las tareas que el trabajador realiza diariamente. Para lograr este objetivo, hemos rescatado la importancia de los siguientes puntos clave que detallamos a continuación:

5.2.1.1. Signals como Primitive Reactivo

A diferencia de las versiones anteriores, donde todo dependía casi exclusivamente de RxJS y del ciclo de detección de cambios de zone.js, Angular 21 introduce los **signals** como primitive reactivo de primera clase. Un **signal** es un contenedor de valor que avisa automáticamente a la aplicación cuando algo cambia, eliminando la necesidad de observables para el estado local del componente. Esto simplifica drásticamente la gestión de estado en aplicaciones de mediana complejidad, que en el caso de nuestra aplicación es fundamental para asegurar que la información de los usuarios se actualice al instante en pantalla sin errores de sincronización.

5.2.1.2. Standalone Components por Defecto

En Angular 21, los componentes son **standalone** de forma nativa. Esto significa que ya no es necesario declararlos dentro de un **NgModule**; cada componente importa directamente lo que necesita (otros componentes, directivas, pipes). Esto permite tener una arquitectura más limpia y directa. Al importar sólo lo que cada parte de la interfaz necesita, el código es más fácil de mantener y permite que la aplicación crezca de forma ordenada sin volverse lenta.

5.2.1.3. Built-in Control Flow

El framework ha reemplazado las directivas estructurales tradicionales como ***ngIf**, ***ngFor**, ***ngSwitch** por un control flow integrado en el propio lenguaje de plantillas: **@if**, **@for**, **@switch**. Además de una sintaxis más limpia, esta mejora proporciona optimizaciones de rendimiento a nivel de compilación que las directivas estructurales no podían alcanzar. Esto es especialmente útil cuando el personal del centro visualiza listas largas, como el cuadrante de transporte o el listado de tareas diarias.

5.2.1.4. Zoneless Change Detection

Angular 21 permite ejecutar aplicaciones sin depender de **zone.js**, la librería que tradicionalmente instrumentaba el navegador para detectar cambios. Al eliminarla, se reducen significativamente el tamaño del bundle y los ciclos de detección de cambios innecesarios, delegando la responsabilidad al desarrollador mediante **signals** y referencias inmutables. El resultado es una plataforma más ágil que no consume recursos de más, algo clave para que funcione bien en cualquier equipo del centro.

5.2.1.5. Inyección Funcional con inject()

El patrón de inyección de dependencias ha evolucionado hacia una forma funcional mediante la función **inject()**. Esto permite inyectar servicios en cualquier contexto de ejecución (constructores, funciones, guards, interceptores) sin depender de la sintaxis de clases, alineándose con las tendencias modernas de JavaScript.

5.2.2. Arquitectura del Proyecto (SaaS multi-tenant)

El sistema se ha diseñado siguiendo el patrón **SaaS multi-tenant**, donde cada centro de día funciona como un tenant o cliente independiente. La arquitectura del frontend refleja esta división mediante una separación estricta entre el área operativa (**/tenant/****) y el área de administración de la plataforma (**/admin/****).

5.2.2.1. Separación de Áreas

Área	Ruta	Usuario	Layout	Guards
Tenant	/tenant/**	Trabajadores del centro	ShellComponent	authGuard + tenantGuard
Admin	/admin/**	Superadministradores	AdminShellComponent	authGuard

Esta separación no es meramente visual. Cada área posee su propio layout, menú lateral, conjunto de guards y estrategia de lazy loading. El objetivo es evitar que un trabajador del centro acceda accidentalmente a funcionalidades de gestión de la plataforma, y viceversa.

5.2.2.2. Estructura de Directorios

```

TFGAngular - informe-tecnico-angular.md
1  src/app/
2  |— models/      # Interfaces TypeScript del dominio
3  |— services/    # Servicios HTTP (uno por recurso de API)
4  |— guards/      # Protección de rutas (autenticación y autorización)
5  |— interceptors/ # Transformación de peticiones HTTP
6  |— layout/      # Shells visuales (tenant, admin, auth, landing)
7  |— pages/       # Páginas organizadas por área
8  |   |— tenant/  # App operativa
9  |   |— admin/   # Administración SaaS
10 |— shared/      # Componentes reutilizables
11 |— utils/       # Helpers puros
  
```

Ilustración - Referencia de estructura

5.2.2.3. Lazy Loading Obligatorio

Para asegurar que la aplicación sea rápida, todas las páginas de las áreas **tenant/** y **admin/** se cargan mediante **Lazy Loading**. Esto divide la aplicación en pequeños fragmentos o **chunks** que solo se descargan cuando el usuario navega a una ruta específica. Gracias a esto, se reduce drásticamente el tiempo de carga inicial, ya que el navegador no tiene que descargar todo el sistema de golpe, sino solo lo que el usuario necesita en ese momento.

5.2.3. Flujo de Trabajo

Para que el desarrollo sea ordenado y fácil de mantener, el desarrollo del frontend sigue un flujo de datos unidireccional, que garantiza la separación de responsabilidades en cada etapa:

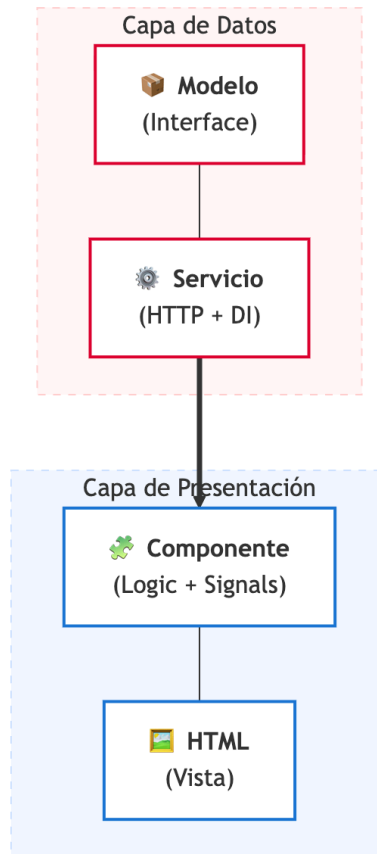


Ilustración - Representación del flujo de trabajo

- **Modelo:** Define la estructura de datos mediante interfaces TypeScript. Esto establece un contrato con el backend, asegurando que toda la información esté tipada en todas las operaciones.
- **Servicio:** Se encarga exclusivamente de la comunicación HTTP. Cada servicio es responsable de un único recurso de la API (usuarios, medicación, transporte), evitando mezclar responsabilidades.
- **Componente:** Consume el servicio y gestiona el estado local mediante signals. En esta etapa se evita colocar lógica de negocio en la plantilla, dejando la parte visual lo más limpia posible.
- **Plantilla HTML:** Declara la interfaz de usuario utilizando el control flow moderno (**@if**, **@for**) y bindings unidireccionales.

Este flujo asegura que cada capa del frontend tenga una única responsabilidad, facilitando el testing y el mantenimiento a largo plazo.

5.2.4. Explicación Modelo

Los modelos del proyecto se implementan como interfaces TypeScript en lugar de clases. Esta decisión es importante porque reduce el tamaño final de la aplicación y permite que el frontend esté alineado directamente con los DTOs que expone el backend en Java.

Para mantener el orden en las operaciones, utilizamos una estructura de tres modelos distintos para cada entidad:

```
TFGAngular - user.model.ts

1  export interface User {
2    readonly id: number
3    firstName: string
4    secondName: string | null
5    firstSurname: string
6    secondSurname: string | null
7    alias: string | null
8    email: string | null
9    phone: string | null
10   cellphone: string | null
11   dni: string
12   sexName: string | null
13   birthDate: string
14   dependencyName: string | null
15   isActive: boolean
16   readonly createdAt: string
17   readonly createdBy: string | null
18   updatedAt: string | null
19   updatedBy: string | null
20 }
```

***Ilustración** - Representación de modelo base*

```
TFGAngular - user.model.ts

1  export interface UserRequest {
2    firstName: string
3    secondName?: string | null
4    firstSurname: string
5    secondSurname?: string | null
6    alias?: string | null
7    email?: string | null
8    phone?: string | null
9    cellphone?: string | null
10   dni: string
11   sexId: number
12   birthDate: string
13   dependencyId: number
14 }
```

***Ilustración** - Representación de modelo request*

```
TFGAngular - user.model.ts
1  export interface UserUpdate {
2    firstName?: string | null
3    secondName?: string | null
4    firstSurname?: string | null
5    secondSurname?: string | null
6    alias?: string | null
7    email?: string | null
8    phone?: string | null
9    cellphone?: string | null
10   dni?: string | null
11   sexId?: number | null
12   birthDate?: string | null
13   dependencyId?: number | null
14   isActive?: boolean | null
15 }
```

Ilustración - Representación de modelo update

- **User:** Representa la entidad completa tal como se recibe del backend. Definimos los campos como **id**, **createdAt** y **createdBy** con la propiedad **readonly**, asegurando que estos datos que genera el servidor nunca deben ser mutados desde el cliente.
- **UserRequest:** Modelo DTO que utilizamos para la creación de un nuevo registro de usuario. Todos los campos son obligatorios excepto aquellos marcados con “?”, que el backend los acepta como **null**.
- **UserUpdate:** Modelo DTO para la actualización parcial mediante peticiones **PATCH**. Todos los campos son opcionales, permitiendo enviar solo los valores que han sido modificados por el usuario de la aplicación.

Esta metodología de trabajo (**entidad + request + update**) se replica en los 57 modelos del proyecto. Gracias a esto, se asegura que cada operación de lectura o escritura esté correctamente tipada, evitando errores de datos en toda la plataforma.

5.2.5. Explicación de un Servicio

Los servicios son los encargados de manejar toda la comunicación con el servidor. Siguen el sistema moderno de inyección de dependencias mediante la función **inject()** y cada uno está especializado en un único recurso de la API, lo que ayuda a que el código sea mucho más ordenado y fácil de seguir.

```

TFGAngular - informe-tecnico-angular.md
1 @Injectable({ providedIn: 'root' })
2 export class AuthService {...}
  
```

Ilustración - Representación de servicio

```

TFGAngular - informe-tecnico-angular.md
1 private http = inject(HttpClient)
2 private baseUrl = `${environment.apiUrl}/api/auth`
  
```

Ilustración - Atributos de servicio

```

TFGAngular - informe-tecnico-angular.md
1 login(data: LoginRequest): Observable<ApiResponse<AuthData>> {
2   return this.http.post<ApiResponse<AuthData>>(`${this.baseUrl}/login`, data)
3 }
4
5 register(data: RegisterRequest): Observable<ApiResponse<AuthData>> {
6   return this.http.post<ApiResponse<AuthData>>(`${this.baseUrl}/register`, data)
7 }
8
9 saveToken(token: string): void {
10   localStorage.setItem('auth_token', token)
11 }
12
13 getToken(): string | null {
14   return localStorage.getItem('auth_token')
15 }
  
```

Ilustración - Métodos de servicio

- **inject(HttpClient):** Utilizamos la inyección funcional para conectar el cliente HTTP. Al no tener que utilizar constructores, el código es más corto y limpio, permitiendo usar los servicios en cualquier parte del sistema.
- **environment.apiUrl:** La dirección del servidor se centraliza en un archivo de configuración. Esto es muy útil porque permite cambiar entre el entorno de pruebas y el servidor real del centro sin tener que modificar cada servicio por separado.
- **Almacenamiento del token:** Se utiliza **localStorage** del navegador para guardar el token de seguridad (JWT). Esto asegura que la sesión se mantenga activa aunque el trabajador refresque la página o cierre el navegador por error.
- **Decodificación del JWT:** El método **getUserRoles()** extrae el **payload** del token y lo decodifica en base64 para obtener los claims sin necesidad de librerías externas.
- **Diferenciación de roles:** Mediante funciones como **isSuperAdmin()** e **isTenantUser()**, el frontend decide qué áreas del SaaS están disponibles para el usuario autenticado.

5.2.6. Componente y su plantilla

Los componentes del proyecto son standalone y utilizan signals para la gestión de estado. Se ha optado por **ChangeDetectionStrategy.OnPush** en todos los componentes, lo que significa que Angular solo reevalúa la plantilla cuando cambian las entradas (inputs) o los **signals**. Gracias a esto, la aplicación es mucho más eficiente y evitamos que el navegador trabaje de más innecesariamente.

5.2.6.1. PaginatedTableComponent

Para no repetir código en cada sección, se diseñó un componente mudo o **dumb component** que es totalmente reutilizable. Su función es centralizar toda la lógica de las tablas del sistema, incluyendo la paginación, el ordenamiento de columnas y los estados de carga.

```
TFGAngular - informe-tecnico-angular.md

1  @Component({
2    selector: 'app-paginated-table',
3    changeDetection: ChangeDetectionStrategy.OnPush,
4    imports: [NgTemplateOutlet],
5    templateUrl: './paginated-table.component.html',
6  })
7  export class PaginatedTableComponent {
8    readonly data = input.required<unknown[]>()
9    readonly columns = input.required<TableColumn[]>()
10   readonly isLoading = input(false)
11   readonly error = input<string | null>(null)
12   readonly currentPage = input(0)
13   readonly totalElements = input<number | null>(null)
14   readonly pageSize = input(10)
15   readonly emptyMessage = input('No se encontraron registros')
16   readonly pageChange = output<number>()
17   readonly rowClick = output<unknown>()
18   readonly customCell = contentChild<TemplateRef<unknown>>('customCell')
19
20   readonly showPagination = computed(() => {
21     const total = this.totalElements()
22     return total !== null && total > this.pageSize()
23   })
24
25   readonly paginationInfo = computed(() => {
26     const page = this.currentPage()
27     const size = this.pageSize()
28     const total = this.totalElements() ?? 0
29     const start = total === 0 ? 0 : page * size + 1
30     const end = Math.min((page + 1) * size, total)
31     return { start, end, total }
32   })
33
34   onPrevPage(): void {
35     if (this.currentPage() > 0) {
36       this.pageChange.emit(this.currentPage() - 1)
37     }
38   }
39
40   onNextPage(): void {
41     this.pageChange.emit(this.currentPage() + 1)
42   }
43 }
```

Ilustración - Componente PaginatedTable TypeScript

```

TFGAngular - informe-tecnico-angular.md

1  @if (isLoading()) {
2    <div class="bg-white rounded-lg shadow-sm border border-gray-200 p-6">
3      <p class="text-gray-500">Cargando...</p>
4    </div>
5  } @else if (data().length === 0) {
6    <div class="bg-white rounded-lg shadow-sm border border-gray-200 p-6 text-center">
7      @if (error()) {
8        <p class="text-sm text-red-700">{{ error() }}</p>
9      } @else {
10       <p class="text-gray-500">{{ emptyMessage() }}</p>
11     }
12   </div>
13 } @else {
14   <div class="bg-white rounded-lg shadow-sm border border-gray-200 overflow-hidden">
15     <table class="w-full">
16       <thead class="bg-gray-50 border-b border-gray-200">
17         <tr>
18           @for (col of columns(); track col.key) {
19             <th>{{ col.header }}</th>
20           }
21         </tr>
22       </thead>
23       <tbody class="divide-y divide-gray-200">
24         @for (row of data(); track $index) {
25           <tr>
26             @for (col of columns(); track col.key) {
27               <td>
28                 @if (customCell()) {
29                   <ng-container *ngTemplateOutlet="customCell"; context: { $implicit: row, key: col.key }" />
30                 } @else {
31                   {{ getCellValue(row, col.key) }}
32                 }
33               </td>
34             }
35           </tr>
36         }
37       </tbody>
38     </table>
39     @if (showPagination()) {
40       <div class="flex items-center justify-between px-4 py-3 border-t border-gray-200">
41         <button (click)="onPrevPage()" [disabled]="isPrevDisabled()">Anterior</button>
42         <span>Mostrando {{ paginationInfo().start }} - {{ paginationInfo().end }} de {{ paginationInfo().total }}</span>
43         <button (click)="onNextPage()" [disabled]="isNextDisabled()">Siguiente</button>
44       </div>
45     }
46   </div>
47 }

```

Ilustración - Componente PaginatedTable HTML

- **input.required<>():** Define las entradas de datos obligatorias. Si el componente padre no proporciona data o columns, Angular lanza un error en tiempo de ejecución, evitando que el usuario vea una interfaz rota.
- **output<>():** Define eventos que el componente emite hacia arriba para avisar cuando ocurre algo, por ejemplo, **pageChange** cuando el usuario cambia de página.
- **computed():** Realiza cálculos reactivos automáticos. En **showPagination** se recalcula automáticamente cuando cambia **totalElements** o **pageSize**, sin necesidad de lógica imperativa.
- **contentChild():** Permite la proyección de contenido. El componente padre puede inyectar un **ng-template** personalizado para renderizar celdas específicas, haciendo al componente altamente flexible.
- **Control flow @if/@for:** Reemplaza a ***ngIf** y ***ngFor**, ofreciendo mejor rendimiento y sintaxis más limpia.

5.2.7. Funcionamiento de JWT

La autenticación y autorización del sistema se basan en **JSON Web Tokens (JWT)**. El flujo completo involucra cuatro piezas clave del frontend.

5.2.7.1. Almacenamiento: AuthService

Tras un login exitoso, el backend devuelve un JWT que el frontend almacena en **localStorage** mediante **saveToken()**. Este token persiste entre recargas de página.

5.2.7.2. Decodificación: jwt.helper.ts

```
TFGAngular - informe-tecnico-angular.md

1  export interface JwtPayload {
2    id: number
3    firstName: string
4    firstSurname: string
5    sub: string
6    roles: string[]
7    iat: number
8    exp: number
9  }
10
11 export function decodeJwtPayload(token: string): JwtPayload | null {
12   try {
13     const parts = token.split('.')
14     if (parts.length !== 3) return null
15     const payload = atob(parts[1])
16     return JSON.parse(payload) as JwtPayload
17   } catch {
18     return null
19   }
20 }
```

Ilustración - Decodificación del JWT

Esta función pura decodifica el payload del JWT sin verificar la firma (la verificación es responsabilidad del backend). Extrae claims como **roles**, **id** y **exp** (expiración).

5.2.7.3. Interceptor: auth.interceptor.ts

```
TFGAngular - informe-tecnico-angular.md
1  export const authInterceptor: HttpInterceptorFn = (req, next) => {
2    const authService = inject(AuthService)
3    const token = authService.getToken()
4
5    if (token) {
6      const cloned = req.clone({
7        headers: req.headers.set('Authorization', `Bearer ${token}`)
8      })
9      return next(cloned)
10   }
11
12   return next(req)
13 }
```

Ilustración - Intercepción de solicitudes

Cada petición HTTP que sale del frontend pasa por este interceptor. Si existe un token, se añade automáticamente la cabecera **Authorization: Bearer <token>**. Esto evita que cada servicio tenga que gestionar manualmente la autenticación.

5.2.7.4. Protección de Rutas: Guards

```
TFGAngular - informe-tecnico-angular.md
1  export const authGuard: CanActivateFn = () => {
2    const authService = inject(AuthService)
3    const router = inject(Router)
4
5    if (authService.isLoggedIn()) {
6      return true
7    }
8
9    return router.createUrlTree(['/login'])
10 }
```

Ilustración - Guard de routes

- **authGuard**: Bloquea cualquier ruta si el usuario no tiene un token válido, redirigiendo al login.
- **tenantGuard**: Permite el acceso a **/tenant/**** solo si el usuario es un trabajador del centro (no superadmin).
- **adminGuard**: Permite el acceso a **/admin/**** solo si el usuario posee el rol **ROLE_SUPERADMIN**.

5.2.7.5. Flujo

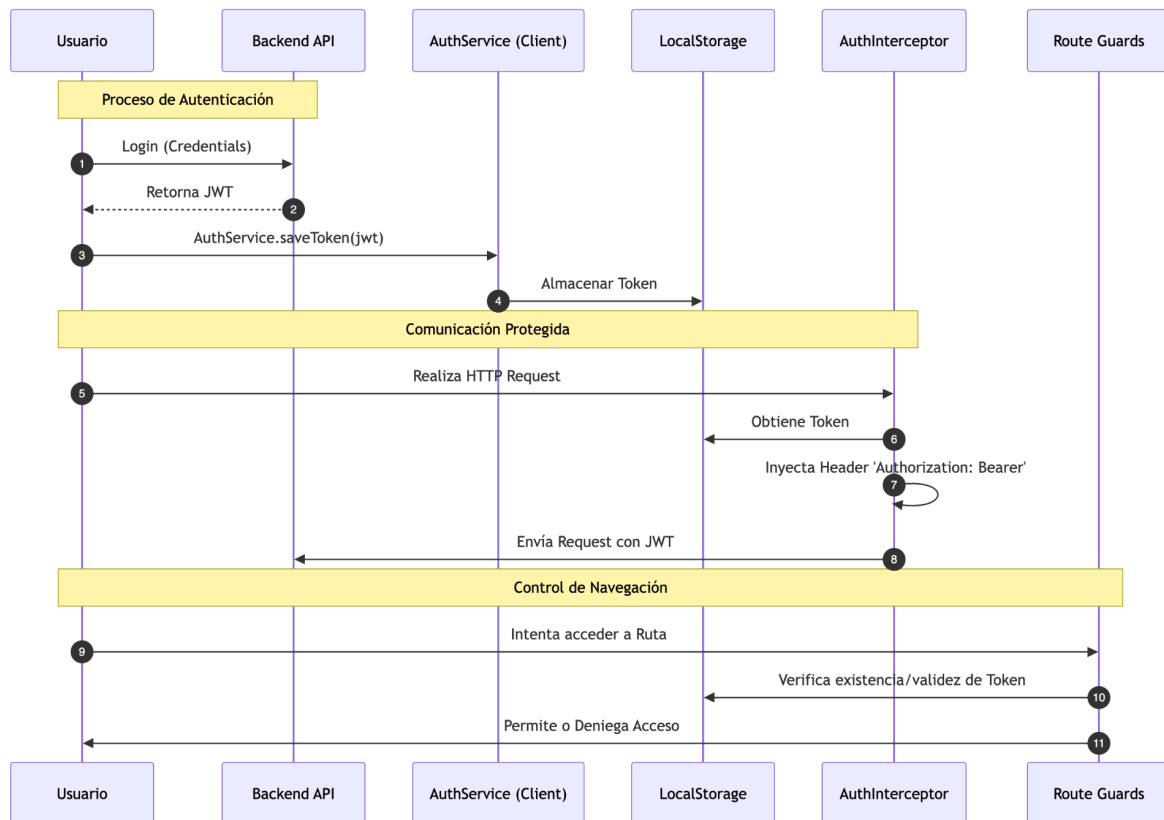


Ilustración - Flujo de funcionamiento para el JWT

5.2.8. Shared Components

Los shared components son piezas de **interfaz dumb**: no contienen lógica de negocio ni conocen el estado global de la aplicación. Su única responsabilidad es recibir datos mediante **input()**, emitir eventos mediante **output()** y renderizar la interfaz.

5.2.8.1. Relevancia Arquitectónica

- **Principio DRY (Don't Repeat Yourself):** Un componente como **PaginatedTableComponent** se utiliza en múltiples módulos (usuarios, transporte, incidencias) sin duplicar código.
- **Consistencia visual:** Todos los catálogos utilizan **GenericCatalogEditor**, garantizando que la experiencia de edición sea idéntica en toda la aplicación.
- **Aceleración del desarrollo:** Al componer páginas a partir de bloques reutilizables, el desarrollo de nuevas funcionalidades se reduce a ensamblar componentes existentes.

5.2.8.2. Componentes compartidos del proyecto

Componente	Propósito
PaginatedTableComponent	Tabla genérica con paginación, ordenamiento y estados de carga/error
PageHeaderComponent	Encabezado de página con título y botón de acción principal
SearchFilterComponent	Barra de búsqueda con debounce para filtrado en tiempo real
ModalComponent	Contenedor genérico de diálogos modales
DeleteConfirmationModalComponent	Modal estandarizado para confirmar eliminaciones
GenericCatalogEditorComponent	Formulario dumb de un solo campo para catálogos simples
CatalogCrudTabComponent	Tab genérica con tabla + editor para páginas de catálogos
AddressFormComponent/AddressCardComponent	Formulario y visualización de direcciones
ContactFormComponent/ContactCardComponent	Formulario y visualización de contactos
BathroomTaskEditorComponent	Formulario específico para tareas de baño
OpenDayEditorComponent	Formulario para días de apertura del centro

5.2.9. Sistema de Rutas

El routing de la aplicación se define en **app.routes.ts** y sigue una estructura jerárquica que refleja la arquitectura SaaS.

```
TFGAngular - informe-tecnico-angular.md

1 export const routes: Routes = [
2   {
3     path: '',
4     component: LandingLayoutComponent,
5     canActivate: [landingGuard],
6     children: [{ path: '', component: LandingPageComponent }],
7   },
8   {
9     path: 'login',
10    component: AuthLayoutComponent,
11    canActivate: [guestGuard],
12    children: [{ path: '', component: LoginComponent }],
13  },
14  {
15    path: 'admin-login',
16    component: AuthLayoutComponent,
17    canActivate: [guestGuard],
18    children: [
19      {
20        path: '',
21        loadComponent: () =>
22          import('./pages/admin-login/admin-login.component').then(
23            (m) => m.AdminLoginComponent
24          ),
25      },
26    ],
27  },
28  {
29    path: 'cd',
30    redirectTo: 'tenant',
31    pathMatch: 'prefix',
32  },
33  {
34    path: 'tenant',
35    component: ShellComponent,
36    canActivate: [authGuard, tenantGuard],
37    loadChildren: () =>
38      import('./pages/tenant/tenant.routes').then((m) => m.tenantRoutes),
39  },
40  {
41    path: 'admin',
42    component: AdminShellComponent,
43    canActivate: [authGuard, adminGuard],
44    loadChildren: () =>
45      import('./pages/admin/admin.routes').then((m) => m.adminRoutes),
46  },
47  {
48    path: '**',
49    redirectTo: '',
50  },
51 ]
```

Ilustración - Mapeo de Rutas con lazy y eager loading

- **Rutas públicas:** landing y login utilizan **eager loading** porque deben estar disponibles inmediatamente.
- **Lazy loading:** tenant y admin cargan sus rutas hijas mediante **loadChildren**, dividiendo el bundle en chunks separados.
- **Guards combinados:** canActivate: [authGuard, tenantGuard] garantiza que ambas condiciones se cumplan antes de permitir la navegación.
- **Retro-compatibilidad:** La ruta /cd redirige permanentemente a /tenant, preservando bookmarks de versiones anteriores.
- **Wildcard:** Cualquier ruta no definida redirige a la **landing page**.

5.2.10. Conclusión Desarrollo Angular

El frontend del sistema SaaS ha sido construido sobre Angular 21, aprovechando sus capacidades más modernas: signals para la reactividad, componentes standalone para una arquitectura plana, y control flow integrado para un rendimiento superior.

La decisión de adoptar una arquitectura SaaS multi-tenant con separación estricta entre el área operativa (**/tenant**) y el área de administración (**/admin**) responde directamente a los requisitos del proyecto: un único despliegue que sirva a múltiples centros de día, cada uno con sus propios usuarios, datos y permisos.

El uso de JWT para la autenticación, combinado con interceptores HTTP y guards de rutas, proporciona un mecanismo robusto y escalable de seguridad. Los shared components y el flujo de trabajo modelo-servicio-componente garantizan un código mantenible, testeable y libre de redundancias.

En conjunto, estas decisiones arquitectónicas y tecnológicas sientan las bases para una plataforma extensible, capaz de incorporar nuevos módulos (billing, analytics, gestión avanzada de tenants) sin comprometer la estabilidad del sistema existente.

5.3. Etapa 4: Contenedorización con Docker y Despliegue del Sistema

Dockerizar una aplicación (tanto el backend como el frontend) se ha convertido en el estándar de la industria ya que elimina el "en mi máquina sí funciona".

Al empaquetar el código con todas las dependencias en un contenedor, nos aseguramos que el software se comporte exactamente igual en cualquier computadora, en el servidor de pruebas y en la nube.

- **Docker resuelve esto:** Define el sistema operativo, las bibliotecas y las variables de entorno en un Dockerfile, aislando la lógica de negocio, la base de datos y los sistemas de caché sin que las librerías choquen entre sí.
- **Resultado:** Si corre en Docker, correrá en cualquier lugar que tenga Docker instalado.

5.3.1. Backend

```
TFG - Dockerfile

1  # Etapa 1: Construcción
2  FROM maven:3.9-eclipse-temurin-21 AS builder
3  WORKDIR /app
4  # Se copia el pom.xml primero para aprovechar la caché de Docker
5  COPY pom.xml .
6  RUN mvn dependency:go-offline
7  # Se copia el código fuente y se compila el proyecto
8  COPY src ./src
9  RUN mvn clean package -DskipTests
10
11 # Etapa 2: Ejecución (Contenedor final súper ligero)
12 FROM eclipse-temurin:21-jre-alpine
13 WORKDIR /app
14 # Se copia únicamente el .jar compilado de la etapa anterior
15 COPY --from=builder /app/target/*.jar app.jar
16 # Se expone el puerto que indicaste
17 EXPOSE 8080
18 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Ilustración - Dockerfile Backend

Este Dockerfile es un **multi-stage build** que se compone de dos etapas:

- **Etapa 1:** En esta fase se prepara el entorno necesario para *compilar* el código fuente de Java (**Spring Boot/Jakarta EE**) usando **Maven**.
 - **FROM maven:3.9-eclipse-temurin-21 AS builder:** Define la imagen base que contiene el JDK 21 y Maven 3.9. Se le asigna el alias **builder** para poder referenciarla más adelante.
 - **WORKDIR /app:** Define el directorio de trabajo dentro del contenedor donde ocurrirán los comandos.
 - **COPY pom.xml y RUN mvn dependency:go-offline:** Estos pasos son importantes para la **optimización de caché**. Al copiar primero solo el archivo de dependencias y descargarlas, Docker no volverá a descargar las librerías a menos que el archivo **pom.xml** cambie, acelerando significativamente las futuras construcciones.
 - **COPY src ./src:** Copia el código fuente del proyecto.

- **RUN mvn clean package -DskipTests**: Ejecuta la compilación y empaquetado del proyecto. El resultado es un archivo **.jar** en la carpeta **/target**. Se saltan los tests (**-DskipTests**) para agilizar el despliegue.
- **Etapa 2**: Esta es la imagen que finalmente se despliega en el servidor.
 - **FROM eclipse-temurin:21-jre-alpine**: Cambia la base a una imagen mucho más pequeña que solo contiene el **JRE (Java Runtime Environment)** y está basada en **Alpine Linux** (una distribución mínima). No incluye Maven ni el compilador, lo que reduce la superficie de ataque y el peso de la imagen (de **~600MB** a **~150MB** aprox.).
 - **WORKDIR /app**: Establece el directorio de trabajo limpio para la ejecución.
 - **COPY --from=builder /app/target/*.jar app.jar**: Esta es la instrucción clave del *multi-stage*. Copia el artefacto (**.jar**) generado en la etapa anterior (**builder**) hacia la nueva imagen limpia. Todo lo demás (código fuente, caché de Maven, etc.) se descarta.
 - **EXPOSE 8080**: Informa que el contenedor escuchará en el puerto 8080 (puerto por defecto de Spring Boot).
 - **ENTRYPOINT ["java", "-jar", "app.jar"]**: Define el comando principal que se ejecutará al iniciar el contenedor.

5.3.2. Frontend

El siguiente **Dockerfile** para el frontend sigue la misma filosofía de *multi-stage build* que el del **backend**, pero adaptado al ecosistema de **Node.js** y **Angular**.

Este archivo está diseñado para transformar el código fuente de Angular en archivos estáticos (**HTML**, **CSS**, **JS**) y servirlos de la manera más eficiente posible.

```

TFGAngular - Dockerfile

1  # Etapa 1: Construcción
2  FROM node:24.15.0-alpine AS builder
3  WORKDIR /app
4  COPY package*.json ./
5  # Usamos npm ci para una instalación más limpia y rápida basada en el package-lock.json
6  RUN npm ci
7  COPY . .
8  RUN npm run build -- --configuration production
9
10 # Etapa 2: Servidor (Nginx)
11 FROM nginx:alpine
12 # Copiamos nuestra configuración de rutas
13 COPY nginx.conf /etc/nginx/conf.d/default.conf
14
15 # Puedes verificar este nombre mirando la carpeta que se genera dentro de /dist al compilar
16 COPY --from=builder /app/dist/TFGAngular/browser /usr/share/nginx/html
17
18 EXPOSE 80
19 CMD ["nginx", "-g", "daemon off;"]
  
```

Ilustración - Dockerfile Frontend

- **Etapa 1**: Esta fase se encarga de compilar el proyecto Angular.
 - **FROM node:24.15.0-alpine AS builder**: Utiliza una imagen ligera de **Node.js** sobre **Alpine Linux**. El alias **builder** permite extraer los archivos compilados más adelante.
 - **WORKDIR /app**: Define el directorio raíz donde se ejecutará la construcción.

- **COPY package*.json ./ y RUN npm ci:**
 - **Estrategia de Caché:** Al copiar primero los archivos de dependencias, **Docker** evita reinstalar todos los módulos de **Node** si solo cambiaste el código fuente y no las librerías.
 - **npm ci:** Es preferible sobre **npm install** en entornos de **CI/CD**, ya que es más rápido, estricto con las versiones del **package-lock.json** y garantiza instalaciones limpias.
- **COPY . .:** Copia todo el código del frontend al contenedor.
- **RUN npm run build -- --configuration production:** Ejecuta el compilador de **Angular** (AOT - Ahead of Time). Esto genera la carpeta **/dist** con el código minificado y optimizado para producción.
- **Etapas 2:** Una vez compilado, el frontend ya no necesita Node.js. Solo necesita un servidor de archivos estáticos.
 - **FROM nginx:alpine:** Utiliza **Nginx**, el estándar de oro para servir aplicaciones de una sola página (SPA). Es extremadamente ligero y rápido.
 - **COPY nginx.conf /etc/nginx/conf.d/default.conf:** Paso crucial. Se sobrescribe la configuración por defecto de **Nginx** con una configuración personalizada.
 - **COPY --from=builder /app/dist/TFGAngular/browser /usr/share/nginx/html:** Extrae el resultado de la Etapa 1 y lo coloca en la carpeta pública de **Nginx**.
 - **EXPOSE 80:** Abre el puerto estándar de **HTTP**.
 - **CMD ["nginx", "-g", "daemon off;"]:** Inicia **Nginx** en primer plano para que el contenedor no se detenga inmediatamente.

5.3.3. Orquestación

Este archivo **Docker Compose** es el "director de orquesta". Su función es coordinar los tres contenedores (**Base de datos, Backend y Frontend**) para que trabajen como un único sistema interconectado.

```
undefined - docker-compose.yaml
1  services:
2    postgres-db:
3      image: postgres:18
4      container_name: centro_dia_db
5      restart: always
6      env_file:
7        - .env
8      environment:
9        - POSTGRES_DB=${DB_NAME}
10       - POSTGRES_USER=${DB_USER}
11       - POSTGRES_PASSWORD=${DB_PASS}
12     ports:
13       - "5434:5432"
14     volumes:
15       - db_data:/var/lib/postgresql
16     networks:
17       - app-network
18
19   backend:
20     build: ./TFG
21     container_name: springboot_backend
22     ports:
23       - "8080:8080"
24     depends_on:
25       - postgres-db
26     env_file:
27       - .env
28     environment:
29       - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres-db:5432/${DB_NAME}
30       - SPRING_DATASOURCE_USERNAME=${DB_USER}
31       - SPRING_DATASOURCE_PASSWORD=${DB_PASS}
32     networks:
33       - app-network
34
35   frontend:
36     build: ./TFGAngular
37     container_name: angular_frontend
38     ports:
39       - "80:80"
40     depends_on:
41       - backend
42     networks:
43       - app-network
44
45   volumes:
46     db_data:
47
48   networks:
49     app-network:
50       driver: bridge
```

Ilustración - docker-compose.yaml

Docker Compose permite definir una infraestructura *multi-contenedor* mediante un archivo **YAML**. En tu proyecto, gestiona el ciclo de vida, las redes y los volúmenes de todo el stack.

- **Servicio: postgres-db (Base de Datos):** Es el motor de persistencia de datos.
 - **image: postgres:18:** Utiliza la imagen oficial de PostgreSQL.
 - **env_file: - .env:** Lee variables sensibles (como contraseñas) de un archivo externo, lo cual es una **buena práctica de seguridad** para no exponer credenciales en el código.
 - **ports: "5434:5432":** Mapea el puerto 5432 del contenedor al 5434 de tu máquina física. Esto evita conflictos si ya tienes otro Postgres instalado localmente.
 - **volumes: db_data:/var/lib/postgresql:** Define la **persistencia**. Sin esto, si el contenedor se borra, los datos desaparecen. El volumen **db_data** asegura que la información sobreviva a reinicios.
- **Servicio: backend (Spring Boot):** Es la lógica de negocio que conecta el cliente con los datos.
 - **build: ./TFG:** En lugar de usar una imagen pre-hecha, Docker construye la imagen usando el **Dockerfile** que está en esa carpeta (el que explicamos anteriormente).
 - **depends_on: - postgres-db:** Establece un orden de inicio. El backend esperará a que el contenedor de la base de datos esté "arriba" antes de intentar arrancar.
 - **SPRING_DATASOURCE_URL:** Aquí ocurre la "magia" de Docker. Nota que la URL es **jdbc:postgresql://postgres-db:5432/...**. Docker usa el nombre del servicio (postgres-db) como nombre de host (DNS interno), permitiendo que los contenedores se hablen entre sí fácilmente.
- **Servicio: frontend (Angular):** Es la interfaz de usuario.
 - **build: ./TFGAngular:** Construye la imagen del frontend usando su respectivo **Dockerfile**.
 - **ports: "80:80":** Expone la aplicación al mundo exterior a través del puerto estándar HTTP.
 - **depends_on: - backend:** Asegura que la API esté disponible antes de levantar el servidor web del frontend.
- **Componentes Globales de Infraestructura**
 - **networks: app-network:** Crea una red privada virtual llamada bridge. Todos los contenedores dentro de esta red pueden verse entre sí usando sus nombres de servicio, pero están aislados del tráfico externo que no pase por los puertos explícitamente mapeados.
 - **volumes: db_data:** Declara el espacio de almacenamiento persistente gestionado por Docker para la base de datos.

5.3.3.1. Comandos de ejecución:

- **docker-compose up --build:** Ejecutar la primera vez, luego quitar la flag **--build**.
- **docker-compose down:** Detener los servicios.
- **docker-compose down --rmi -v:** Detiene los servicios, elimina las imágenes construidas, elimina volúmenes de datos.

Una vez que **docker** se ha ejecutado, podemos acceder a la aplicación ingresando a **localhost** en el navegador-

IMPORTANTE: debe existir un archivo **.env** con las variables de **DB_NAME**, **DB_USER**, **DB_PASS** y **JWT_SECRET**.

6. GUÍA DE FUNCIONAMIENTO DEL SISTEMA

La plataforma se ha diseñado para integrarse de forma natural en el día a día del centro, permitiendo una experiencia de usuario ágil y sencilla para el personal, con el objetivo de que puedan gestionar toda la información de manera intuitiva, asegurando que el registro de datos sea rápido y sin errores, pero manteniendo siempre el orden y la seguridad que exige una arquitectura SaaS.

6.1. Acceso al Sistema y Autenticación

Como solución basada en un modelo SaaS, el acceso está centralizado y protegido para garantizar la integridad de cada centro (tenant). El proceso de entrada se divide en el portal de bienvenida y la posterior validación de identidad.

6.1.1. Página de Inicio (Landing Page)

El primer punto de contacto es un portal de bienvenida que presenta el propósito del software. Esta sección sirve como punto de entrada tanto para nuevos centros interesados en el servicio como para los trabajadores que ya disponen de una cuenta activa.

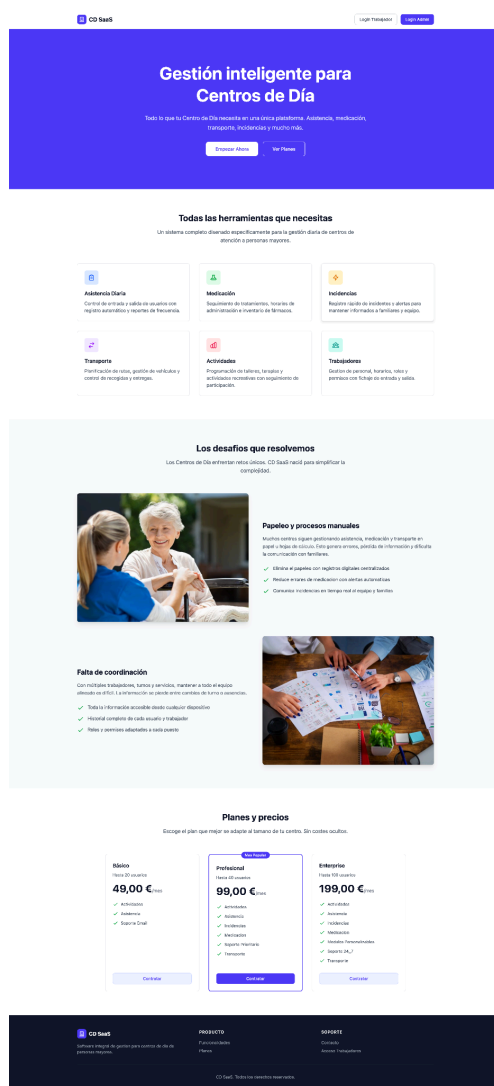


Ilustración - Interfaz de presentación de la plataforma SaaS.

Desde esta pantalla, el acceso al panel de gestión se realiza a través de los botones de **Login Trabajador** o **Login Admin**, que redirigen al usuario hacia el entorno seguro de autenticación.

6.1.2. Acceso y Validación de Credenciales

El sistema de autenticación actúa como el primer filtro de seguridad del software. Independientemente del perfil del usuario, el acceso se realiza a través de un formulario único donde se validan las credenciales para dirigir al personal hacia su entorno de trabajo correspondiente.

El flujo de acceso es sencillo y directo para asegurar que el personal no pierda tiempo en procesos complejos:

- **Introducción de Datos:** El usuario introduce su correo profesional y su clave secreta.
- **Validación de Identidad:** El sistema comprueba que el usuario pertenezca a un centro (tenant) activo y verifica sus permisos.
- **Carga de Entorno:** Dependiendo de si la cuenta es de nivel operativo o administrativo, la plataforma habilita los módulos específicos. Mientras el trabajador accede a las herramientas de seguimiento diario, el administrador obtiene el control total sobre la configuración del centro y la gestión de la plantilla.

Este diseño asegura que, aunque el punto de entrada sea el mismo, la experiencia esté totalmente personalizada según la responsabilidad de cada persona dentro del centro.

Ilustración - Interfaz de inicio de sesión estándar para el personal del centro

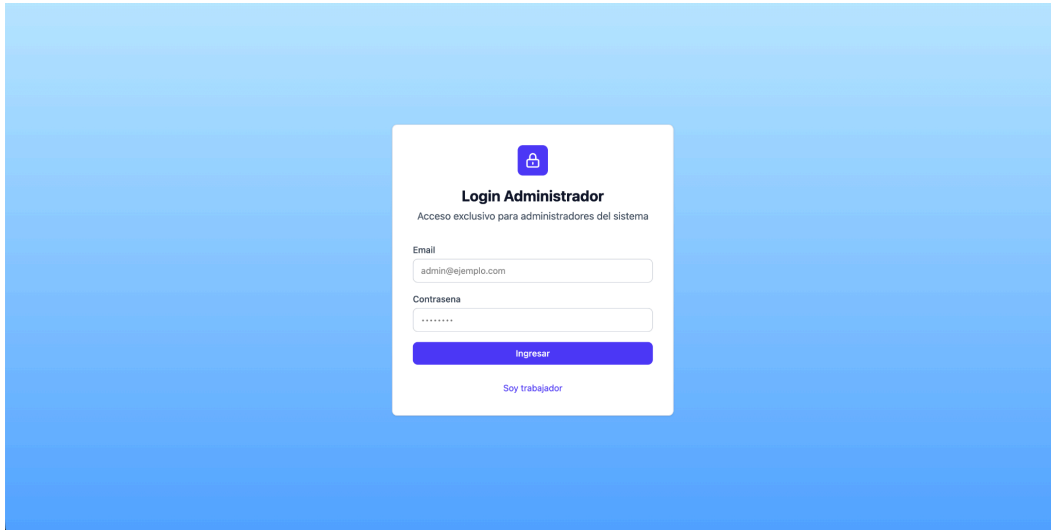


Ilustración - Interfaz de inicio de sesión estándar para administradores y nuevos centros.

6.2. Panel de Control y Navegación (Dashboard)

Tras completar la autenticación, el sistema redirige al trabajador al **Dashboard**, que funciona como el centro operativo de la actividad diaria. Esta interfaz ha sido diseñada para que la información más relevante esté siempre a la vista, permitiendo una gestión rápida y sin distracciones desde el primer momento.

La pantalla se organiza de forma clara en tres zonas principales para facilitar la navegación:

- **Menú Lateral (Sidebar):** Ubicado a la izquierda, permite moverse entre los diferentes módulos del sistema. Su estructura permite que el usuario siempre tenga acceso a todas las herramientas sin importar en qué sección se encuentre. Mientras que el trabajador visualiza opciones como incidencias o transporte, el administrador dispone de herramientas de gestión como clientes, planes y administradores.
- **Cabecera Global (Header):** Contiene las funciones que deben estar accesibles en todo momento. Destaca en el dashboard del trabajador el botón de "**Fichar Entrada**" en color verde, pensado para facilitar el registro de la jornada laboral de forma inmediata. Al activarse, este botón cambia de estado a "**Fichar Salida**" en color azul, permitiendo cerrar el turno de manera sencilla al finalizar la jornada. En la vista de administración, el header mantiene la identidad de la plataforma.
- **Área Central de Trabajo:** Es el espacio donde se desarrolla la actividad principal. El contenido de esta zona se adapta dinámicamente según las necesidades del perfil que accede. En la vista del trabajador el contenido está pensado para simplificar el seguimiento de los usuarios mediante un listado de incidencias y la operativa diaria. Por el contrario, en el perfil de administrador, el área se convierte en un panel de control que permite supervisar el estado de los centros y las métricas de control.

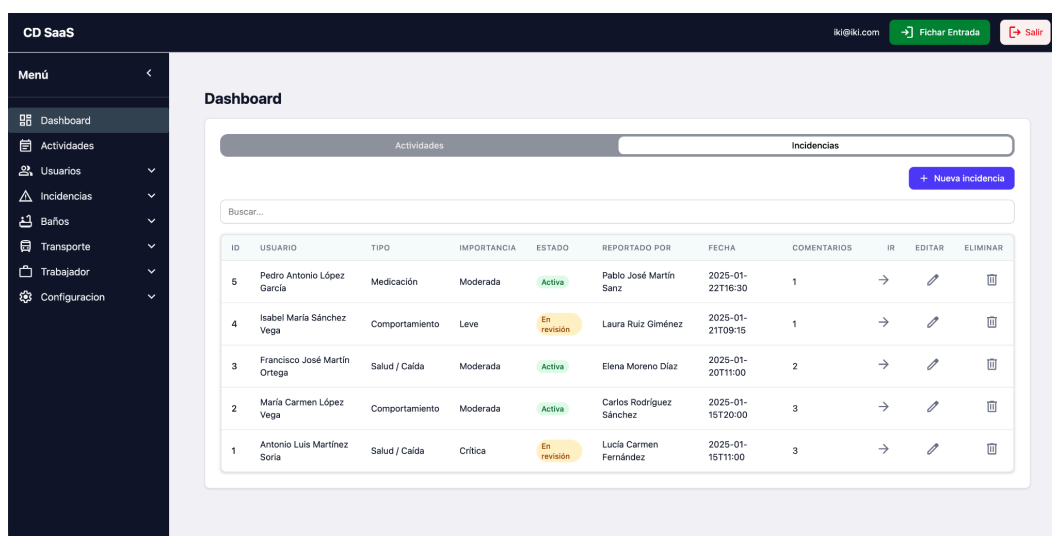


Ilustración - Vista operativa del panel de control central (Perfil Trabajador).

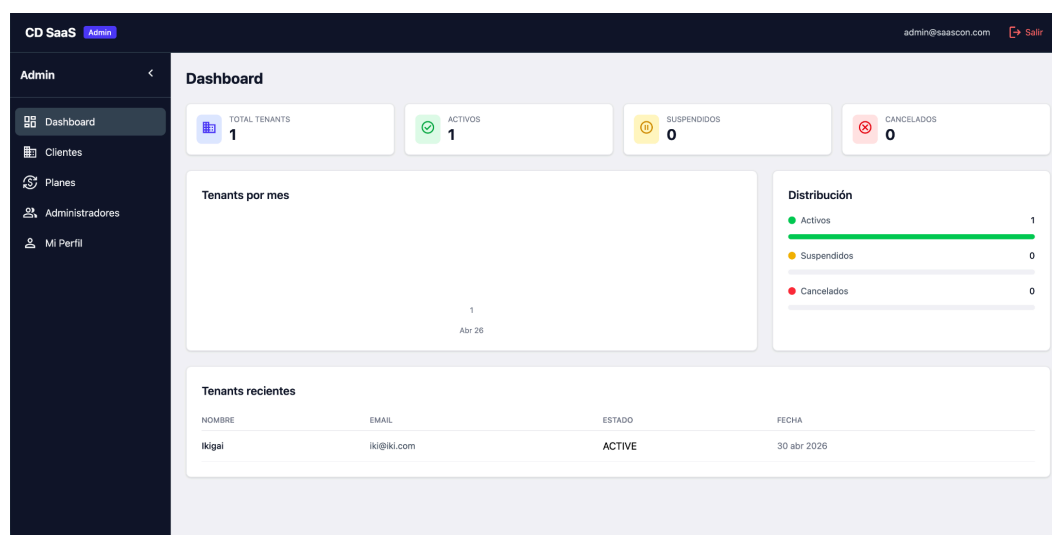


Ilustración - Panel de administración global del SaaS (Perfil Administrador).

6.3. Portal del Trabajador

Este portal es la herramienta principal para el personal, diseñada para que el seguimiento de los adultos mayores sea fluido y no suponga una carga administrativa adicional en su día a día.

6.3.1. Gestión de Actividades (Ampliación MVP)

La plataforma se ha proyectado con una mentalidad modular para permitir el crecimiento del centro. El módulo de Actividades centralizará la planificación de talleres, terapias y eventos recreativos. Actualmente, la interfaz sirve como una base preparada para recibir la lógica de negocio en futuras fases, asegurando que el diseño escale junto con las necesidades del servicio.

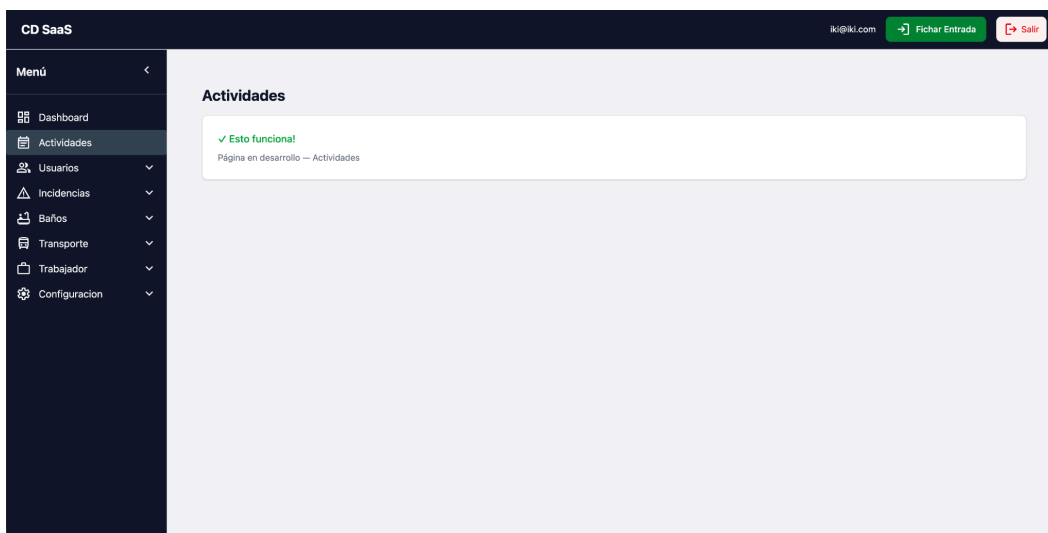


Ilustración - Interfaz base del módulo de Actividades.

6.3.2. Gestión de Usuarios

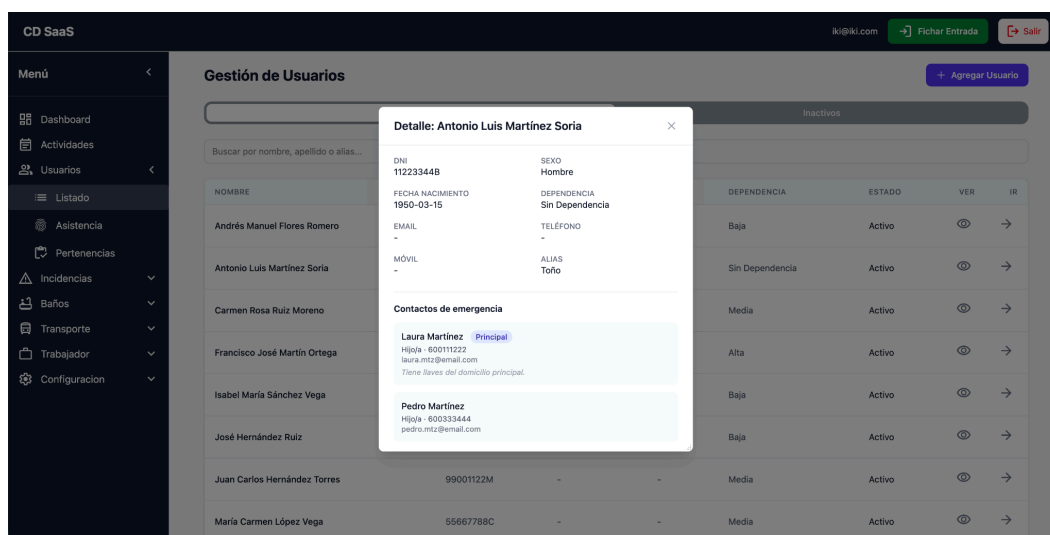
Es el módulo principal para la administración de los residentes del centro. Se ha diseñado para que el personal pueda gestionar tanto la información administrativa como el seguimiento de salud de manera rápida y organizada.

6.3.2.1. Listado y Consulta Rápida

La pantalla principal permite la gestión de todas las personas atendidas. El sistema incluye un buscador dinámico y filtros de estado para localizar perfiles rápidamente. Para situaciones que requieran una consulta inmediata, el botón de "**Ver**" abre un modal de detalle que muestra la información de contacto y familiares de emergencia sin necesidad de navegar a otra sección.

NOMBRE	DNI	TELÉFONO	EMAIL	DEPENDENCIA	ESTADO	VER	IR
Andrés Manuel Flores Romero	112233440	-	-	Baja	Activo	👁️	➔
Antonio Luis Martínez Soria	112233448	-	-	Sin Dependencia	Activo	👁️	➔
Carmen Rosa Ruiz Moreno	66778899J	-	-	Media	Activo	👁️	➔
Francisco José Martín Ortega	33445566G	-	-	Alta	Activo	👁️	➔
Isabel María Sánchez Vega	44556677H	-	-	Baja	Activo	👁️	➔
José Hernández Ruiz	99887766D	-	-	Baja	Activo	👁️	➔
Juan Carlos Hernández Torres	99001122M	-	-	Media	Activo	👁️	➔

Ilustración - Listado general de usuarios del centro.



***Ilustración** - Información de contactos de emergencia mediante ventana modal.*

6.3.2.2. Acceso al Expediente Completo del Usuario

Al pulsar la flecha de navegación en la columna "Ir", el sistema redirige al trabajador al expediente completo del usuario. Esta sección está organizada en pestañas para que la gestión de la información sea mucho más estructurada:

- **Perfil:** Recoge los datos personales, la dirección de residencia y los contactos familiares principales.
- **Asistencia:** Permite configurar los días y horarios en los que el usuario acude al centro.
- **Medicación:** Gestiona las pautas de fármacos con avisos visuales de stock bajo y fechas de caducidad próximas.
- **Enfermedades:** Incluye un apartado específico para tener siempre a mano el historial de diagnósticos clínicos activos.
- **Alergias:** Registro de sensibilidades y contraindicaciones para garantizar la seguridad del usuario.
- **Registros:** Permite consultar el historial de todas las asistencias confirmadas del usuario en el centro de día.

CD SaaS iki@iki.com Fichar Entrada Salir

Menú

- Dashboard
- Actividades
- Usuarios
 - Listado
 - Asistencia
 - Pertenencias
 - Incidencias
 - Baños
 - Transporte
 - Trabajador
 - Configuración

Usuario #1

Perfil Asistencia Medicación Enfermedades Alergias Registros

Información personal Editar Eliminar

NOMBRE	Antonio Luis Martínez Soria	DNI	112233448
SEXO	Hombre	FECHA NACIMIENTO	1950-03-15
DEPENDENCIA	Sin Dependencia	ALIAS	Toño
EMAIL	-	TELÉFONO	-
MÓVIL	-	ESTADO	Activo

Direcciones Agregar dirección

DIRECCIÓN	CP
Calle de la Libertad, 45, 3ª	28931
CIUDAD	PROVINCIA
Sevilla Capital	Almería
REGIÓN	ESTADO
Andalucía	Activo

Contactos de emergencia Agregar contacto

Ilustración - Datos personales del usuario y red de contactos de emergencia.

CD SaaS iki@iki.com Fichar Entrada Salir

Menú

- Dashboard
- Actividades
- Usuarios
 - Listado
 - Asistencia
 - Pertenencias
 - Incidencias
 - Baños
 - Transporte
 - Trabajador
 - Configuración

Usuario #1

Perfil **Asistencia** Medicación Enfermedades Alergias Registros

Días de asistencia programados Agregar día

DÍA	INICIO	FIN	ESTADO	ACCIONES
Lunes	09:00	17:00	Activo	✎ 🗑️
Martes	09:00	17:00	Activo	✎ 🗑️
Miércoles	09:00	17:00	Activo	✎ 🗑️
Jueves	09:00	17:00	Activo	✎ 🗑️
Viernes	09:00	17:00	Activo	✎ 🗑️

Ilustración - Panel de configuración de los días de asistencia semanal al centro.

CD SaaS iki@iki.com Fichar Entrada Salir

Menú

- Dashboard
- Actividades
- Usuarios
 - Listado
- Asistencia
- Pertenencias
- Incidencias
- Baños
- Transporte
- Trabajador
- Configuración

Usuario #1

Perfil Asistencia **Medicación** Enfermedades Alergias Registros

Medicación del usuario Agregar medicamento

MEDICAMENTO	DOSIS	RECEPCIÓN	CADUCIDAD ↑	STOCK	VÍA	ALMACENAMIENTO	VER	EDITAR	ELIMINAR
Acenocumarel (Sintrom)	4mg (Según pauta)	01-01-2025	01-06-2026	30	Oral	Temperatura Ambiente			
Omeprazol	20mg	01-01-2025	01-01-2027	60	Oral	Temperatura Ambiente			

***Ilustración** - Listado de medicación con alertas de falta de stock y caducidad.*

CD SaaS iki@iki.com Fichar Entrada Salir

Menú

- Dashboard
- Actividades
- Usuarios
 - Listado
- Asistencia
- Pertenencias
- Incidencias
- Baños
- Transporte
- Trabajador
- Configuración

Usuario #1

Perfil Asistencia Medicación **Enfermedades** Alergias Registros

Trabajador asignado **Elena** Agregar enfermedad

ENFERMEDAD	ESTADO	VER	EDITAR	ELIMINAR
Hipertensión Arterial	Activo			

***Ilustración** - Seguimiento de las enfermedades y diagnósticos del usuario.*

CD SaaS iki@iki.com [Fichar Entrada](#) [Salir](#)

Menú

- Dashboard
- Actividades
- Usuarios
 - Listado
- Asistencia
 - Pertenencias
- Incidencias
- Baños
- Transporte
- Trabajador
- Configuración

Usuario #1

Perfil Asistencia Medicación Enfermedades **Alergias** Registros

Trabajador asignado: **Elena** [Agregar alergia](#)

ALERGIA	COMENTARIO	MEDICAMENTOS	ESTADO	EDITAR	ELIMINAR
Polen / Gramíneas	Rinitis estacional fuerte en primavera.	Ebastina 10mg	Activo	✎	🗑️

***Ilustración** - Seguimiento de alergias del usuario.*

CD SaaS iki@iki.com [Fichar Entrada](#) [Salir](#)

Menú

- Dashboard
- Actividades
- Usuarios
 - Listado
- Asistencia
 - Pertenencias
- Incidencias
- Baños
- Transporte
- Trabajador
- Configuración

Usuario #1

Perfil Asistencia Medicación Enfermedades Alergias **Registros**

FECHA	DÍA	HORARIO	¿ASISTIÓ?
30-04-2026	Lunes	09:00 - 17:00	SI
30-04-2026	Martes	09:00 - 17:00	SI

***Ilustración** - Historial de los días que el usuario ha acudido al centro.*

En todas las secciones del expediente del usuario, el botón para agregar nuevos elementos se ubica siempre en la esquina superior derecha, justo debajo de la cabecera. Al pulsarlo, se abre una ventana modal que permite completar los datos rápidamente sin tener que salir de la página actual.

Nuevo medicamento

Medicamento *
Seleccionar...

Dosis *
Ej: 500mg cada 8 horas

Fecha recepción *
dd-mm-yyyy

Fecha caducidad *
dd-mm-yyyy

Condición almacenamiento *
Seleccionar...

Vía administración *
Seleccionar...

Stock *

Cancelar Guardar

Ilustración - Ejemplo de formulario en ventana modal para el alta de registros (Caso: Medicación).

6.3.3. Control de Asistencia Diaria

Mediante esta interfaz, el trabajador puede marcar el estado de asistencia de cada usuario (Presente o Ausente) con un solo clic. Esto permite un registro histórico fiel que facilita la planificación de los recursos y el seguimiento de la actividad diaria.

Asistencia de Usuarios
Viernes — viernes, 1 de mayo de 2026

Buscar por nombre o alias...

NOMBRE COMPLETO	ALIAS	HORARIO	ESTADO	ACCIONES
Antonio Luis Martínez Soria	Toño	09:00:00 - 17:00:00	Sin marcar	Presente Ausente
Francisco José Martín Ortega	Paco	09:00:00 - 17:00:00	Sin marcar	Presente Ausente
Juan Carlos Hernández Torres	Juanca	08:00:00 - 14:00:00	Sin marcar	Presente Ausente
Luis Fernando Gómez Serrano	-	10:00:00 - 16:00:00	Sin marcar	Presente Ausente
María Carmen López Vega	Mari	10:00:00 - 18:00:00	Sin marcar	Presente Ausente
Pedro Antonio López García	Perico	08:30:00 - 14:30:00	Sin marcar	Presente Ausente
Pilar Navarro Gil	Pili	08:00:00 - 18:00:00	Sin marcar	Presente Ausente
Teresa Vidal Castillo	Tere	09:30:00 - 15:30:00	Sin marcar	Presente Ausente

Ilustración - Panel de control de asistencia de usuarios.

6.3.4. Gestión de Pertenencias (Ampliación MVP)

Este módulo permite realizar un inventario de los objetos personales que los usuarios llevan al centro, como gafas, audífonos o prendas de ropa, para evitar pérdidas o confusiones. Aunque actualmente la interfaz se encuentra en maqueta visual, el sistema ya dispone de los endpoints de backend desarrollados, dejando la lógica de inventario lista para su implementación a futuro.

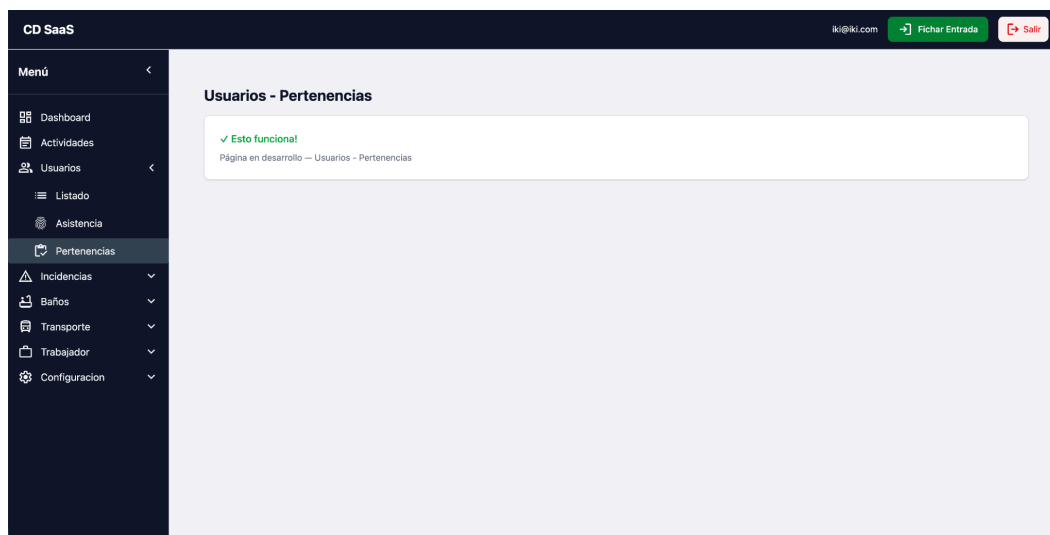


Ilustración - Módulo de control de pertenencias individuales

6.3.5. Gestión de Incidencias

El módulo de incidencias permite registrar y realizar un seguimiento de cualquier evento relevante, pudiendo categorizarse como incidencias de **centro** o de **usuario**; para ilustrar su funcionamiento, utilizaremos ejemplos basados en las de usuario. En la pantalla principal, el personal cuenta con una vista general de todos los registros y puede filtrar rápidamente entre situaciones activas e inactivas mediante pestañas. Este listado muestra el tipo de incidencia, su nivel de importancia y el estado actual del caso.

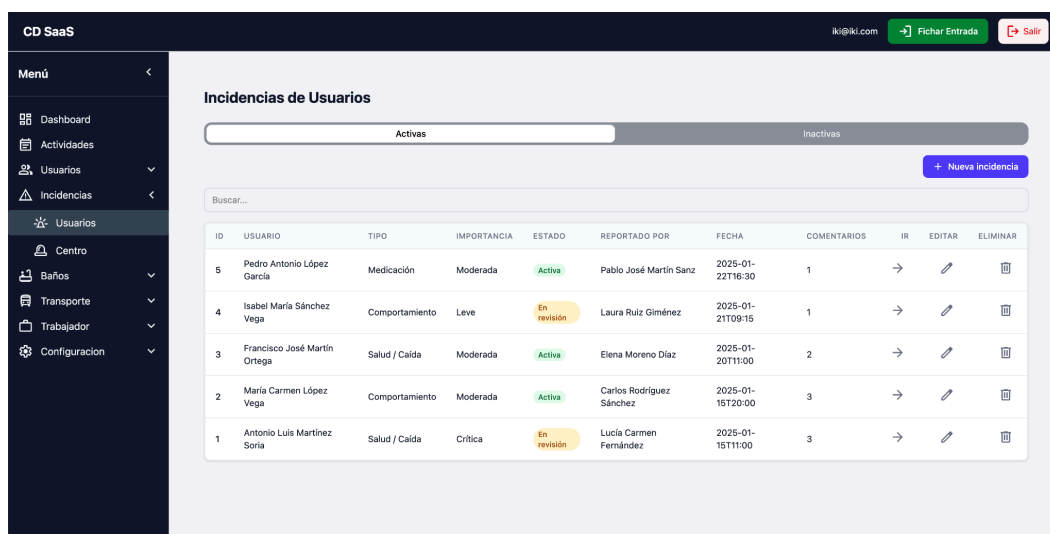


Ilustración - Panel central de gestión de incidencias de usuario.

Siguiendo la lógica de diseño de la plataforma, el reporte de una nueva situación se realiza desde el botón ubicado en la esquina superior derecha. Al pulsarlo, se abre una ventana modal que permite seleccionar al usuario afectado, categorizar el tipo de evento y describir lo sucedido antes de guardar el registro.

Ilustración - Formulario en ventana modal para el reporte de nuevas incidencias.

6.3.6. Gestión de Higiene y Turnos de Baño

Este módulo permite organizar las rutinas de aseo e higiene personal de los usuarios a lo largo de la jornada. La interfaz principal presenta un sistema de pestañas que divide el día en diferentes franjas horarias, como la ronda de llegada, la higiene post-siesta o la preparación para la salida, facilitando que el equipo sepa en todo momento qué tareas de aseo están pendientes o activas.

USUARIO	TAREA	ESTADO	EDITAR	ELIMINAR
Antonio Luis Martínez Soria	Aseo en Lavabo	Activo		
Francisco José Martín Ortega	Cambio de Pañal	Activo		
Pedro Antonio López García	Aseo en Lavabo	Activo		
Rosa María Díaz Jiménez	Corte de uñas	Activo		
Andrés Manuel Flores Romero	Cambio de ropa	Activo		

Ilustración - Listado de turnos de higiene organizado por franjas horarias.

Para asignar a un usuario a una de estas rondas, se utiliza el botón situado en la esquina superior derecha. Al hacer clic en él, aparece una ventana modal que permite buscar al usuario por nombre o DNI, seleccionar el turno correspondiente y especificar la tarea de higiene necesaria, como aseo en lavabo o cambio de absorbente. Esto asegura que cada intervención quede planificada y registrada en el sistema.

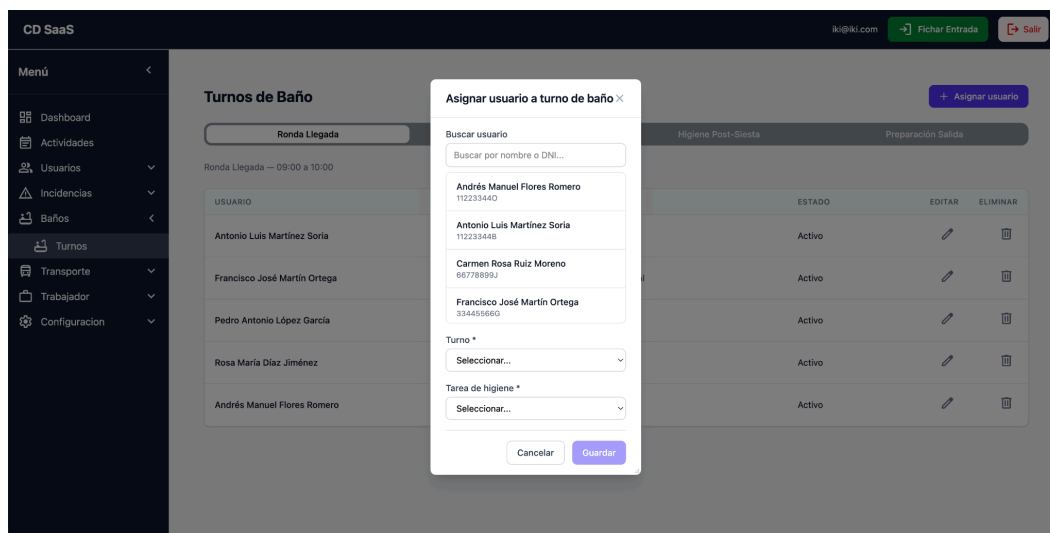


Ilustración - Formulario en ventana modal para la asignación de tareas de higiene personal.

6.3.7. Gestión de Transporte

Este módulo centraliza la logística de los traslados de los usuarios, permitiendo que la coordinación entre sus domicilios y el centro sea eficiente. En el listado de rutas, el personal puede organizar los itinerarios diarios y filtrar rápidamente entre trayectos activos e inactivos. La interfaz muestra datos como el horario, el turno, el vehículo asignado y el personal encargado (conductor y copiloto), además de controlar la ocupación mediante el número de pasajeros asignados a cada viaje. Al igual que en toda la plataforma, el alta de nuevas rutas se realiza desde el botón superior derecho mediante una ventana modal, manteniendo al usuario en la misma pantalla.

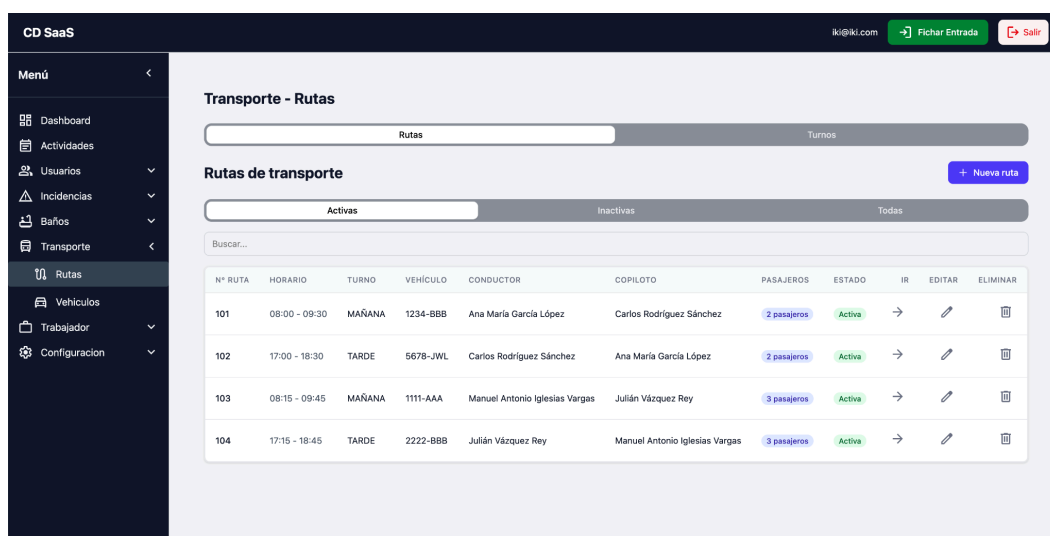


Ilustración - Panel de gestión de rutas con información operativa

Para que la asignación de estas rutas sea precisa, el módulo cuenta con una sección dedicada al control de los vehículos disponibles. En este listado se detalla no solo la matrícula y la capacidad total, sino también indicadores de accesibilidad, mostrando si el transporte está adaptado para sillas de ruedas y cuántas plazas de este tipo tiene disponibles.

MATRÍCULA	CAPACIDAD	SILLA DE RUEDAS	CAP. SILLA	ESTADO	EDITAR	ELIMINAR
1111-AAA	10	Si	1	Activo		
1234-BBB	12	No	0	Activo		
2222-BBB	6	No	0	Activo		
5678-JWL	8	Si	2	Activo		
9999-XXX	5	No	0	Activo		

Ilustración - Listado de vehículos con control de capacidad y plazas adaptadas.

6.3.8. Gestión de los Trabajadores

Este módulo centraliza la administración del personal del centro, permitiendo controlar tanto el equipo activo como la configuración individual de cada empleado. En la vista principal, el sistema muestra un directorio organizado mediante tarjetas, donde se puede consultar el nombre, los datos de contacto y, de forma visual mediante etiquetas, los roles o permisos asignados a cada profesional dentro de la plataforma. Para facilitar la navegación, la interfaz incluye un buscador superior y pestañas para filtrar entre trabajadores activos e inactivos, además de mantener el botón de "Nuevo trabajador" en la esquina superior derecha para el alta rápida.

Nombre	Email	Tel.	Horarios	Roles	Acciones
Ana María García López	ana.garcia@email.com	Tel: 600123456 / 911223344	Horarios: 5	ROLE_ADMIN, ROLE_DIRECTOR	
Beatriz Molina Pérez	bea.recep@email.com	Tel: 600777777	Horarios: 5	ROLE_COORDINADOR	
Carlos Rodríguez Sánchez	carlos.rod@email.com	Tel: 655987654	Horarios: 5	ROLE_COORDINADOR, ROLE_TAS	
Carmen Luisa Ortiz Vega	carmen.gero@email.com	Tel: 611333444	Horarios: 5	ROLE_TAS	
David Serrano Molina	david.pisco@email.com	Tel: 611222333	Horarios: 5	ROLE_PSICO	
Elena Moreno Díaz	elena.moreno@email.com	Tel: 622333444	Horarios: 5	ROLE_ENFERMERO	

Ilustración - Directorio del personal en formato tarjeta

Cada trabajador dispone de una vista de perfil. Desde la pestaña de información personal, se pueden consultar y editar los datos básicos. Esta misma estructura de pestañas, siguiendo la línea de diseño de los expedientes de usuario, permite al administrador navegar hacia otros apartados, como la gestión de los horarios individuales o el registro de fichajes, sin perder el contexto del empleado seleccionado.

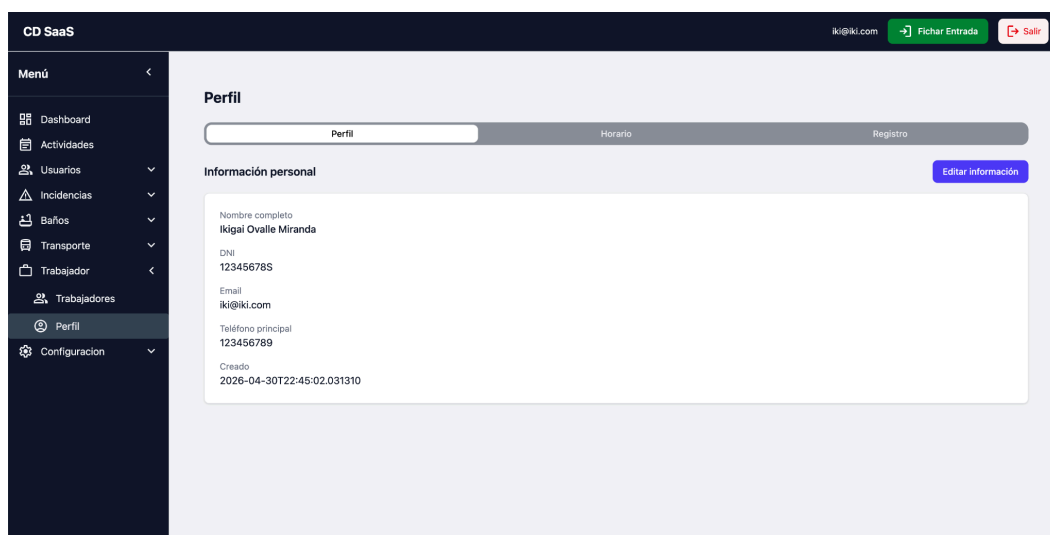


Ilustración - Vista de perfil detallada del trabajador organizada mediante pestañas.

6.3.9. Gestión de Configuraciones (Catálogos)

Este módulo permite adaptar la plataforma a las necesidades operativas de cada centro mediante la definición de datos maestros. A través de diversos catálogos, el administrador puede configurar las opciones que alimentan los formularios de toda la aplicación, asegurando que la información esté estandarizada.

A modo de ejemplo, se muestra el catálogo de salud, que permite gestionar las alergias, enfermedades y medicamentos que luego el personal asignará a los expedientes de los usuarios. La interfaz mantiene la coherencia visual del resto del sistema, presentando un listado con herramientas de edición y el botón de creación en la esquina superior derecha, el cual abre una ventana modal para añadir nuevos registros.

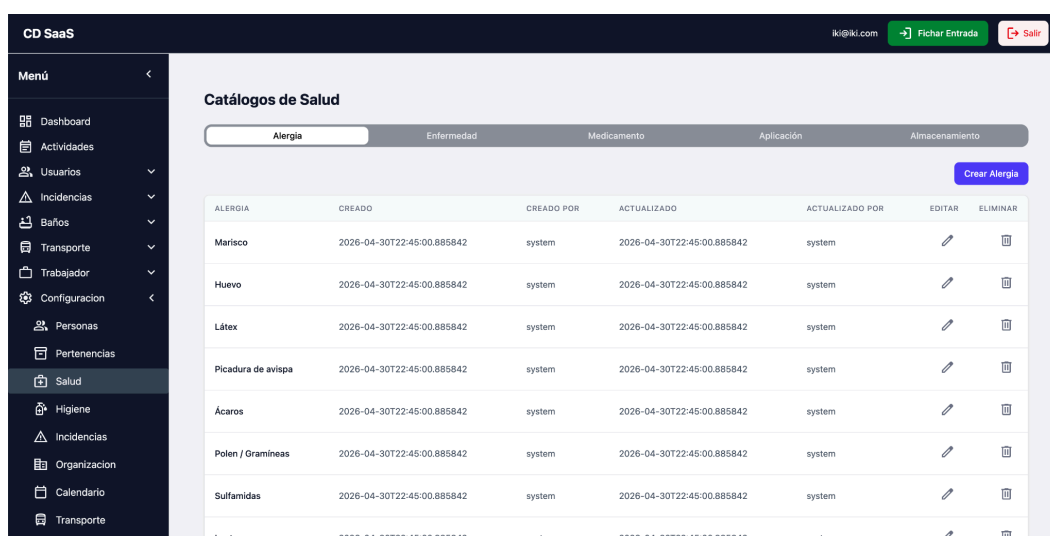


Ilustración - Interfaz de gestión de catálogos maestros (Configuración de Salud).

6.4. Portal de Administración

Este portal centraliza la gestión de toda la plataforma SaaS. Desde aquí se supervisa el estado de los centros registrados, se definen los planes de precios y se administra el acceso de los usuarios con permisos globales.

6.4.1. Gestión de Clientes

Presenta un listado de todos los centros de día que utilizan el software. Permite consultar de forma rápida datos como el nombre del centro, su identificador único, el correo del responsable y la fecha de alta para facilitar las labores de soporte técnico y administrativo.

NOMBRE	SLUG	EMAIL ADMIN	ESTADO	FECHA CREACIÓN	EDITAR
Centro Iki	centro-iki	iki@iki.com	ACTIVE	29 abr 2026	

Ilustración - Listado de centros registrados en la plataforma.

6.4.2. Planes de Suscripción

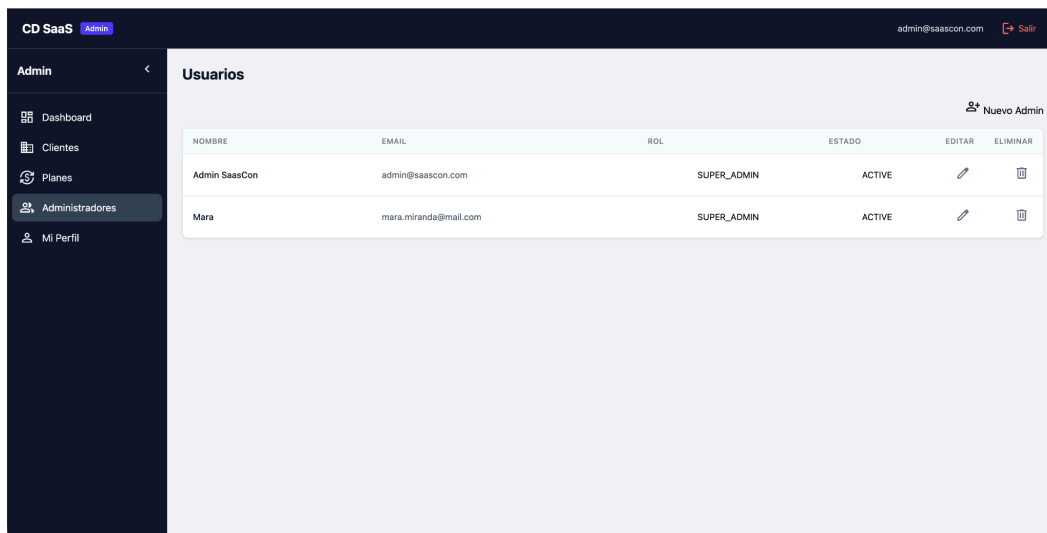
Permite configurar los niveles de servicio disponibles: Básico, Profesional y Enterprise. Cada plan detalla visualmente el precio, el límite de usuarios y los módulos específicos habilitados, facilitando el control de la escalabilidad del negocio y las capacidades técnicas de cada cliente.

Plan	Estado	Precio	Usuarios	Funcionalidades
Básico basico	ACTIVE	49,00 €/mes	Hasta 20 usuarios	- Actividades - Asistencia - Soporte Email
Profesional profesional	ACTIVE	99,00 €/mes	Hasta 40 usuarios	- Actividades - Asistencia - Incidentes - Medicación - Soporte Prioritario - Transporte
Enterprise enterprise	ACTIVE	199,00 €/mes	Hasta 100 usuarios	- Actividades - Asistencia - Incidentes - Medicación - Modulos Personalizables - Soporte 24_7 - Transporte

Ilustración - Gestión de planes y funcionalidades del modelo SaaS.

6.4.3. Administradores del Sistema

Gestiona las cuentas con permisos de super administración dentro del sistema. El alta de nuevos responsables se realiza mediante el botón superior derecho que abre una ventana modal, manteniendo la coherencia visual y funcional con el resto de la aplicación.

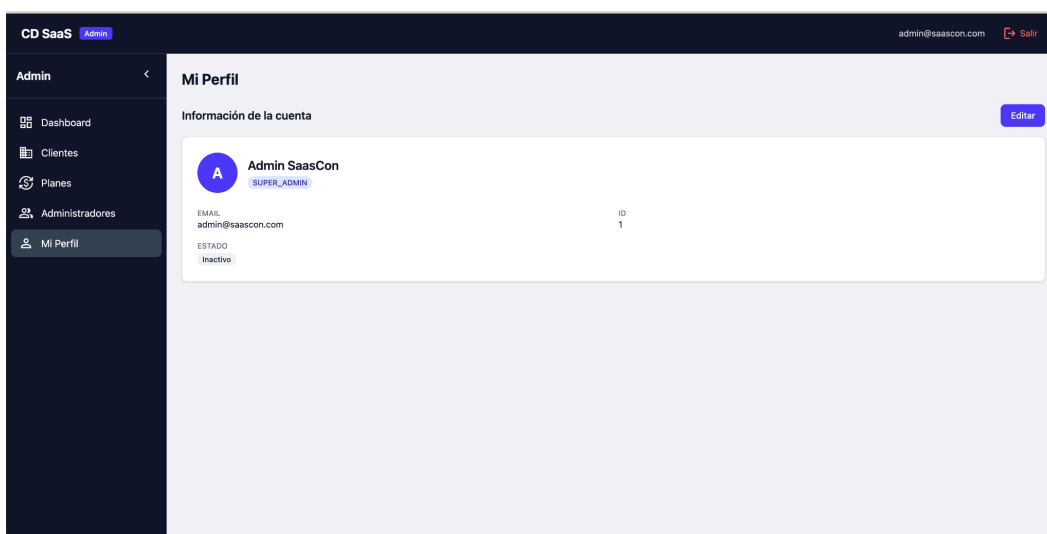


NOMBRE	EMAIL	ROL	ESTADO	EDITAR	ELIMINAR
Admin SaasCon	admin@saascon.com	SUPER_ADMIN	ACTIVE		
Mara	mara.miranda@mail.com	SUPER_ADMIN	ACTIVE		

Ilustración - Listado de usuarios con roles de administración global.

6.4.4. Administradores del Sistema

Espacio personal del administrador para consultar y editar sus datos básicos de cuenta rápidamente. Muestra información relevante como el nombre, el correo electrónico vinculado, el identificador único de sistema y el estado actual del perfil.



Mi Perfil

Información de la cuenta Editar



Admin SaasCon
SUPER_ADMIN

EMAIL
admin@saascon.com

ID
1

ESTADO
Inactive

Ilustración - Vista detallada del perfil y datos de cuenta del superadministrador.

7. CONCLUSIONES Y LÍNEAS FUTURAS

7.1. Conclusiones Generales

Tras la finalización del desarrollo de la plataforma para la gestión integral de centros de día, se pueden extraer las siguientes conclusiones alineadas con los objetivos iniciales del proyecto:

- **Especialización del software:** Se ha logrado solventar la carencia de herramientas adaptadas a los centros de día. A diferencia de las soluciones genéricas para residencias, esta plataforma elimina módulos irrelevantes (como gestión de camas o turnos nocturnos), permitiendo que el personal se centre exclusivamente en el bienestar del usuario mediante una interfaz optimizada.
- **Robustez de la arquitectura SaaS:** La implementación de una arquitectura *multi-tenant* con aislamiento lógico en **PostgreSQL** garantiza que la plataforma sea escalable y comercialmente viable, asegurando la privacidad absoluta de los datos sociosanitarios entre diferentes centros.
- **Eficiencia Técnica:** El uso de un stack moderno basado en **Java 21 con Spring Boot 4 y Angular 21** ha permitido construir una herramienta de alto rendimiento. El uso de *signals* en el frontend y una arquitectura de capas en el backend asegura un código limpio y mantenible.
- **Garantía de Integridad y Trazabilidad:** Gracias a la implementación de **Spring Auditing y Soft Delete**, el sistema garantiza que ninguna acción sea anónima y que el historial clínico y asistencial de los usuarios sea inalterable y siempre recuperable para futuras auditorías.
- **Portabilidad y Despliegue:** La contenedorización mediante **Docker y Docker Compose** ha demostrado ser fundamental para asegurar que el entorno de desarrollo sea idéntico al de producción, facilitando la instalación en cualquier infraestructura sin errores de configuración.

7.2. Líneas de Trabajo Futuro

El sistema se ha diseñado con una mentalidad modular que permite la incorporación de nuevas funcionalidades en fases posteriores del ciclo de vida del software:

- **Módulo de Facturación y Nóminas:** Implementar un sistema de generación automática de recibos para las familias según el plan contratado y la gestión de las nóminas del personal basada en los registros de jornada.
- **App Móvil Nativa:** Desarrollar aplicaciones para Android e iOS que permitan a los conductores de las rutas de transporte gestionar las recogidas en tiempo real y a las familias recibir notificaciones sobre incidencias o asistencia.
- **Integración de Sensores IoT:** Conectar el sistema con dispositivos de salud (tensiómetros, básculas inteligentes) para que los datos de las constantes vitales se vuelquen automáticamente en la ficha médica del usuario sin intervención manual.
- **Analítica de Datos y Cuadros de Mando:** Desarrollar un módulo de *Business Intelligence* para los administradores que permita visualizar estadísticas de ocupación, niveles de dependencia promedio y tendencias de incidencias para la mejora de la calidad del servicio.
- **Portal de Familias:** Crear un acceso restringido para los familiares donde puedan consultar el menú diario, las actividades en las que participa el usuario y el control de medicación, mejorando la transparencia y la comunicación externa del centro.
- **Finalización de Módulos MVP:** Completar la lógica de negocio de los módulos de "Actividades" y "Pertenencias", cuya base visual y de endpoints ya se encuentra maquetada y preparada en la versión actual.

7.3. Experiencia de Aprendizaje Individual

7.3.1. María Elena Miranda Villablanca

La culminación de este proyecto final no solo ha representado un gran desafío personal, sino que se ha convertido en una etapa fundamental de mi crecimiento y consolidación profesional. Es muy importante para mí porque en el pasado jamás pensé que me decidiría por algo como la programación. En Chile me titulé de Profesora de Educación Básica con mención en Lengua y Ciencias Sociales y también obtuve mi licenciatura en Educación. Si bien, es algo que amé hacer y que tiene un espacio importante en mi vida, lo cierto es que aún conservo ese pensamiento docente: organizada, estructurada y metódica, que me permite leer a las personas al observar, tal cual lo hacía con los niños que eduqué durante mis 8 años de carrera. Hoy me estoy convirtiendo en programadora, lo que integra una parte mucho más lógica de lo que acostumbraba y me ha costado mucho esfuerzo y sacrificio, pero que sin duda ha sido posible gracias al apoyo constante que James me ha dado en todo momento. Él estuvo conmigo demostrándome que sí podía hacerlo cuando las dudas aparecían, desde enseñarme cuando no entendía algo hasta motivarme cuando pensaba que no podía, ahí estaba conmigo demostrando que sí era capaz. Y aquí estoy, a punto de terminar este proceso, aplicando lo aprendido durante estos dos años y demostrando que independiente de todo, somos capaces de lograr muchas cosas si somos constantes, pacientes y metódicos.

Particularmente, este proceso de aprendizaje comenzó con una necesidad real del Centro de Día El Bosque, lo cual me llena de orgullo porque el proyecto está pensado para solucionar un problema real y con sentido, pues no se trata de hacer un inventario y CRUD de todo lo necesario, sino que es una solución a una problemática de la vida cotidiana. En cuanto a mi trabajo, me centré en la arquitectura y construcción de la API, una labor que desempeñé mano a mano con James, investigando constantemente para aplicar los estándares más exigentes basados en las buenas prácticas. Un hito importante en nuestra evolución como desarrolladores fue el rediseño de las validaciones, donde decidimos trasladarlas de los modelos a los **DTOs** e implementar **MapStruct** para la gestión de *mappers*. Este proceso fue ensayo y error, donde nos apoyamos en herramientas como **NotebookLM** y documentación técnica de **Springboot** hasta lograr que el código fuera eficiente y cumpliera con los estándares que queríamos. Hicimos pruebas en un proyecto paralelo para seguir aprendiendo y llevar este conocimiento a la práctica en este **SaaS para Centros de Día**. Todo esto me permitió interiorizar la importancia de una arquitectura desacoplada y escalable, donde la creación de los **57 modelos**, la creación de la **lógica de negocio**, los **endpoints** y el testeo mediante una colección de **Postman** me brindaron la seguridad de entregar un *backend* fiable y diseñado con precisión para las necesidades que James requería en la capa de Angular.

A pesar de que me encargué en gran parte del backend, la comunicación con James fue constante para asegurar que los servicios se integraran perfectamente en la interfaz y según las necesidades del proceso de desarrollo, construyendo los endpoints a la medida que él necesitara, ajustándome a sus requerimientos del frontend. Esta experiencia me motiva y me deja como desafío para el futuro, seguir explorando el *frontend* para alcanzar un perfil más completo en el futuro y ser una mejor desarrolladora. En este sentido, trabajar con James ha sido un privilegio, ya que independientemente de nuestro vínculo, su calidad como desarrollador, su capacidad de liderazgo y su compromiso por realizar un trabajo ordenado y limpio, lo posiciona como un profesional con un potencial extraordinario que, sin duda, destacará en el sector donde quiera que vaya. Por otro lado, la colaboración con Juan José en las áreas de multitenant y seguridad requirió un desarrollo más individualizado debido a la naturaleza técnica de su parte. En este caso, él trabajó de manera más independiente, lo que requirió investigación y pruebas en otro proyecto antes de integrarlo a este, pero sin duda ha realizado un trabajo estupendo y ha asumido un gran desafío, ya que

tuvo que aprender los conocimientos necesarios de multitenant, lo cual es algo valioso para un estudiante que está por terminar esta etapa de formación. Sin duda será un gran desarrollador y tendrá éxito.

Finalmente, es importante mencionar que este cierre de etapa puso a prueba mi resiliencia frente a imprevistos que afectaron significativamente nuestro flujo de trabajo como equipo. Recibir el aviso inesperado de que la fecha de entrega se adelantaba al 29 de abril, fue un giro inesperado que nos obligó a reaccionar bajo una presión innecesaria derivada de una gestión de tiempos ajena a nuestra planificación inicial. Sin duda quiero señalar que nuestro tutor, José Manuel Rayo, siempre nos apoyó, por lo tanto entiendo que no es responsabilidad de él, sino de organización del centro. Como persona autoexigente consigo misma, esta precipitación en las fechas me resultó bastante frustrante y estresante, pues mi compromiso era entregar un producto de una potencia aún mayores de lo que ya es. Esto me costó tranquilidad y creó momentos de tensión con mis compañeros ante la presión de la entrega y los detalles para dejar la aplicación operativa, los cuales fueron resueltos como adultos que somos. Sin embargo, frente a esta falta de organización y a las dificultades personales enfrentadas en los últimos meses, el haber alcanzado una aplicación de este nivel es testimonio de nuestra capacidad de entrega, compromiso y de que somos personas que pensamos en grande. Nos gusta cumplir con lo solicitado y más no por la necesidad de brillar especialmente frente a otros, sino como desafío personal para ser mejores desarrolladores. Concluyo este ciclo con la satisfacción de haber mantenido mis estándares, demostrando que incluso ante plazos ajustados y disparidad de criterios en el equipo, es posible entregar un software de excelencia y mantener la fortaleza incluso en momentos difíciles.

7.3.2. James Alexander Ovalle Barrera

La finalización de este **Trabajo de Fin de Grado** representa para mí un hito en mi transición hacia el sector tecnológico. Mi formación previa como ingeniero constructor me ha permitido aportar una visión estratégica y una metodología de planificación rigurosa, capacidades que han sido determinantes para gestionar los flujos de trabajo y establecer una arquitectura escalable, tanto en el **frontend** como en el **backend**.

El desarrollo de esta plataforma ha sido un reto de aprendizaje intensivo. Dado que el programa académico ofrecía una base introductoria en **Angular** y **Spring Boot**, la necesidad de profundizar en estas tecnologías fue evidente. Junto a mi compañera *María Elena*, implementamos una metodología de estudio basada en herramientas de inteligencia artificial y documentación oficial para contrastar versiones y adoptar prácticas modernas de desarrollo. Este proceso de investigación autónoma me permitió comprender la transición hacia arquitecturas basadas en *signals* en **Angular** y el desarrollo de servicios robustos en **Java**, asegurando que el código no solo fuera funcional, sino también eficiente y fácil de mantener.

Mi participación activa se centró en la organización de las estrategias de trabajo y, especialmente, en el desarrollo del **frontend**. Mi objetivo principal fue materializar una experiencia de usuario intuitiva, implementando componentes reutilizables y aprovechando las nuevas estructuras de control de **Angular**. Me genera una gran satisfacción personal el haber cumplido con los objetivos del proyecto a pesar de la reducción imprevista del tiempo de entrega. Esta situación de estrés puso a prueba mi capacidad de resiliencia y gestión bajo presión, reafirmando mi compromiso con la entrega de resultados de calidad en entornos exigentes.

Lo que otorga un valor diferencial a este proyecto es su origen en una necesidad real que pude presenciar durante mi etapa laboral en el **Centro de Día El Bosque**. Esta experiencia previa me permitió diseñar funcionalidades que realmente resuelven problemas operativos diarios. La recepción positiva por

parte de profesionales del sector, como *Belén Barros (propietaria de CD el Bosque)*, valida que nuestra solución supone un salto cualitativo respecto a los sistemas actuales. Estoy convencido de que este software tiene un potencial real de comercialización y éxito en el mercado sociosanitario.

A nivel humano, la coordinación con *María Elena Miranda* ha sido excepcional. Nuestra comunicación fluida y el respeto mutuo por las responsabilidades asignadas permitieron un avance constante y ordenado. Su capacidad para entender los mecanismos internos del **software** ha sido un apoyo fundamental para el éxito del proyecto. Por otro lado, la arquitectura del sistema se vio complementada por el trabajo técnico de *Juan José Restrepo* en el área de **multi-tenancy**. Su labor, centrada en aspectos muy específicos y de alta complejidad técnica como el manejo de hilos (**threads**), requirió un enfoque de trabajo autónomo e independiente, cuya integración final permite que la plataforma soporte de manera segura a múltiples clientes de forma simultánea.

En conclusión, este proyecto no solo ha consolidado mis conocimientos técnicos en el stack de desarrollo, sino que ha reforzado mi convicción de que la tecnología, cuando se planifica con una base de conocimiento sectorial sólida, es la herramienta más potente para mejorar la calidad de vida de las personas.

7.3.3. Juan José Restrepo Arango

Mi participación en este proyecto se centró en uno de los componentes técnicamente más exigentes: la arquitectura multitenant y la seguridad del sistema. Profundizar en el aislamiento por esquemas en PostgreSQL, comprender cómo Spring Security gestiona tokens JWT dentro de un entorno multitenant, y entender cómo la reutilización de hilos puede comprometer la integridad de los datos si no se controla correctamente, fueron aprendizajes que no estaban presentes en ningún apunte del programa académico. Tuve que construir este conocimiento desde la práctica: equivocándome, investigando, y dedicando muchas horas a analizar logs y stack traces que al inicio parecían incomprensibles, pero que con el tiempo se convirtieron en una fuente clara de diagnóstico y aprendizaje.

A nivel personal y humano, este proyecto también reforzó mi visión sobre el valor del trabajo colaborativo entre perfiles distintos. Trabajar junto a **María Elena**, cuya rigurosidad y altos estándares técnicos elevaron la calidad del desarrollo, y junto a **James**, cuya visión de producto y liderazgo técnico en el frontend fueron claves para convertir el sistema en una aplicación coherente y funcional, fue una experiencia que fortaleció mi crecimiento profesional. Cada integrante aportó desde sus fortalezas, y esa combinación se refleja directamente en el resultado final.

En conclusión este proyecto no solo me permitió adquirir habilidades técnicas avanzadas, sino también confirmar que el desarrollo de software de calidad nace tanto del conocimiento como de la colaboración y la disciplina en equipo.

8. REFERENCIAS BIBLIOGRÁFICAS

- **Angular.** (s.f.). *Angular Documentation: The modern web developer's platform*. Recuperado de <https://angular.dev/>
- **Java SE 21.** (s.f.). *Java Platform Standard Edition 21 Documentation*. Recuperado de <https://docs.oracle.com/en/java/javase/21/>
- **PostgreSQL.** (s.f.). *PostgreSQL Documentation: The World's Most Advanced Open Source Relational Database*. Recuperado de: <https://www.postgresql.org/docs/>
- **Docker.** (s.f.). *Docker Documentation: Containerizing applications for development and deployment*. Recuperado de <https://docs.docker.com/>
- **Project Lombok.** (s.f.). *Lombok Features: Boilerplate reduction for Java*. Recuperado de <https://projectlombok.org/features/all>
- **Spring Boot.** (s.f.). *Spring Boot Reference Documentation*. Recuperado de <https://docs.spring.io/spring-boot/index.html>
- **MapStruct.** (s.f.). *MapStruct Reference Guide: Java bean mappings, the easy way*. Recuperado de <https://mapstruct.org/documentation/reference-guide/>
- **TypeScript.** (s.f.). *TypeScript Documentation: JavaScript with syntax for types*. Recuperado de <https://www.typescriptlang.org/docs/>
- **Baeldung.** (s.f.). *Java Bean Validation Basics*. Baeldung. Recuperado de: <https://www.baeldung.com/java-validation>
- **Coppola, G.** (2025, 27 de octubre). *Spring Boot and Validation: A Complete Guide with @Valid and @Validated*. DEV Community. Recuperado de: <https://dev.to/gianfocp98/spring-boot-and-validation-a-complete-guide-with-valid-and-validated-471p>
- **Dhandala, N.** (2025, 22 de diciembre). *How to Handle Validation Errors in Spring Boot*. OneUptime. Recuperado de: <https://oneuptime.com/blog/post/2025-12-22-spring-boot-validation-errors/view>
- **JavaTechOnline.** (2025, 22 de enero). *Java Records Vs JPA Entities And Lombok: A Comprehensive Comparison*. JavaTechOnline. Recuperado de: <https://javatechonline.com/java-records-vs-jpa-entities-and-lombok/>
- **Seddiki, A.** (s.f.). *Mastering DTOs: Best Practices and Pitfalls to Avoid*. Medium. Recuperado de: <https://medium.com/@ayoubseiddiki132/mastering-dtos-best-practices-and-pitfalls-to-avoid-fda9b3db1eb9>
- **Zichil, F.** (2024, 3 de octubre). *Entity to DTO: A Performance Comparison of Three Mapping Approaches*. Level Up Coding. Recuperado de: <https://levelup.gitconnected.com/entity-to-dto-a-performance-comparison-of-three-mapping-approaches-ed3f81b7179c>
- **Mamidi, S.** (2026, 2 de febrero). *Building Secure REST APIs in Spring Boot: Techniques and Tools*. International Journal of Emerging Trends in Computer Science and Information Technology, 7(1), 87-91. Recuperado de: <https://doi.org/10.63282/3050-9246.IJETCSIT-V7I1P111>
- **Pasuparthi, S.** (2025, 18 de abril). *Spring security integration with spring boot*. World Journal of Advanced Engineering Technology and Sciences, 15(1), 1563-1568. Recuperado de: <https://doi.org/10.30574/wjaets.2025.15.1.0380>
- **Zahir, R. M., Esenalieva, G., & Kaldybaev, S.** (2025, 13 de enero). *Authentication Approaches in Spring Security*. Preprints.org. Recuperado de: <https://doi.org/10.20944/preprints202501.0899.v1>
- **Katta, V. K.** (2025, 13 de mayo). *Building Robust REST APIs with Spring Boot: A Practical Guide*. World Journal of Advanced Engineering Technology and Sciences, 15(2), 1748-1758. Recuperado de: <https://doi.org/10.30574/wjaets.2025.15.2.0723>

- **Kuehlers, J., & Thompson, M.** (2025, 19 de noviembre). *Announcing Angular v21*. Angular Blog. Recuperado de: <https://blog.angular.dev/announcing-angular-v21-57946c34f14b>
- **Prasad, J.** (2026, 11 de enero). *What is New in Angular 21: Key Features, Benefits, and Migration Guide*. Radixweb. Recuperado de: <https://radixweb.com/blog/what-is-new-in-angular-21>
- **Patel, P.** (2026, 21 de enero). *Angular 21 Release Guide: Why the Latest Angular Version Matters for Developers*. CMARIX. Recuperado de: <https://www.cmarix.com/blog/latest-angular-version/>
- **Steyer, M.** (2025, 1 de diciembre). *What's new in Angular 21?*. ANGULARarchitects. Recuperado de: <https://www.angulararchitects.io/blog/whats-new-in-angular-21-signal-forms-zone-less-vitest-angular-aria-cli-with-mcp-server/>
- **Degni, R.** (s.f.). *Inside Angular 21: A New Reactive Core for the Modern Web*. devmio. Recuperado de: <https://devm.io/angular/inside-angular-21-a-new-reactive-core-for-the-modern-web>
- **Postman.** (s.f.). *Postman Learning Center: API testing, documentation, and collaboration*. Recuperado de <https://learning.postman.com/>
- **Google.** (s.f.). *NotebookLM: AI-powered research and writing assistant*. Recuperado de <https://notebooklm.google/>
- **Mermaid.** (s.f.). *Mermaid.js: Diagramming and charting tool for documentation*. Recuperado de <https://mermaid.js.org/intro/>